

Everything About Arrays

A print handnote for programmers who are new to this topic.

177 topics · C++17/20, with PHP contrasts

Runnable code, memory diagrams, a trap and a cost strip on every topic

10 practice sets with fully worked answer keys · 17 revision cards

Master cheatsheet, pattern-recognition table, spaced-repetition tracker

Contents

FRONT MATTER

How to use this book	9
----------------------------	---

PART I — FOUNDATIONS AND MEMORY

Section 1 • Foundation	13
1. Introduction to Arrays	13
2. Why Arrays Matter	13
3. History of Arrays	14
4. Core Properties	15
5. Array Terminology	16
Section 2 • Memory & Internal Working	17
6. Internal Working	17
7. Memory Model & Layout	17
8. Address Calculation Formula	18
9. Row-Major vs Column-Major	19
Practice set • Answer key • Summary card	21
PART I CHEATSHEET — The Mental Model	26

PART II — ARRAYS IN C++

Section 3 • C++ Basics	29
10. C++ Array Syntax	29
11. Array Initialization Methods	29
12. Input and Output	30
13. Traversal Techniques	31
14. Types of Arrays	32
15. One-Dimensional Arrays	32
Section 4 • Pointers, Functions & References	35
16. Arrays vs Pointers	35
17. Array Decay	35
18. Passing Arrays to Functions	36
19. Returning Arrays	37
20. const Arrays	37
21. Array of Pointers	38
22. Pointer to Array	38
23. References to Arrays	39
Practice set • Answer key • Summary card	40
Section 5 • Dynamic Memory	45
24. Dynamic Arrays	45
25. Create Dynamic Array	45
26. Resize Dynamic Array	46
27. Delete Dynamic Array	47
28. Dynamic Objects	47
29. RAII and Smart Pointers	48

Section 6 • Modern C++ Array Tools	49
30. std::array	49
31. std::vector	49
32. std::span	50
33. std::valarray	51
34. C-Style Arrays vs Modern Containers	51
Section 7 • Strings & Character Arrays	53
35. Character Arrays	53
36. C-Style Strings	53
37. Arrays of Strings	54
Practice set • Answer key • Summary card	55
PART II CHEATSHEET — C++ Array Toolkit	60
 PART III — OPERATIONS, PATTERNS & SORTING	
Section 8 • Basic Array Operations	63
38. Basic Array Operations	63
39. Insert at Any Index	63
40. Delete from Any Index	64
41. Linear Search	64
42. Binary Search	65
43. Update Element	65
44. Reverse an Array	66
45. Merge Two Arrays	66
46. Merge Two Sorted Arrays	67
47. Check if Array is Sorted	67
48. Find Min and Max	68
49. Count Frequency	68
50. Remove Duplicates	69
51. Move Zeros	69
Practice set • Answer key • Summary card	71
Section 9 • Rotation & Rearrangement	77
52. Array Rotation	77
53. Left Rotation	77
54. Right Rotation	77
55. Rotation by K	78
56. Rotation Using Mod	78
57. Rotation Using Reverse	78
58. Partitioning Arrays	79
59. Dutch National Flag	80
Section 10 • Array Problem-solving Patterns	81
60. Prefix Sum	81
61. Suffix Sum	81
62. Difference Array	82
63. Kadane's Algorithm	82
64. Sliding Window	83
65. Two Pointers	84

66. Fast and Slow Pointers	84
67. Majority Element	85
68. Missing Number	85
69. Duplicate Number	86
70. Subarray Concepts	87
71. Subsequence Concepts	87
72. Permutations	87
73. Next Permutation	88
Practice set · Answer key · Summary card	89
Section 11 · Sorting	95
74. Sorting Arrays	95
75. Bubble Sort	95
76. Selection Sort	96
77. Insertion Sort	96
78. Merge Sort	97
79. Quick Sort	97
80. Heap Sort	98
81. Counting Sort	99
82. Radix Sort	99
83. Bucket Sort	100
84. C++ Built-in Sorting	100
85. Custom Comparator	101
86. Stable vs Unstable Sorting	101
87. Sorting Best Practices	102
88. PHP Array Functions	102
89. C++ vs PHP Arrays	103
Practice set · Answer key · Summary card	104
PART III CHEATSHEET — Operations & Patterns	109
 PART IV — TWO DIMENSIONS	
Section 12 · 2d Arrays & Matrix	113
90. Multi-Dimensional Arrays	113
91. Jagged Arrays Concept	113
92. 2D Matrix Basics	114
93. Declaring 2D Matrix	115
94. Initializing 2D Matrix	115
95. Taking Matrix Input	116
96. Printing Matrix	116
97. Accessing Rows	116
98. Accessing Columns	117
99. Diagonal Values	117
100. Matrix Addition	118
101. Matrix Subtraction	118
102. Matrix Multiplication	118
103. Transpose Matrix	120
104. Rotate Matrix	120
105. Spiral Traversal	121
106. Search in Matrix	121

107. Mirror Matrix	122
108. Boundary Traversal	123
109. 8 Neighbors	123
110. Flood Fill	124
111. Prefix Sum Matrix	125
112. Sparse Matrix	125
Practice set · Answer key · Summary card	127
PART IV CHEATSHEET — Grids in One Page	132

PART V — COST, CACHE AND CORRECTNESS

Section 13 · Complexity & Performance	135
113. Time Complexity	135
114. Space Complexity	135
115. Amortized Complexity	136
116. Performance & Cache Behavior	137
117. CPU Cache Lines	137
118. Spatial Locality	138
119. Temporal Locality	138
120. False Sharing	139
121. Alignment and Padding	140
122. SIMD and Vectorization	140
Section 14 · Bugs, Safety & Best Practices	143
123. Bounds Checking	143
124. Undefined Behavior	143
125. Common Bugs	144
126. Memory Leaks	144
127. Buffer Overflow	145
128. Dangling Pointer	145
129. Uninitialized Values	146
130. Integer Overflow in Indexing	146
131. Best Practices	147
132. Prefer vector / array	147
133. Use size_t Carefully	148
134. Use .at() When Needed	148
135. Avoid Magic Numbers	149
136. Prefer Algorithms Library	149
Practice set · Answer key · Summary card	150
PART V CHEATSHEET — Diagnosis in One Page	155

PART VI — WHERE ARRAYS LIVE, AND WHAT ELSE THERE IS

Section 15 · Real-world Uses	159
137. Real-World Use Cases	159
138. Buffers	159
139. Lookup Tables	160
140. Image Processing	160
141. Game Development	161
142. Databases	161
143. Operating Systems	162

144. Embedded Systems	162
145. Machine Learning	163
146. Graphics Programming	163
147. Networking	164
Section 16 • Comparisons WITH Other Structures	165
148. Arrays vs Linked List	165
149. Arrays vs Stack	165
150. Arrays vs Queue	166
151. Arrays vs Hash Table	166
152. Arrays vs Tree	167
153. Arrays vs Heap	167
Practice set • Answer key • Summary card	169
PART VI CHEATSHEET — Choosing the Structure	174
PART VII — ADVANCED C++ AND THE INTERVIEW	
Section 17 • Advanced C++ & System-level Arrays	177
154. Arrays in STL Algorithms	177
155. Iterators	177
156. Range Library	178
157. Templates with Arrays	178
158. Compile-Time Arrays	179
159. Bit Arrays	179
160. std::bitset	180
161. vector<bool>	180
162. Memory-Mapped Arrays	180
163. Parallel Array Processing	181
164. GPU Arrays	181
165. External Memory Arrays	182
166. Lesser-Known Facts	182
167. Unique Uses	183
Section 18 • Interview, Dsa & Cp	185
168. Interview Notes	185
169. Beginner Problems	185
170. Intermediate Problems	186
171. Advanced Problems	186
172. DSA Roadmap	187
173. Competitive Programming	188
174. Practice Roadmap	188
175. Mini Projects	189
176. Final Cheatsheet	190
Section 19 • Final	191
177. Summary & Conclusion	191
Practice set • Answer key • Summary card	192
PART VII CHEATSHEET — Beyond Ordinary Memory	197
END MATTER — MASTER CHEATSHEET AND TRACKER	
Master cheatsheet 1 — the model and the memory	200

Master cheatsheet 2 — the operations	201
Master cheatsheet 3 — traps and diagnosis	202
Master cheatsheet 4 — pattern recognition	203
Revision tracker — Day 0 / +1 / +7 / +30 / +90	204
Afterword	206

HOW TO USE THIS BOOK

This is the second book of Phase 1. It covers all 177 items of your arrays checklist, in your numbering: topic 42 in this book is item 42 on your list, so you can tick the original as you go.

The shape of a topic

Every topic follows the same skeleton, so you learn where to look rather than reading linearly: **what it is** (mechanism first, not a label), **why it exists**, **runnable C++**, an **ASCII memory or step diagram**, one **key insight**, one **trap** naming the symptom you will actually see, and a **cost strip** giving time, space and the constant factor that dominates in practice. The cost strip is the line to memorise.

The toolchain assumed

```
g++ -std=c++20 -O2 -Wall -Wextra prog.cpp -o prog      # normal build
g++ -std=c++20 -g -fsanitize=address,undefined prog.cpp -o prog
                                                         # for every Group C task
g++ -std=c++20 -O2 -fopt-info-vec prog.cpp             # did it vectorize?
perf stat -e cache-misses,instructions ./prog          # for Topics 116-122
```

Group C of every practice set is *build it, break it, observe the failure, fix it*. Those tasks are the point of the book. An out-of-bounds write that you have watched AddressSanitizer catch, with the byte offset printed, is worth more than ten pages about bounds checking. Do not skip them because the code "obviously" works — the whole lesson is that it obviously does not.

Where the numbers come from

Every measurement quoted in this book was taken on the machine that built it, with GCC at **-O2**: 5.6 ms versus 1346.6 ms for the same 10^7 integers, 7.8 ms versus 172.1 ms for the same 64 MB matrix, 21 reallocations to reach 10^6 appends. Your hardware will give different figures. Re-run them — a number you measured is an argument, and "cache-friendly" is not.

The schedule

Read one Part, do its practice set the same day, and put the date in the tracker at the back of the book. Revisit with the summary card only — never the chapter — at +1 day, +7, +30 and +90. Seven Parts at one Part per day is a one-week pass; the reviews are what make it stick.

THE ONE IDEA IN THIS BOOK

An array is not a container of values. It is a *rule for computing an address*: **base + index × sizeof(T)**. Every strength (O(1) access, cache friendliness, pointer arithmetic, matrices, hash tables, heaps) and every weakness (fixed size, O(n) insert, no bounds check) follows from that one line of arithmetic. When something about arrays surprises you, return to it.

PART I

Foundations and Memory

What an array actually is, why the address formula makes every other property inevitable, where the bytes live, and why a 2-D array is a polite fiction. Checklist items 1-9.

SECTION 1 — FOUNDATION

1. Introduction to Arrays

An array is a block of memory holding n objects of identical type laid end to end, with no gaps and no bookkeeping between them. That single sentence is the whole data structure. Everything else — indexing, iteration, why insertion is slow, why a linear scan of ten million ints beats a linked list by a factor of ten — follows from it mechanically.

The consequence that matters: because the elements are identical in size and adjacent in memory, the machine does not have to *search* for element i . It computes the address arithmetically and issues one load. A linked list is *told* where the next element is and must read a node to find out; an array *computes* where any element is without reading anything. That is the difference between $O(1)$ and $O(n)$ random access, and it is the reason arrays underpin nearly every other structure in this book (Topics 149–153).

```
#include <iostream>

int main() {
    int a[5] = {10, 20, 30, 40, 50};

    std::cout << a[3] << "\n";           // 40  - one load, no search
    std::cout << sizeof(a) << "\n";       // 20  = 5 * sizeof(int)
    std::cout << &a[0] << " " << &a[1];   // addresses differ by exactly 4
}
```

index	0	1	2	3	4
a =	10	20	30	40	50
addr	0x1000	0x1004	0x1008	0x100C	0x1010
	_____ 20 bytes _____/				
a[3] is not looked up: 0x1000 + 3*4 = 0x100C					

THE KEY INSIGHT

Contiguity is not a detail of the implementation — it *is* the data structure. Every array property in this book is contiguity plus a multiplication.

THE TRAP

C++ never checks that your index is in range. `a[7]` on a 5-element array compiles silently, reads whatever bytes sit past the end, and often *prints a plausible-looking number*. The symptom is not a crash; it is a value that is wrong only sometimes, on some machines, after some other change. See Topic 123.

array – access $O(1)$ · search $O(n)$ · insert/delete at middle $O(n)$ · space $n \cdot \text{sizeof}(T)$, zero per-element overhead

2. Why Arrays Matter

Arrays matter for three distinct reasons, and interviewers, systems people and competitive programmers each care about a different one.

1. They are the default building block. `std::vector`, `std::string`, hash table buckets, binary heaps (Topic 153), the adjacency structure of a graph, the frame buffer on your screen, the pages of a database table (Topic 142) — all of them are arrays with a policy bolted on. Learning arrays is not learning one structure out of twenty; it is learning the substrate of the other nineteen.

2. The hardware is built for them. A modern CPU fetches memory in 64-byte cache lines and runs a hardware prefetcher that recognises constant strides. Scanning an `int` array touches DRAM once per 16 elements and the prefetcher hides even that. A pointer-chasing structure of the same length can pay a cache miss *per element* — roughly 80–100 ns versus about 1 ns amortised. That is the 5–20× gap you will measure yourself in Practice Set 8, Task C3.

3. Interviews are 40% array problems. Two pointers, sliding window, prefix sums, Kadane, Dutch national flag, binary search — Topics 60–73 are the pattern vocabulary that shows up in the majority of screens. They are all "do it in one pass with $O(1)$ extra space on a contiguous block."

```
// The same 10M-element sum, two layouts. Compile: g++ -O2
// array (contiguous):   ~4 ms      1 miss per 16 elements
// list (heap nodes):   ~45 ms     up to 1 miss per element
long sumArray(const int* a, int n) {
    long s = 0;
    for (int i = 0; i < n; ++i) s += a[i];    // stride 4, prefetched
    return s;
}
```

THE KEY INSIGHT

An array's advantage is not its Big-O — a linked list also scans in $O(n)$. The advantage is the *constant*, and the constant comes from the cache (Topics 116–119).

why it matters — substrate of most containers · 1 DRAM touch per 16 ints · ~40% of coding-interview problems

3. History of Arrays

The array is older than the pointer. FORTRAN (1957) had subscripted variables because scientific code is matrix code; its arrays were 1-based and column-major (Topic 9), a choice that still ripples through BLAS, LAPACK, MATLAB, R and Julia today. ALGOL 60 introduced arrays with computed bounds; BCPL and then C (1972) collapsed the array into a pointer plus arithmetic, which is why C arrays decay (Topic 17) and carry no length.

C++ inherited the C array and then spent thirty years apologising for it: `std::vector` (1994, STL) added ownership and growth, `std::array` (C++11) put the length back into the type, `std::span` (C++20) gave a non-owning pointer+length pair a name, and `std::mdspan` (C++23) does the same for multidimensional views. The direction of travel is one-way: *keep contiguity, attach the length*.

Year	Language / feature	What it fixed
1957	FORTRAN subscripts	Matrix maths in source form; column-major
1960	ALGOL 60	Dynamic bounds, nested scopes
1972	C []	Array = base pointer + arithmetic; no length
1994	STL <code>vector</code>	Ownership, growth, RAII
2011	<code>std::array</code>	Length restored to the type; no decay
2020	<code>std::span</code>	Pointer+length passed as one object

THE KEY INSIGHT

Every C++ array container is the same bytes as a C array. The evolution added *metadata and rules*, never a different memory layout — which is why `vector` and `span` can hand a raw pointer to a C API for free.

history – layout unchanged since 1972 · what changed: length, ownership, bounds discipline

4. Core Properties

Four properties define the structure. Any language feature that keeps all four is an array; drop one and you have something else with different costs.

Property	What it buys	What it costs
Homogeneous type	Element size is a compile-time constant, so the address multiply is a shift	Cannot mix types without a variant or union
Contiguous storage	Cache lines and SIMD (Topic 122) work; pointer arithmetic is legal	Needs one unbroken block — a 100 MB array can fail while 100 MB of free RAM exists (fragmentation)
Fixed size	No capacity field, no reallocation, no indirection	Growth means allocate + copy (Topic 26)
Zero-based index	<code>base + i*size</code> , no subtraction	Off-by-one is the most common bug in the language (Topic 125)

THE FOUR PROPERTIES AND WHAT BREAKS WITHOUT EACH
homogeneous ---> element size known ---> 0(1) address maths
contiguous ---> neighbours prefetched -> fast scan, SIMD
fixed size ---> no capacity field -----> zero overhead per array
0-based ---> base + i*size -----> one multiply-add

THE TRAP

"Fixed size" means fixed at the point of *declaration*, not fixed at compile time. `int a[n];` with a runtime `n` is a VLA — a C99 feature that GCC and Clang accept in C++ as an extension and MSVC rejects. It compiles on your laptop and fails in CI. Use `std::vector` (Topic 31).

properties – 4 invariants · violate contiguity and every cost in this book changes

5. Array Terminology

Precise vocabulary prevents a class of bugs on its own, because most array bugs are confusions between two of these words.

Term	Meaning	In code
Element	One stored object	<code>a[2]</code>
Index / subscript	Distance from the start, in elements, 0-based	the <code>2</code>
Base address	Address of element 0	<code>a</code> or <code>&a[0]</code>
Length / size	Number of elements currently valid	<code>v.size()</code>
Capacity	Elements that fit before reallocation; $\geq$ size	<code>v.capacity()</code>
Stride	Bytes between consecutive elements = <code>sizeof(T)</code>	4 for <code>int</code>
Offset	Byte distance from base = <code>index * stride</code>	<code>2*4 = 8</code>
Bounds	Valid index range <code>[0, size-1]</code> ; <code>[0, size]</code> for iterators	—

```
std::vector<int> v;
v.reserve(10);           // capacity 10, size 0 – 10 ints of raw storage
v.push_back(7);          // capacity 10, size 1
std::cout << v.size() << " " << v.capacity(); // prints "1 10"
// v[5] is inside capacity but OUTSIDE size -> undefined behaviour
```

THE KEY INSIGHT

Size is a promise about *constructed objects*; capacity is a promise about *bytes*. Indexing past size but within capacity reads memory you legally own and objects that do not exist. It will not crash — it will return garbage.

terminology – offset = index × stride · valid index [0,size) · valid iterator [begin,end]

SECTION 2 — MEMORY & INTERNAL WORKING

6. Internal Working

At the machine level an array subscript is not a function call, a lookup or a search. It is one addressing-mode operand. On x86-64, `a[i]` where `a` is in `rdi` and `i` in `esi` compiles to a single instruction, because the hardware has a base+index×scale+displacement addressing mode built for exactly this.

```
int get(int* a, int i) { return a[i]; }

// g++ -O2 -S ->
//      mov     eax, DWORD PTR [rdi+rsi*4]
//      ret
// One instruction. The *4 is free – it is part of the addressing mode.
```

Compare a linked-list access at position `i`: `i` dependent loads, each of which must complete before the next address is known. The CPU cannot overlap them, so the latencies add. That is why the array wins by a constant factor that grows with cache pressure, not by a Big-O class (Topic 148).

ARRAY <code>a[3]</code>	LINKED LIST <code>at(3)</code>
base --+	head -> <code>n0</code> -> <code>n1</code> -> <code>n2</code> -> <code>n3</code>
compute	(load) (load) (load) (load)
v	each address unknown until the
one load at base+12	previous load returns: serial

THE KEY INSIGHT

Array indexing is *parallelisable*: the CPU can compute ten addresses at once because none depends on a previous load. Pointer chasing is serial by construction. This, not instruction count, is the real gap.

THE TRAP

Writing `a[i]` inside a loop where `a` is a `vector<vector<int>>` looks like an array access but is two dependent loads — outer buffer, then inner buffer. A "2D array" built that way loses the contiguity you thought you had bought (Topic 91).

indexing – 1 instruction · 1 load · ~4 cycles from L1, ~200 cycles from DRAM

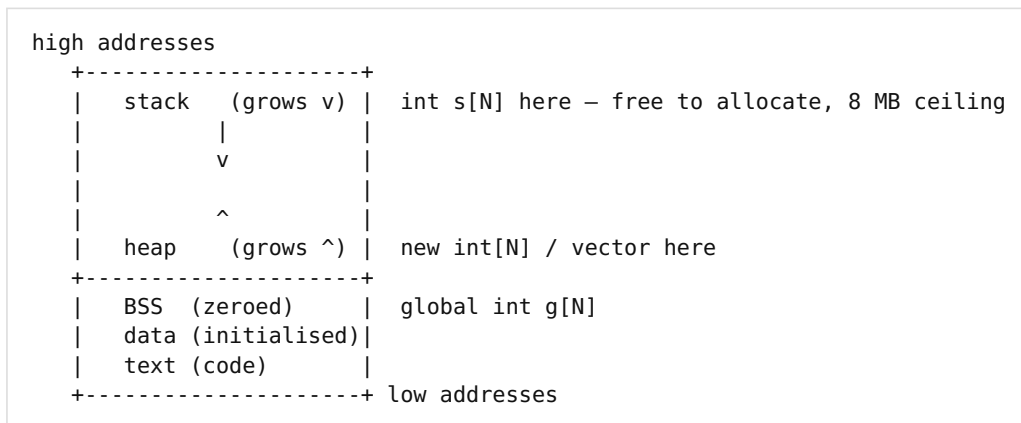
7. Memory Model & Layout

Where an array lives decides its size limit, its lifetime, and its cost to create. Three regions matter.

Region	Declaration	Limit and cost
Stack	<code>int a[1000];</code> inside a function	Typically 8 MB total on Linux (<code>ulimit -s</code>), 1 MB on Windows threads. Allocation cost: <i>zero</i> — one subtraction from <code>rsp</code> . Dies at scope exit.
Static / BSS	<code>int a[1000];</code> at file scope	Limited by address space; zero-initialised for free by the loader. Lives for the whole program. Common in competitive programming for exactly this reason (Topic 173).
Heap	<code>new int[n], vector</code>	Limited by RAM plus contiguous virtual space. Cost: ~50–200 ns per allocation, plus 8–32 bytes of allocator header <i>outside</i> your block.

```
int g[1 << 20]; // BSS: 4 MB, zeroed by loader, no runtime cost

void f() {
    int s[1 << 20]; // STACK: 4 MB – over half a default 8 MB stack
    std::vector<int> h(1 << 20); // HEAP: 4 MB, one malloc, value-initialised
}
```



THE TRAP

A large stack array does not report an error. `int a[3000000];` in `main` overflows the 8 MB stack and the symptom is a bare `Segmentation fault` at *function entry*, before your first line runs — which makes people debug the wrong code for an hour. Rule of thumb: anything over ~1 MB (256K ints) goes on the heap or in BSS.

stack 0(1) alloc, 8 MB cap · BSS free+zeroed, whole-program lifetime · heap ~50–200 ns/alloc + header

8. Address Calculation Formula

The formula is the structure. For a 1D array:

```
addr(a[i]) = base + i * sizeof(T)

base       = address of a[0]
sizeof(T)  = stride, a compile-time constant
i          = 0-based index
```

For a 2D array declared `T m[R][C]`, C++ stores it row-major (Topic 9), so a row is itself a contiguous array and the formula nests:

```
addr(m[r][c]) = base + (r * C + c) * sizeof(T)
```

Note C (columns per row) appears, R does not — which is why a function taking a 2D array must know the column count: `void f(int m[][C])`.

```
#include <stdio>
int main() {
    int m[3][4] = {};
    char* base = reinterpret_cast<char*>(m);
    char* elem = reinterpret_cast<char*>(&m[2][1]);
    std::printf("%td\n", elem - base);    // 36 = (2*4 + 1) * 4
    std::printf("%zu %zu\n", sizeof(m), sizeof(m[0]));    // 48 16
}
```

If the language used 1-based indexing, every access would cost an extra subtraction: `base + (i-1)*size`. Zero-basing is not a style choice; it removes an instruction from the hottest operation in computing.

THE KEY INSIGHT

Because `sizeof(T)` is a compile-time constant, the multiply becomes a shift for 1/2/4/8-byte types. For a 12-byte struct the compiler must emit a real `imul` — a measurable reason padding to 16 bytes sometimes speeds up a scan (Topic 121).

THE TRAP

Do the arithmetic in `int` and a 3 GB `char` array silently wraps: `i * 4` overflows at `i = 536,870,912` and you index backwards from the base. The symptom is a segfault at a plausible-looking index, or worse, a silent write into unrelated data. Use `size_t/ptrdiff_t` (Topics 130, 133).

address maths – 1 multiply + 1 add, folded into the addressing mode · 0(1) for any number of dimensions

9. Row-Major vs Column-Major

A 2D array is a lie the compiler tells: memory is one-dimensional, so the implementation must pick an order to flatten rows and columns. C, C++, Python (NumPy default) and Java use **row-major** — the whole of row 0, then the whole of row 1. FORTRAN, MATLAB, R and Julia use **column-major**.

```
m = 1 2 3      ROW-MAJOR (C++)      : 1 2 3 4 5 6
    4 5 6      COLUMN-MAJOR (FORTRAN): 1 4 2 5 3 6

row-major: m[r][c] at r*C + c    -> walking c is stride 1 (fast)
col-major: m[r][c] at c*R + r    -> walking r is stride 1
```

The practical consequence is a factor of 5–8 in real code. Iterating a 4096×4096 `int` matrix row-first touches each 64-byte cache line once and gets all 16 ints from it. Column-first touches a new line every element: 16.7 million cache misses instead of about 1 million.

```
// Row-major traversal: ~40 ms on a 4096x4096 int matrix
for (int r = 0; r < R; ++r)
    for (int c = 0; c < C; ++c) sum += m[r][c];    // stride 4 bytes

// Column-major traversal of the SAME array: ~250 ms. Identical Big-O.
for (int c = 0; c < C; ++c)
    for (int r = 0; r < R; ++r) sum += m[r][c];    // stride 16384 bytes
```

THE KEY INSIGHT

In C++ the *last* index varies fastest. Loop nesting should match: the innermost loop varies the last subscript. Get that backwards and you pay 5–8× for an algorithm with the same complexity — the single cheapest optimisation in the book (see Topics 102 and 116).

THE TRAP

Passing a C++ matrix to a BLAS/LAPACK routine without transposing gives you the transpose of the answer, not an error — LAPACK assumes column-major. Symptom: results that are correct for symmetric inputs and wrong for everything else.

layout – same $O(R \cdot C)$, 5–8× wall clock · row-major: last index fastest · 1 miss per 16 ints vs 1 per element

PRACTICE SET — Topics 1-9

A. Conceptual

1. State the one property from which $O(1)$ indexing, $O(n)$ middle insertion and cache friendliness all follow. Then explain each of the three in one sentence using only that property.
2. Why does a linked list traversal not benefit from the hardware prefetcher while an array traversal does?
3. `v.size()` is 3 and `v.capacity()` is 16. Is `v[7]` undefined behaviour? Is it a segfault? Answer both, separately.
4. A function takes `int m[][C]`. Why must `C` be supplied but not the number of rows?
5. Explain why zero-based indexing removes an instruction compared with one-based, referring to the address formula.
6. Give two situations where a global (BSS) array is a better choice than a `vector`, and one where it is clearly worse.
7. Name the property that `vector<vector<int>>` loses compared with `int m[R][C]`, and name the cost that loss imposes.
8. You are told a program "runs fine on my machine, segfaults in CI." The array in question is `int buf[2'000'000]` declared inside a function. Give the most likely mechanism and the one-line fix.
9. Why do LAPACK bindings need to know whether your matrix is row-major?
10. Under what circumstance does the multiply in `base + i*sizeof(T)` become an actual `imul` instruction rather than a shift, and why might that motivate padding a struct?

B. Tracing and analysis

1. An array `double d[6]` has base address `0x7ffd_1000`. Give the address of `d[4]` and the total byte size.
2. `int m[5][7]` has base `0x2000`. Compute the byte offset and absolute address of `m[3][5]`. Then compute which element lives at offset 96.
3. A 4096×4096 `int` matrix is scanned column-first. Assuming a 64-byte cache line and no prefetching, how many cache lines are loaded? How many in row-first order? State the ratio.
4. A struct is `{ char id; double v; }`. Give `sizeof` on a typical x86-64 ABI, the offset of `v`, and the stride of an array of 1000 of them. How many whole structs fit in one 64-byte line?
5. Compute the largest `int` index `i` for which `i * sizeof(int)` does not overflow a signed 32-bit int. What is the array size in gigabytes at that point?
6. A default Linux stack is 8 MB. How many `int`s fit? How many `std::strings` (32 bytes each on `libstdc++`)?

C. Code tasks

1. **Measure the address formula.** Declare `int a[8]` and print `(char*)&a[i] - (char*)a` for every `i`. Repeat with `double` and with a 12-byte struct. Confirm the printed offsets equal `i * sizeof(T)` in all three cases.
2. **Break it: read past the end.** Write a loop `for (int i = 0; i <= 5; ++i) print(a[i]);` over a 5-element array. Run it plain — record what it prints. Then rebuild with `g++ -fsanitize=address -g` and record the exact first line of the report. Only then fix the `<=`. The lesson is the gap between the two runs, not the fix.
3. **Break it: overflow the stack.** Declare `int big[3'000'000]` inside a function, print `"entered"` as the very first statement, and run it. Note whether `"entered"` appears. Re-run under `ulimit`

- `-s unlimited`, then move the array to `static` and compare. Explain the ordering you observed.
4. **Row-major cost.** Allocate a 4096×4096 `int` matrix on the heap, sum it row-first and column-first, and time both with `std::chrono::steady_clock`. Compile `-O2`. Report both times and the ratio; then run `perf stat -e cache-misses` on each and report the miss counts.
 5. **Stack versus heap versus BSS.** Time 1,000,000 iterations of a function that creates a 1000-element array on the stack, versus one that does `new int[1000]; delete[]`. Report ns per iteration for each.
 6. **Break it: the capacity gap.** Build a `vector<int>`, `reserve(16)`, `push_back` three values, then print `v[7]` and `v.at(7)` in that order, each in its own run. Record what each does. Explain why one is silent.

ANSWER KEY — Topics 1-9

A. Conceptual

1. **Contiguity of equal-sized elements.** Indexing is $O(1)$ because the address is computed, not searched. Middle insertion is $O(n)$ because contiguity must be restored by shifting every following element. Cache friendliness follows because one 64-byte line delivers 16 neighbouring ints.
2. The prefetcher recognises *constant strides*. Array access has stride `sizeof(T)`; list nodes sit at addresses the allocator chose, so there is no stride to detect. **Worse, each node's address is unknown until the previous load returns** — the loads are serial and their latencies add.
3. **Yes, undefined behaviour; almost certainly not a segfault.** The memory is owned by the vector (capacity 16) so the OS page is mapped and readable — you read a raw, unconstructed `int`. Being inside capacity makes it *more* dangerous, not less: the failure is silent.
4. Because the address formula is `base + (r*C + c)*sizeof(T)`. **C appears, R does not.** Rows are only needed for bounds, which C++ does not check.
5. One-based requires `base + (i-1)*size` — an extra subtraction on every access. Zero-based makes the index the offset directly.
6. Better: (a) competitive programming, where a fixed 10^7 -entry frequency array must be zeroed for free by the loader and never reallocated; (b) embedded/firmware with no heap. Worse: any library or reentrant code — a global is one shared instance, not thread-safe and not sizeable at runtime.
7. It loses **contiguity between rows**. Cost: an extra dependent load per row access, no whole-matrix `memcpy`/SIMD, and roughly `R` separate allocations with allocator headers between rows.
8. **Stack overflow** — 8 MB of stack, 8 MB requested. The frame is reserved on entry, so the crash happens *before* the first statement. Fix: `static int buf[2'000'000]`; or a `vector`.
9. LAPACK is FORTRAN heritage and assumes column-major. The same bytes read the other way are the transpose, so results are silently wrong for non-symmetric input.
10. When `sizeof(T)` is not a power of two (e.g. 12). Padding to 16 turns `imul` into `shl 4`, at the cost of 33% more memory — worth it only when the scan is index-heavy and fits in cache either way.

B. Tracing and analysis

1. `0x7ffd1000 + 4*8 = 0x7ffd1020`; size **48 bytes**.
2. Offset `(3*7 + 5)*4 = 26*4 = 104`, address **0x2068**. Offset 96 → element 24 → `24 = r*7 + c` → **m[3][3]**.
3. Row-first: $4096 \text{ rows} \times (16384 \text{ B} / 64 \text{ B}) = 4096 \times 256 = \mathbf{1,048,576 \text{ lines}}$. Column-first: every access is a new line → **16,777,216 lines**. Ratio **16:1** (measured wall-clock ratio is smaller, 5–8×, because the prefetcher and TLB recover some of it).
4. `sizeof = 16` (1 byte `char`, 7 padding, 8 `double`); `offsetof(v) = 8`; stride **16**; array of 1000 = 16,000 B; **4 structs per line**.
5. `i < 231 / 4 = 536,870,912`, so max `i = 536,870,911`, i.e. **2 GB** of ints. Above that, 32-bit index arithmetic wraps.
6. $8 \text{ MB} / 4 \text{ B} = \mathbf{2,097,152 \text{ ints}}$; $8 \text{ MB} / 32 \text{ B} = \mathbf{262,144 \text{ strings}}$ — and each of those may own a heap buffer as well.

C. What to verify

1. Offsets must be exactly `0,4,8,...,28`; `0,8,...,56`; and `0,12,...,84`. If the struct prints 0,16,32... your compiler padded it — check `sizeof`.
2. Plain run typically prints five values plus one arbitrary number (often 0, a loop counter, or the saved frame pointer's low word) — **and does not crash**. ASan prints `ERROR: AddressSanitizer: stack-buffer-overflow with READ of size 4 at ... 0 bytes after` and a `'a' (line N)` frame annotation. **The point: the same bug is invisible without the tool.**
3. `"entered"` **does not print** — the frame (12 MB) is allocated at entry and touching it faults before your statement. With `ulimit -s unlimited` it prints. As `static` it prints and uses BSS instead. Expect `Segmentation fault (core dumped)`, exit code 139.
4. Typical: row-first **35-60 ms**, column-first **200-350 ms**, ratio **5-8×**. `perf stat` should show roughly **1M** vs **16M** `cache-misses`. If the ratio is 1.0, the compiler interchanged your loops — check with `-O1` or add a volatile sink.
5. Stack version: **~0-2 ns** per iteration (often optimised to nothing; keep a sink). Heap version: **~40-120 ns**. The gap is the allocator, not the memory.
6. `v[7]` prints garbage (frequently `0`) and exits `0`. `v.at(7)` throws: `terminate called after throwing an instance of 'std::out_of_range' / what(): vector::_M_range_check: __n (which is 7) >= this->size() (which is 3)`, exit code **134**. Silent because the memory is *owned* — reserve made it legal to touch and illegal to use.

HANDNOTE SUMMARY CARD — Foundations & Memory

FOUNDATIONS (Topics 1-9)

$\text{addr}(a[i]) = \text{base} + i * \text{sizeof}(T)$

DEFINITION n objects, identical type, contiguous, no gaps, no per-element header
-> index is COMPUTED, never searched. Everything else follows.

ADDRESS MATHS

1D $\text{addr}(a[i]) = \text{base} + i * \text{sizeof}(T)$
2D $\text{addr}(m[r][c]) = \text{base} + (r * C + c) * \text{sizeof}(T)$ C appears, R does not
ND last index varies fastest (row-major) FORTRAN/MATLAB: first
compile-time sizeof -> multiply folds to shift for 1/2/4/8-byte types

FOUR PROPERTIES drop one and the cost model changes

homogeneous -> element size constant -> $O(1)$ maths
contiguous -> prefetch + SIMD -> needs one unbroken block
fixed size -> no capacity field -> growth = allocate + copy
0-based -> index == offset -> off-by-one is the classic bug

VOCABULARY

size constructed objects, valid indices $[0, \text{size})$
capacity bytes reserved, $\geq \text{size}$; index in $[\text{size}, \text{capacity})$ = silent garbage
stride $\text{sizeof}(T)$ offset = index * stride
base $\&a[0]$ bounds $[0, \text{size}-1]$; iterators $[\text{begin}, \text{end}]$

WHERE IT LIVES

region	decl	limit	alloc cost	lifetime
stack	<code>int a[N]</code> in function	~8 MB Linux	~0 (sub rsp)	scope
BSS	<code>int a[N]</code> at file scope	address space	0, pre-zeroed	program
heap	<code>new int[n]</code> / vector	RAM + VA	50-200 ns	manual/RAII

COSTS

$a[i]$ access $O(1)$ 1 instruction, ~4 cyc L1, ~200 cyc DRAM
scan n $O(n)$ 1 cache line per 16 ints
insert/delete mid .. $O(n)$ memmove of the tail
vs linked list 5-20x faster scan; list loads are SERIAL (dependent)

TRAPS

$a[n]$ compiles, does not crash, prints a plausible number -> ASan
index in $[\text{size}, \text{capacity})$ is owned memory + no object -> `.at()`
`int a[n]` with runtime $n = \text{VLA}$, GCC/Clang only, not MSVC -> vector
`int index * 4` overflows at $i = 536,870,911$ (2 GB) -> `size_t`
1 MB+ on the stack: segfault BEFORE the first statement -> static/heap
column-first scan of 4096^2 : 16M misses vs 1M, 5-8x slower -> swap loops
`vector<vector>` is not contiguous between rows -> flat + $r * C + c$

TEST MATRIX (every array routine)

$n = 0$ | $n = 1$ | $n = 2$ | all equal | already sorted | reversed | max index
first element | last element | duplicates | overflow-sized values

PART I CHEATSHEET — The Mental Model

PART I: ONE PAGE THAT PREDICTS THE REST OF THE BOOK

THE THREE QUESTIONS TO ASK OF ANY ARRAY OPERATION

1. Does it move elements? -> if yes, it is $O(n)$: shifting restores contiguity
2. Does it walk with stride 1? -> if yes, ~1 miss / 16 ints; if not, 1 miss / element
3. Does it change the size? -> if yes, someone allocates and copies

WHY EACH COST IS WHAT IT IS

access $O(1)$	one multiply-add, folded into the addressing mode
search $O(n)$	no ordering assumed; $O(\log n)$ only once sorted (Topic 42)
insert $O(n)$	the tail must slide; the copy, not the write, dominates
append $O(1)^*$	amortised, doubling capacity (Topic 115)
delete $O(n)$	same slide, backwards
resize $O(n)$	new block + copy + free; old pointers die (Topic 26)

THE COMPARISON THAT DECIDES DESIGNS

need	array/vector	linked list	hash table
index i	$O(1)$	$O(n)$	n/a
scan 10M	~4 ms	~45 ms	worse (scattered)
insert front	$O(n)$	$O(1)$	n/a
memory per elem	<code>sizeof(T)</code>	<code>sizeof(T)+16..32</code>	~1.4x + buckets
cache behaviour	prefetched	a miss per node	a miss per probe

WHEN THE ARRAY IS THE WRONG ANSWER (say this out loud in interviews)

- many inserts/deletes at the FRONT or MIDDLE, no index access -> deque/list
- the size grows unpredictably AND elements are large/non-movable -> deque
- keys are sparse (ids up to 10^9 , only 10^4 present) -> hash map, not `a[id]`
- stable addresses required across growth -> vector reallocation invalidates all
- one contiguous 100 MB block may fail while 100 MB free exists (fragmentation)

LOOKUP: HOW BIG IS IT

10^6 ints = 4 MB 10^7 ints = 40 MB 10^8 ints = 400 MB (too big for CP)
`bool[ $10^8$ ]` = 100 MB `bitset< $10^8$ >` = 12.5 MB (Topic 160)
`int m[1000][1000]` = 4 MB, fine on the heap, fatal on an 8 MB stack

CROSS-REFERENCES OUT OF PART I

address maths	-> Topic 16 (decay), 90 (2D), 111 (2D prefix sums)
stack vs heap	-> Topic 24 (<code>new[]</code>), 29 (RAII), 173 (CP globals)
row-major	-> Topic 102 (<code>matmul</code>), 103 (transpose), 116 (cache)
bounds	-> Topic 123 (checking), 127 (overflow), 134 (<code>.at</code>)

PART II

Arrays in C++

Syntax, decay, function boundaries, the heap, and the four modern containers that replaced raw arrays without changing a single byte of layout. Sections 3-7 · Topics 10-37.

SECTION 3 — C++ BASICS

10. C++ Array Syntax

The declaration `T name[N]` reserves `N*sizeof(T)` bytes and binds `name` to the first of them. `N` must be a constant expression: a literal, a `constexpr`, or an `enum`. It is part of the *type* — `int[5]` and `int[6]` are different types, which is why `sizeof` works and why you cannot assign one array to another.

```
constexpr int N = 5;
int a[N];           // 5 ints, indeterminate values
int b[] = {1, 2, 3}; // size deduced: 3
const int c[3] = {7, 8, 9}; // read-only (Topic 20)

a[0] = 42;          // update
int x = a[0];        // access
// a = b;           // ERROR: array type is not assignable
std::cout << sizeof(a) / sizeof(a[0]); // 5 — only while a is a real array
```

`a[i]` is defined by the standard as `*(a + i)`. That is not trivia: it makes `a[i]` and `i[a]` equivalent (`3[a]` compiles and works), and it explains array decay in Topic 17.

```
int a[5];
type of a : int[5]      sizeof(a) = 20
type of a[0]: int       sizeof(a[0]) = 4
type of &a : int(*)[5]  sizeof(&a) = 20
a decays to : int*      sizeof(ptr) = 8 <- length lost here
```

THE TRAP

`sizeof(a)/sizeof(a[0])` silently becomes `8/4 = 2` the moment `a` is a function parameter, because parameters are pointers. The symptom is a loop that processes exactly two elements of a 1000-element array and reports success. Use `std::size(a)` (C++17) — it refuses to compile on a pointer, which is precisely the value of it.

declaration — 0 runtime cost on stack · size is part of the type · assignment not supported

11. Array Initialization Methods

Six forms, three of which are silent traps if you assume the wrong default.

Form	Result	Cost
<code>int a[5];</code> (local)	Indeterminate. Whatever was on the stack	0
<code>int a[5];</code> (global/static)	All zero — the loader maps BSS pre-zeroed	0
<code>int a[5] = {};</code>	All zero	5 stores, or one <code>memset</code>
<code>int a[5] = {1,2};</code>	1, 2, 0, 0, 0 — the rest are value-initialised	5 stores
<code>int a[5] = {0};</code>	All zero (same rule; the 0 is not special)	5 stores
<code>std::fill(a, a+5, 7);</code>	All 7 — the only way to fill with non-zero	O(n), vectorised

```

int local[3];                // e.g. 32767, 0, 4198400 – garbage
int zeroed[3] = {};         // 0 0 0
int partial[5] = {1, 2};    // 1 2 0 0 0 <- NOT 1 2 garbage
char buf[8] = {};           // 8 NUL bytes
std::array<int,5> m{};       // 0 0 0 0 0 (Topic 30)
int wrong[5] = {7};         // 7 0 0 0 0 <- NOT five 7s
std::fill(std::begin(wrong), std::end(wrong), 7); // now five 7s

```

THE KEY INSIGHT

`= {0}` works only because *zero* is what missing initialisers default to. `= {7}` does not give you five sevens, and the compiler will not warn. `memset(a, 1, n)` does not either — it writes byte `0x01`, producing `16843009` per int.

THE TRAP

Reading an uninitialised local is undefined behaviour, not "reads garbage" — the optimiser is allowed to assume it never happens and delete surrounding code. The symptom is a function that works at `-O0` and returns nonsense at `-O2`. Catch it with `-fsanitize=memory` (Clang) or `valgrind --track-origins=yes`.

init – BSS zeroing free · `{}` = `memset`-speed · fill `O(n)` SIMD · uninitialised local = UB, not garbage

12. Input and Output

Reading n values is a loop; the interesting part is the cost. Unsynchronised `cin` reads 10^6 ints in roughly 90 ms; synchronised with C stdio, roughly 400 ms; `scanf`, about 120 ms; a hand-rolled `getchar_unlocked` parser, about 25 ms. In competitive programming that difference is the verdict (Topic 173).

```

#include <iostream>

int main() {
    std::ios::sync_with_stdio(false); // ~4x faster cin
    std::cin.tie(nullptr);            // stop flushing cout before each read

    int n;
    std::cin >> n;
    std::vector<int> a(n);
    for (int i = 0; i < n; ++i) std::cin >> a[i];

    for (int x : a) std::cout << x << ' '; // range-for, Topic 13
    std::cout << '\n';                     // '\n' not endl: endl flushes
}

```

READ 10^6 INTS	time	why
<code>cin</code> (syncd)	~400 ms	every op goes through the C stdio buffer
<code>cin + sync_with_stdio</code>	~90 ms	own buffer, no interleaving guarantee
<code>scanf</code>	~120 ms	format string parsed at runtime
<code>getchar_unlocked</code>	~25 ms	no locking, no parsing machinery

THE TRAP

After `sync_with_stdio(false)`, mixing `cin` with `scanf` in the same program reads the same bytes twice or loses them — they now own separate buffers. Symptom: the first value read by the second API is garbage or the input appears truncated. Pick one API per program.

I/O – 10^6 ints: 400 ms → 90 ms with two lines · `endl` flushes, `'\n'` does not

13. Traversal Techniques

Five ways to walk an array. They compile to nearly identical code at -O2; choose on safety and intent, not speed.

```
int a[5] = {1,2,3,4,5};

for (int i = 0; i < 5; ++i) sum += a[i];           // index: you need i
int i = 0; while (i < 5) { sum += a[i]; ++i; }    // same, more room to slip
for (int x : a) sum += x;                         // range-for: copies x
for (int& x : a) x *= 2;                          // range-for by reference
for (int* p = a; p != a + 5; ++p) sum += *p;      // pointer
for (auto it = std::begin(a); it != std::end(a); ++it) sum += *it; // iterator
```

Prefer `const auto&` in a range-for when the element is a class type: `for (auto s : strings)` copies every string, which turns an $O(n)$ scan into $O(n)$ allocations. For `int` it makes no difference.

p		end	
+---+---+---+---+			
1 2 3 4			half-open range [begin, end)
+---+---+---+---+			end is ONE PAST the last element:
0 1 2 3			legal to form, illegal to dereference

THE KEY INSIGHT

Every C++ range is half-open `[begin, end)`. That is why `end - begin` is the size, why an empty range is `begin == end`, and why loop conditions use `!=` rather than `<` for iterators. The same convention governs Topics 44, 64 and 78.

THE TRAP

`for (auto x : v) v.push_back(x);` invalidates the iterators mid-loop when the vector reallocates. Symptom: it usually runs, occasionally crashes, and under ASan reports *heap-use-after-free* inside the loop header — a place your code does not appear to touch.

traversal – all forms $O(n)$, identical asm at -O2 · range-for by value copies class types

14. Types of Arrays

Kind	Declaration	Use when
Static / automatic	<code>int a[100];</code>	Size known at compile time, small enough for the stack
Dynamic (raw)	<code>new int[n]</code>	Almost never — use vector (Topic 24)
Dynamic (owning)	<code>std::vector<int> v(n)</code>	The default choice for runtime size
Fixed container	<code>std::array<int,100></code>	Compile-time size, but you want a real type (Topic 30)
Multidimensional	<code>int m[R][C]</code>	True contiguous matrix (Topic 90)
Jagged	<code>vector<vector<int>></code>	Rows of differing lengths (Topic 91)
View	<code>std::span<int></code>	Function parameters — borrow, do not own (Topic 32)

THE KEY INSIGHT

The choice is two independent questions: *who owns the memory* (array, vector, span-of-someone-else's) and *when the size is known* (compile time, construction time, never). Answer both and the type is determined.

selection – vector by default · array when N is constant · span at API boundaries · raw new[] essentially never

15. One-Dimensional Arrays

The 1D array is the base case every other layout reduces to: a 2D array is a 1D array of rows (Topic 90), a matrix flattened is a 1D array with computed indices (Topic 8), a bitset is a 1D array of words (Topic 160). Master the five operations here and the rest are re-indexings.

```
// The five primitive operations, all on int a[n] with logical size len:
a[i]                                // access      0(1)
a[i] = v;                           // update      0(1)
for (i = len; i > pos; --i) a[i] = a[i-1]; a[pos] = v; // insert  0(n)
for (i = pos; i < len-1; ++i) a[i] = a[i+1];           // delete  0(n)
for (i = 0; i < len; ++i) if (a[i] == key) return i;   // search  0(n)
```

```
insert 99 at index 2 into [10,20,30,40,_] len 4 -> 5
step 1 shift right from the back: [10,20,30,30,40]
step 2 write:                      [10,20,99,30,40]
cost: len - pos moves = 2 writes here, up to n at pos 0
```


THE KEY INSIGHT

Shift *from the far end towards* the insertion point. Shifting forwards overwrites the value you have not copied yet — the single most common bug in Topic 39, and the reason `memmove` exists while `memcpy` does not handle overlap.

1D primitives – access/update $O(1)$ · insert/delete $O(n)$, n -pos moves · search $O(n)$ unsorted

SECTION 4 — POINTERS, FUNCTIONS & REFERENCES

16. Arrays vs Pointers

They are not the same thing, and confusing them causes a specific family of bugs. An array *is* the storage; a pointer *refers* to storage. The array's name converts to a pointer in most expressions (Topic 17), which makes them look identical until you take `sizeof` or `&`.

Expression	<code>int a[5]</code>	<code>int* p = a</code>
<code>sizeof</code>	20 — the whole block	8 — the pointer
<code>&x</code> type	<code>int(*)[5]</code>	<code>int**</code>
Assignable	No	Yes
<code>++x</code>	Illegal	Legal, advances 4 bytes
Storage	20 bytes of ints	8 bytes, plus the ints elsewhere

```
int a[5] = {1,2,3,4,5};
int* p = a;           // decay: p points at a[0]

p += 2;               // pointer ARITHMETIC scales by sizeof(int)
                     // p is now &a[2], the address advanced by 8
std::cout << p[1];    // 4 - p[1] means *(p+1) means a[3]
std::cout << p - a;    // 2 - difference is in ELEMENTS, not bytes
```

THE TRAP

Pointer difference is in elements. `(char*)q - (char*)p` gives bytes; `q - p` gives elements. Mixing them produces a value off by a factor of `sizeof(T)` — a bug that is invisible for `char` arrays and appears the moment someone changes the element type.

pointer arithmetic – scaled by `sizeof(T)` · difference in elements · pointer costs 8 bytes, array costs 0 extra

17. Array Decay

In almost every expression, an array name converts implicitly to a pointer to its first element. This is *decay*, and it is where the length goes to die. Three exceptions preserve the array type: `sizeof(a)`, `&a`, and binding to a reference-to-array (Topic 23).

```
void f(int a[100]) {           // the 100 is a comment; parameter is int*
    std::cout << sizeof(a);    // 8, not 400 – and no warning by default
}

int main() {
    int a[100];
    std::cout << sizeof(a);    // 400 (no decay: sizeof is an exception)
    f(a);                     // decays here
    int (*pa)[100] = &a;      // no decay: &a is int(*)[100]
    std::cout << sizeof(*pa);  // 400
}
```

<code>int a[100]</code>	<code>f(int a[100])</code>	<code>f(std::span<int> s)</code>
+-----+	+-----+	+-----+
100 ints	int* ptr	int* ptr 100
+-----+	+-----+	+-----+
length in the type	length GONE	length travels with it

THE KEY INSIGHT

Every "why doesn't my function know the array size?" question is decay. The fix is never to recover the length — it no longer exists — but to pass it: as a second parameter, as a `span` (Topic 32), or by taking a reference to the array (Topic 23).

THE TRAP

`void f(int a[100])` reads like a contract and enforces nothing: callers may pass a 3-element array and the compiler is silent. Turn on `-Wsizeof-pointer-div -Wsizeof-array-argument`; GCC will then flag the `sizeof(a)/sizeof(a[0])` case specifically.

decay – implicit, free, irreversible · exceptions: `sizeof`, `&a`, `T(&)[N]` binding

18. Passing Arrays to Functions

Five ways, in ascending order of how much the compiler can check for you.

```
void a1(int* p, int n);           // 1. pointer + count. C style, works.
void a2(int p[], int n);         // 2. identical to a1. The [] is decoration.
void a3(int (&r)[5]);             // 3. reference to array: size checked!
template <std::size_t N>
void a4(int (&r)[N]);             // 4. any size, size deduced, checked
void a5(std::span<int> s);        // 5. C++20: ptr+len, accepts array,
                                //    vector, std::array, subrange
void a6(std::vector<int>& v);      // 6. only vectors; forces a copy otherwise
```

Form 5 is the modern default: one parameter, no template, works with every contiguous container, and `s.size()` is always right. Form 3/4 are the right answer when the size must be part of the contract — passing a 4-element array to `void f(int(&)[3])` is a compile error, which is exactly what you want for fixed-size protocol buffers.

Form	Size known?	Accepts
<code>int*, int n</code>	Only if the caller is honest	Anything, including <code>nullptr</code>
<code>int (&)[5]</code>	Yes, compile-time enforced	Exactly <code>int[5]</code>
<code>template<N> int(&)[N]</code>	Yes, deduced	Any true array; one instantiation per N
<code>span<int></code>	Yes, at runtime	array, vector, <code>std::array</code> , <code>subspan</code>

THE TRAP

Passing `vector<int>` by value copies every element. On a 10^6 -element vector that is 4 MB and roughly 300 μ s *per call*. Symptom: a profile where a trivial helper dominates. Pass `const vector<int>&` or `span`.

passing – pointer/span: $O(1)$, 8–16 bytes copied · vector by value: $O(n)$ + one allocation

19. Returning Arrays

You cannot return a raw array from a function; the language forbids it, and the workarounds people invent are the classic dangling-pointer bug.

```
int* broken() {
    int a[5] = {1,2,3,4,5};
    return a;           // DANGLING: a dies at the closing brace
}                       // GCC: "warning: address of local variable
                        //      'a' returned"

std::vector<int> good1() { return {1,2,3,4,5}; } // moved, no copy
std::array<int,5> good2() { return {1,2,3,4,5}; } // 20 bytes, in regs/NRVO
struct P { int x, y; };
P good3() { return {1,2}; } // struct: fine
void good4(std::span<int> out) { out[0] = 1; } // caller owns memory
```

Returning a **vector** is *not* a copy in any compiler from the last decade: guaranteed copy elision (C++17) constructs it directly in the caller's frame. The "return by out-parameter for speed" advice is obsolete.

broken(): frame: [a: 1 2 3 4 5] return &a[0] -----+ frame destroyed caller reads freed memory	good1(): heap: [1 2 3 4 5] <- owned by the vector return the 24-byte vector object, constructed in place at the call site
--	---

THE TRAP

The dangling read *usually works* immediately after the call, because nothing has reused the stack yet. It breaks after you add an unrelated function call between the return and the read. Symptom: "adding a **printf** fixed it." ASan reports *stack-use-after-return* only with **ASAN_OPTIONS=detect_stack_use_after_return=1**.

returning – vector/array: elided, 0 copies · raw pointer to local: UB · out-param: caller-controlled allocation

20. const Arrays

Const-correctness on arrays is really const-correctness on pointers, and the two positions of **const** mean different things.

```
const int a[3] = {1,2,3};

const int* p = a; // pointer to const int: *p = 5 illegal, p++ fine
int* const q = b; // const pointer to int: *q = 5 fine, q++ illegal
const int* const r = a; // both fixed

void sum(const int* a, int n); // "I will not modify your data"
void sum(std::span<const int> a); // same promise, size included
```

Read declarations right to left: **const int* p** is "p is a pointer to an int which is const." The rule of thumb — **const** applies to what is on its left, unless there is nothing there, in which case it applies to the right.

THE KEY INSIGHT

Marking a read-only parameter `const` is not documentation. It lets the compiler assume no aliasing writes through that pointer, which is often what enables vectorisation (Topic 122) — a real speed difference, not just style.

THE TRAP

`const` arrays at namespace scope have internal linkage in C++ (not C). Defining `const int TABLE[256] = {...};` in a header gives every translation unit its own copy: 1 KB × 40 TUs. Use `inline constexpr` (C++17) instead.

`const` — 0 runtime cost · enables no-alias assumptions · `const` at namespace scope = internal linkage

21. Array of Pointers

`int* a[5]` is five pointers (40 bytes), each of which may point at a block of any size. It is how `argv` works, how a jagged 2D structure is built by hand (Topic 91), and how you build a table of function pointers.

```
const char* names[3] = {"ada", "grace", "alan"}; // 3 pointers into .rodata
int* rows[3];
for (int i = 0; i < 3; ++i) rows[i] = new int[i + 1]; // jagged: 1, 2, 3

int (*ops[4])(int, int) = {add, sub, mul, div_}; // function-pointer table
std::cout << ops[2](6, 7); // 42 — dispatch by index
```

<code>int* rows[3]</code>	heap	
+-----+	+----+	
ptr ----->	1	each row separately
+-----+	+---+---+	allocated: 3 mallocs,
ptr ----->	2 2	3 headers, rows may be
+-----+	+---+---+---+	far apart in memory
ptr ----->	3 3 3	
+-----+	+---+---+---+	

THE TRAP

`int* a[5]` (array of 5 pointers, 40 bytes) versus `int (*a)[5]` (one pointer to an array of 5, 8 bytes). The declarations differ by two characters and by 32 bytes plus one level of indirection. Read them with the "spiral rule", or write `using Row = int[5]; Row* a;` and be understood.

array of pointers — 8n bytes of pointers + n allocations · one extra dependent load per access · rows non-contiguous

22. Pointer to Array

`int (*p)[5]` points at a whole 5-element array. Incrementing it advances **20 bytes**, not 4. This is the type you get from `&a`, and it is the type a 2D array decays to — which is why `int m[3][4]` passed to a function becomes `int (*)[4]`.

```
int a[5] = {1,2,3,4,5};
int (*p)[5] = &a;
std::cout << (*p)[2];      // 3
std::cout << sizeof(*p);   // 20 – the length survived

int m[3][4] = {};
int (*rp)[4] = m;          // m decays to pointer-to-row, not int*
rp += 1;                   // advances 16 bytes = one whole row
std::cout << rp[0][2];     // == m[1][2]
```

THE KEY INSIGHT

A 2D array decays *once*, to a pointer to its first row — never to `int**`. This is the single most common misunderstanding about 2D arrays: `int**` and `int(*)[C]` have incompatible memory layouts, and casting between them compiles under a `reinterpret_cast` and then reads garbage addresses.

pointer-to-array – ++ advances `sizeof(T)·N` bytes · 2D decays to `T(*)[C]`, never `T**`

23. References to Arrays

A reference to an array binds without decay, so the size stays in the type. Combined with a template parameter it gives you compile-time-checked array parameters and the standard idiom behind `std::size`.

```
template <typename T, std::size_t N>
constexpr std::size_t len(const T (&)[N]) { return N; } // exact, checked

void fixed(int (&buf)[16]);      // callers MUST pass an int[16]

int a[16], b[8];
fixed(a);                      // ok
// fixed(b);                   // compile error – this is the feature
static_assert(len(a) == 16);    // computed at compile time, zero cost
```

pass by pointer	<code>f(int* p)</code>	length: lost, trust the caller
pass by reference	<code>f(int(&)[16])</code>	length: in the TYPE, checked at compile
pass by span	<code>f(span<int>)</code>	length: in the OBJECT, checked at runtime

THE KEY INSIGHT

Three places a length can live: in the type (`array`, `T(&)[N]` — free, compile-time), in the object (`vector`, `span` — 8 bytes, runtime), or in the programmer's head (raw pointer — free and wrong). Choosing consciously between the first two eliminates most array bugs.

array reference – 0 bytes passed (as a pointer under the hood) · size checked at compile time · exact-type only

PRACTICE SET — Topics 10-23

A. Conceptual

1. Why is `a[i]` exactly equivalent to `*(a+i)`, and what surprising expression does that equivalence make legal?
2. Name the three contexts in which an array does *not* decay to a pointer.
3. `int a[5] = {7};` — what are the five values, and why is `= {0}` not a special case of the same rule?
4. Explain why `void f(int a[100])` gives no more safety than `void f(int* a)`.
5. Give the type of `&a` for `int a[5]`, and say how many bytes `++` advances a variable of that type.
6. Distinguish `int* a[5]` from `int (*a)[5]` in bytes, indirection levels, and what each is used for.
7. Why does returning `std::vector<int>` by value cost no copy in C++17, and why is the "return via out-parameter for speed" advice obsolete?
8. A function must accept both a C array and a `std::vector` without templates and without copying. Which parameter type, and what does it store?
9. You want a compile error if the caller passes a 15-element array to a function that requires 16. Which parameter form gives you that?
10. Why can `const` on a pointer parameter make the loop *faster*, not merely safer?
11. What is wrong with putting `const int TABLE[256] = {...};` in a header included by 40 translation units?
12. Why does a 2D array decay to `int(*)[C]` rather than `int**`, and what happens if you cast one to the other?

B. Tracing and analysis

1. For `int a[8]`, give `sizeof(a)`, `sizeof(&a)`, `sizeof(*a)`, `sizeof(a+0)` and `sizeof(a[0])` on x86-64.
2. `int a[10]; int* p = a + 7;` Give `p - a`, `(char*)p - (char*)a`, and the value of `p[-3]` in terms of `a`.
3. `int m[3][4]; int (*rp)[4] = m; rp += 2;` How many bytes did `rp` advance? What does `rp[0][1]` alias?
4. Copying a `vector<int>` of 10^6 elements by value at roughly 12 GB/s memory bandwidth: how many bytes, and roughly how many microseconds per call? What is the cost of 1000 such calls?
5. `const char* names[4]` on x86-64: how many bytes does the array occupy, and where do the string bodies live?
6. Trace `sizeof(a)/sizeof(a[0])` for `int a[40]` in `main` and inside `void f(int a[40])`. Give both numbers and say what a loop bounded by the second would do.

C. Code tasks

1. **Prove decay.** Print `sizeof` of the same array in `main` and inside a function taking `int a[100]`. Then add `-Wall -Wextra -Wsizeof-array-argument` and record whether GCC warns in each case.
2. **Break it: the decayed length loop.** Write `void printAll(int a[])` that loops to `sizeof(a)/sizeof(a[0])`, call it with a 40-element array, and record how many elements it actually

prints. Then fix it three ways — extra `n` parameter, `int(&)[40]`, and `std::span` — and confirm all three print 40.

3. **Break it: return a dangling pointer.** Implement the `broken()` function from Topic 19. Print the five values immediately after the call and record them. Then insert an unrelated `std::printf("x\n")` between the call and the read and record them again. Finally run with `ASAN_OPTIONS=detect_stack_use_after_return=1` and record the exact error name.
4. **Initialisation matrix.** Print all five elements of `int a[5];` (local), `= {}`, `= {0}`, `= {7}`, and after `memset(a, 1, sizeof a)`. Explain the value `16843009`.
5. **Break it: iterator invalidation.** Range-for over a `vector<int>` of size 4 and `push_back` inside the loop. Run it plain (note whether it crashes), then under ASan, and record the error class and which line it blames.
6. **Input throughput.** Generate a file of 10^6 random ints. Read it four ways — `cin`, `cin` with `sync_with_stdio(false)`, `scanf`, and a `getchar_unlocked` parser — and report all four times.
7. **Span at the boundary.** Write one function `long total(std::span<const int>)` and call it with a C array, a `std::array<int,5>`, a `vector`, and `v | subspan(2,3)`. Confirm one function body serves all four.

ANSWER KEY — Topics 10-23

A. Conceptual

1. The standard *defines* subscripting as `*(E1 + E2)`. Addition commutes, so `3[a]` is legal and equals `a[3]`.
2. `sizeof(a)`, `&a`, and binding to `T(&)[N]` (plus string-literal initialisation of a char array).
3. `7, 0, 0, 0, 0`. Missing initialisers are value-initialised to zero; `{0}` only looks special because the fill value is zero.
4. The `100` is discarded — the parameter type is adjusted to `int*`. **No caller is checked and no diagnostic is required.**
5. `int (*)[5]`; `++` advances **20 bytes**.
6. `int* a[5]`: **40 bytes**, two levels of indirection to reach data, used for jagged rows / `argv` / function tables. `int (*a)[5]`: **8 bytes**, one level, used to point at a whole array or a matrix row.
7. Guaranteed copy elision constructs the returned object *directly in the caller's storage*; there is no temporary to copy from. The out-parameter version saves nothing and loses composability.
8. `std::span<const int>` — it stores a pointer and a size (16 bytes), copies nothing, and owns nothing.
9. `void f(int (&)[16])`. Anything else is a compile error at the call site.
10. Without `const` the compiler must assume writes through another pointer could alias the data, so it cannot keep values in registers or vectorise. `const` (with `__restrict` where applicable) removes that assumption.
11. `const` at namespace scope implies **internal linkage**: 40 separate 1 KB copies, 40 KB of duplicated `.rodata`. Use `inline constexpr`.
12. Because the elements of `int[3][4]` are *arrays* `int[4]`, and decay converts to a pointer to the first **element**. Casting to `int**` makes the code interpret the first 8 bytes of matrix data *as an address* — a wild pointer, usually a segfault, occasionally a silent wrong read.

B. Tracing and analysis

1. `32, 8, 32, 8, 4`. Note `a+0` forces decay while `*&a` restores the array type.
2. `7; 28; p[-3]` is `a[4]`.
3. `2 * 4 * sizeof(int) = 32 bytes`; `rp[0][1]` is `m[2][1]`.
4. 4,000,000 bytes; read+write $\approx$ 8 MB of traffic $\rightarrow$ about **650 μ s** per call at 12 GB/s (plus one allocation). 1000 calls $\approx$ **0.65 s** of pure copying.
5. **32 bytes** of pointers; the character data lives in `.rodata`, is not writable, and may be pooled so identical literals share an address.
6. `40` in `main`; `2` inside `f` (8/4). The loop would print the first **two** elements and report success.

C. What to verify

1. `400` then `8`. GCC with `-Wsizeof-array-argument` emits `warning: 'sizeof' on array function parameter 'a' will return size of 'int*'`. Without the flag: silence.
2. It prints **2** elements. All three fixes print 40. The `span` version is the only one that also works unchanged for a `vector`.
3. Immediately after the call the five values usually print **correctly** (1 2 3 4 5) — the stack has not been reused. After the intervening `printf` they are **garbage**. ASan reports **ERROR:**

AddressSanitizer: stack-use-after-return. GCC also warns at compile time: address of local variable 'a' returned.

4. Local: arbitrary. {}, {0}: all zero. {7}: **7 0 0 0 0**. After `memset(a,1,20)`: **16843009** five times, because each int is `0x01010101`. `memset` works in bytes, not elements.
5. Plain: often completes with no visible error (the reallocated buffer is still mapped). Under ASan: `ERROR: AddressSanitizer: heap-use-after-free READ of size 4`, blamed on the loop's implicit `operator++` — a line you never wrote.
6. Expect roughly **400 ms / 90 ms / 120 ms / 25 ms**. Ratios matter more than absolute values; if `cin` synced is under 150 ms your libc version already buffers better.
7. All four compile against one body. `subspan(2,3)` proves the span carries its own size rather than the container's.

HANDNOTE SUMMARY CARD — C++ Arrays, Pointers & Functions

C++ ARRAY MECHANICS (Topics 10-23)

DECLARATION AND TYPES `int a[5]`
`a` `int[5]` `sizeof 20` `a[i] == *(a+i) == i[a]`
`a+0 / a[0]` `int* / int` `sizeof 8 / 4` decay happens HERE
`&a` `int(*)[5]` `sizeof 8` `++&a` advances 20 bytes
`int* p[5]` 5 pointers, 40 bytes jagged rows, argv, fn tables
`int (*p)[5]` 1 pointer, 8 bytes `&a`, and what `int[R][C]` decays to

DECAY: array -> pointer to first element. Length is LOST.
exceptions: `sizeof(a)` | `&a` | binding to `T(&)[N]` | `char s[]="lit"`
2D decays to `int(*)[C]` -- NEVER `int**` (incompatible layouts)

INITIALISATION
`int a[5];` local -> indeterminate (UB to read) ; global -> all zero
`int a[5] = {};` all zero `int a[5] = {1,2};` -> 1 2 0 0 0
`int a[5] = {7};` -> 7 0 0 0 0 NOT five 7s -> `std::fill`
`memset(a,1,n)` writes BYTE 0x01 -> ints become 16843009

PASSING (ascending safety)
`f(int* p, int n)` caller must be honest; nullptr accepted
`f(int a[100])` identical to `int*`; the 100 is a comment
`f(int (&r)[16])` exact size, COMPILE-TIME checked
`template<N> f(int(&r)[N])` any array, N deduced, one instantiation per N
`f(span<const int>)` ptr+len, 16 bytes, takes array/vector/std::array
`f(vector<int> v)` COPIES: 4 MB and ~650 us for 10⁶ ints

RETURNING
`int* f(){int a[5]; return a;}` dangling; works until something reuses stack
vector / array / struct guaranteed elision in C++17: zero copies
`void f(span<int> out)` caller owns the buffer

CONST
`const int* p` data fixed, pointer moves (read right to left)
`int* const p` pointer fixed, data moves
`const` enables no-alias assumptions -> vectorisation, not just safety
`const` at namespace scope = internal linkage -> use `inline constexpr` in headers

I/O
`ios::sync_with_stdio(false); cin.tie(nullptr);` 400 ms -> 90 ms per 10⁶ ints
`'\n'` not `endl` (`endl` flushes) never mix `cin` and `scanf` after unsyncing

TRAPS
`sizeof(a)/sizeof(a[0]) == 2` inside a function -> `std::size / span`
`range-for` by value copies class elements -> `const auto&`
`push_back` during `range-for` invalidates iterators -> heap-use-after-free
returning `&local`: "adding a printf fixed it" -> ASAN stack-use-after-return
pointer difference is in ELEMENTS not bytes -> cast to `char*` for bytes

SECTION 5 — DYNAMIC MEMORY

24. Dynamic Arrays

A dynamic array is an array whose length is decided at runtime, so it must live on the heap. In C++ the raw mechanism is `new T[n] / delete[] p`. You should almost never use it — but you must be able to read it, because it is what `vector` does internally and what every legacy codebase does explicitly.

```
int n;
std::cin >> n;

int* a = new int[n];          // n ints, values indeterminate
int* z = new int[n]();        // n ints, ZERO-initialised (note the ())
std::fill(a, a + n, -1);

delete[] a;                   // [] is mandatory – see the trap
delete[] z;
a = nullptr;                  // makes a later delete harmless
```

```
int* a = new int[5];
```

stack	heap
+-----+	+-----+-----+-----+-----+
a ----->	header 0 1 2 3 4
+-----+	+-----+-----+-----+-----+
8 B	8-32 B 20 B of payload
	allocator bookkeeping YOU do not own and must not touch

THE TRAP

`delete p` on memory from `new[]` is undefined behaviour, not a leak. For `int` it usually appears to work; for a type with a destructor it destroys one object out of `n` and often corrupts the allocator. The symptom is a crash inside `malloc` in a *later, unrelated* allocation — the hardest class of bug in the book. ASan reports `alloc-dealloc-mismatch` immediately.

`new[]` – alloc ~50–200 ns + 8–32 B header · access O(1) · manual lifetime · no size stored for you

25. Create Dynamic Array

Four ways to obtain `n` elements at runtime, in descending order of what you should reach for first.

```
std::vector<int> v(n);          // 1. default. zero-init, RAII
auto u = std::make_unique<int[]>(n); // 2. owning, no size stored
int* raw = new int[n];          // 3. manual delete[] required
int* c = (int*)std::calloc(n, sizeof(int)); // 4. C API only; no ctors
```

`vector<int> v(n)` value-initialises: `n` zero stores, about 0.4 ms for 10^6 ints. If you are about to overwrite every element, `v.reserve(n)` plus `push_back` skips that pass — or use `make_unique_for_overwrite<int[]>(n)` (C++20) when you truly want uninitialised memory.

Form	Size stored?	Frees itself?
<code>vector<int>(n)</code>	Yes (<code>size()</code>)	Yes
<code>unique_ptr<int[]></code>	No	Yes
<code>new int[n]</code>	No	No

THE KEY INSIGHT

The allocator *does* record the block size — `delete[]` could not work otherwise — but that number is not portably reachable. You always carry the length yourself. That is the entire reason `vector` and `span` exist.

creation — `vector(n)`: 1 alloc + n zero stores (~0.4 ms/10⁶) · reserve+push_back: 1 alloc, no zero pass

26. Resize Dynamic Array

There is no "resize" at the hardware level. Growing means: allocate a bigger block, copy or move every element, free the old block. Anything that claims to resize in place is either `realloc` getting lucky (the following address range happened to be free) or an OS-level page trick.

```
int* grow(int*& a, int oldN, int newN) {
    int* b = new int[newN];
    std::copy(a, a + oldN, b);      // O(oldN) — the real cost
    delete[] a;
    a = b;                          // EVERY old pointer/iterator is now dangling
    return b;
}

// vector does exactly this, but with a growth FACTOR:
std::vector<int> v;
for (int i = 0; i < 10; ++i) { v.push_back(i);
    std::cout << v.size() << "/" << v.capacity() << " "; }
// libstdc++ doubles: 1/1 2/2 3/4 4/4 5/8 6/8 7/8 8/8 9/16 10/16
```

push_back with doubling, n = 16 elements
 reallocations at size 1,2,4,8,16 -> 5 allocations
 elements copied: 1+2+4+8 = 15 -> under 1 copy per element AMORTISED
 growth by +1 instead: 15 reallocations, 120 copies -> O(n²)

THE KEY INSIGHT

Doubling is what makes `push_back` O(1) amortised: total copies are $n + n/2 + n/4 + \dots < 2n$. Growth by a constant (+1, +10) makes it O(n²). This is the whole content of Topic 115.

THE TRAP

Reallocation invalidates *every* pointer, reference and iterator into the vector — including `int& r = v[0]`; taken ten lines earlier. The symptom is a value that is correct until the vector crosses a capacity boundary, so it fails on input size 9 and passes on 8. `reserve()` up front, or store indices rather than pointers.

growth — O(n) per reallocation, O(1) amortised with doubling · peak memory 3× during copy (old + new)

27. Delete Dynamic Array

Every `new[]` needs exactly one `delete[]` on every path out of the function — including the exception path, which is why manual deletion is unreliable and RAII (Topic 29) is not optional in production code.

```
void leaky(int n) {
    int* a = new int[n];
    if (validate(a) == false) return;    // LEAK: n*4 bytes, every call
    process(a);                          // if this throws -> LEAK
    delete[] a;
}

void fine(int n) {
    std::vector<int> a(n);                // freed on return AND on throw
    if (!validate(a.data())) return;
    process(a.data());
}
```

Mistake	Symptom	Detector
No <code>delete[]</code>	RSS grows; OOM after hours	<code>valgrind</code> , LeakSanitizer
Double <code>delete[]</code>	Crash inside the allocator, later	ASan <code>double-free</code>
<code>delete</code> vs <code>delete[]</code>	Corruption; n–1 destructors skipped	ASan <code>alloc-dealloc-mismatch</code>
Use after delete	Plausible values, then garbage	ASan <code>heap-use-after-free</code>

`delete[]` – $O(n)$ destructor calls for class types, $O(1)$ for trivial types · one `free()` · sets nothing to null

28. Dynamic Objects

An array of class objects behaves differently from an array of ints in exactly one way that matters: constructors and destructors run for every element. `new Widget[1000]` is 1000 constructor calls, and `delete[]` is 1000 destructor calls in reverse order.

```
struct Widget {
    int id;
    Widget() { /* runs 1000 times */ }
    ~Widget() { /* runs 1000 times, in REVERSE order */ }
};

Widget* w = new Widget[1000];    // 1 allocation + 1000 constructions
delete[] w;                      // 1000 destructions + 1 free

std::vector<Widget> v;
v.reserve(1000);                 // 1 allocation, ZERO constructions
v.emplace_back(42);              // constructs in place, no temporary
```

THE KEY INSIGHT

`reserve` separates *allocating storage* from *creating objects*. That separation is why a vector can have capacity 1000 and size 0, and why indexing into that gap is undefined (Topic 5) — the memory exists, the objects do not.

THE TRAP

`new Base[n]` deleted through a `Base*` when the objects are `Derived` is undefined behaviour even with a virtual destructor — array `delete[]` uses the static element size for stride, and `sizeof(Derived) != sizeof(Base)`. Symptom: correct destruction of element 0 and heap corruption after. Never make polymorphic arrays; use `vector<unique_ptr<Base>>`.

object arrays – n ctor + n dtor calls · reserve: storage without construction · polymorphic arrays: never

29. RAII and Smart Pointers

RAII means the destructor releases what the constructor acquired, so the compiler emits your cleanup on every exit path — normal return, early return, and exception unwinding. For arrays this is `vector` in 95% of cases and `unique_ptr<T[]>` in the rest.

```
{
    auto buf = std::make_unique<int[]>(n);    // owns; delete[] on scope exit
    buf[0] = 1;                               // operator[] provided for T[]
    legacy_api(buf.get(), n);                 // hand the raw pointer over
}                                              // freed here, even if we threw

std::vector<int> v(n);                        // owns + knows its size
std::span<int> view(v);                      // borrows, owns NOTHING
```

Type	Owns	Size	Overhead
<code>vector<T></code>	Yes	Yes	24 bytes (3 pointers)
<code>unique_ptr<T[]></code>	Yes	No	8 bytes, zero runtime
<code>shared_ptr<T[]></code>	Shared	No	16 bytes + atomic refcount
<code>span<T></code>	No	Yes	16 bytes

THE TRAP

`shared_ptr` increments and decrements an *atomic* counter on every copy — roughly 20–40 ns under contention, and it prevents inlining across the copy. Passing `shared_ptr` by value into a hot function is a measurable, common performance bug. Pass `const&`, or better, pass a `span`.

RAII – 0 runtime cost for `unique_ptr/vector` · `shared_ptr`: atomic inc/dec ~20–40 ns per copy

SECTION 6 — MODERN C++ ARRAY TOOLS

30. `std::array`

`std::array<T,N>` is a raw array wrapped in a struct so that it is a proper value type: it can be copied, assigned, returned, compared and put inside a `map` — none of which a C array can do. The layout is identical, there is no indirection, and `sizeof(std::array<int,5>)` is 20.

```
#include <array>
std::array<int,5> a{1,2,3,4,5};

a.size();           // 5, constexpr, survives function calls (no decay)
a.at(9);            // throws std::out_of_range
a[9];               // UB, exactly like a C array
auto b = a;         // real copy – C arrays cannot do this
b == a;             // element-wise comparison – C arrays compare POINTERS
a.data();           // int* for C APIs
static_assert(sizeof(a) == 20); // no size overhead at all
```

<code>int a[5]</code>	<code>std::array<int,5></code>
+-----+-----+-----+	struct { int _elems[5]; }
identical 20 bytes	+-----+-----+-----+
+-----+-----+-----+	identical 20 bytes
no size(), no copy,	size() constexpr, copyable, comparable,
decays to int*	does NOT decay

THE KEY INSIGHT

`std::array` costs nothing and removes decay, so the "which element count did that parameter want?" class of bug disappears. If `N` is a compile-time constant, there is no argument for a raw array in new code.

THE TRAP

`std::array<int,1000000>` as a local is still 4 MB *on the stack* — the container changes the type, not the storage class. Same segfault as Topic 7.

`std::array` – `sizeof` identical to `T[N]` · 0 overhead · `size()` constexpr · no decay · stack-resident

31. `std::vector`

`vector` is three pointers — `begin`, `end`, `end-of-capacity` — plus a heap block. That is the whole design. Everything else (growth, `push_back`, iterator invalidation) follows from those three pointers and the doubling policy in Topic 26.

```
std::vector<int> v;
v.reserve(1000); // capacity 1000, size 0, ONE allocation
v.push_back(7);  // amortised O(1)
v.insert(v.begin(), 9); // O(n): shifts every element right
v.erase(v.begin());    // O(n): shifts every element left
v.pop_back();         // O(1)
v.resize(50);         // changes SIZE (constructs/destroys objects)
v.shrink_to_fit();    // request only; may reallocate, may do nothing
int* raw = v.data();  // contiguous, guaranteed since C++03 (DR)
```

Operation	Cost	Invalidates
<code>v[i], back()</code>	$O(1)$	Nothing
<code>push_back</code>	$O(1)$ amortised	All, if it reallocates
<code>insert/erase</code> at <code>i</code>	$O(n-i)$	Everything from <code>i</code> on
<code>reserve(k)</code>	$O(n)$ once, if $k > \text{capacity}$	All
<code>clear()</code>	$O(n)$ destructors	All; capacity unchanged

THE TRAP

`clear()` does not release memory — capacity stays. A vector that peaked at 10^7 elements holds 40 MB after `clear()` for the rest of its life. The symptom is a service whose RSS never falls back after a traffic spike. Use `vector<int>().swap(v)` or `shrink_to_fit`.

vector — 24 B object + heap block · `push_back` $O(1)^*$ · `insert/erase` $O(n)$ · contiguous, C-API compatible

32. `std::span`

`std::span<T>` (C++20) is a pointer and a length, non-owning. It is the correct parameter type for "a contiguous sequence of T that I will not resize", and it makes one function body serve C arrays, `std::array`, `vector`, and slices of any of them.

```
#include <span>

long total(std::span<const int> s) {           // 16 bytes, no template
    long acc = 0;
    for (int x : s) acc += x;                  // s.size() is always correct
    return acc;
}

int c[4]{1,2,3,4}; std::array<int,3> a{5,6,7}; std::vector<int> v{8,9};
total(c); total(a); total(v);                  // all fine
total(std::span(v).subspan(1, 1));              // a 1-element window, no copy
```

```
span<int> over v = [10,20,30,40,50], subspan(1,3)
+-----+-----+-----+-----+
| 10 | 20 | 30 | 40 | 50 |    v owns the memory
+-----+-----+-----+-----+
  ^-----^
                                span: {ptr=&v[1], size=3}. Copies nothing.
```

THE TRAP

A span is a borrowed view: it dangles the instant the owner reallocates or dies. `span<int> s(v); v.push_back(1);` leaves `s` pointing at freed memory, and there is no warning. Treat a stored span like a raw pointer — fine as a parameter, dangerous as a member.

span — 16 bytes, 0 allocations · $O(1)$ construction and subspan · non-owning: lifetime is yours to prove

33. std::valarray

`valarray` is a numeric array supporting whole-array arithmetic: `a + b * 2.0` operates element-wise. It was designed in 1994 for vectorisable numeric code, largely failed to be adopted, and today loses to `vector` plus an algorithm, or to Eigen/BLAS for real linear algebra. Know it so you can recognise it in old code and not choose it in new code.

```
#include <valarray>
std::valarray<double> a{1,2,3,4}, b{10,20,30,40};

std::valarray<double> c = a * 2.0 + b;    // element-wise: 12 24 36 48
double s = c.sum();                      // 120
std::valarray<double> even = c[std::slice(0, 2, 2)]; // stride-2 slice
```

THE KEY INSIGHT

Say plainly when a tool is the wrong choice: use `vector` + `<algorithm>/<numeric>` for general code, and Eigen or BLAS when the maths matters. `valarray`'s expression results are unspecified proxy types, so `auto c = a + b;` can dangle — a defect no one is going to fix.

`valarray` – element-wise ops $O(n)$, temporaries per operation · `auto` with expressions: dangling proxies

34. C-Style Arrays vs Modern Containers

Feature	<code>T a[N]</code>	<code>array</code>	<code>vector</code>	<code>span</code>
Size in type	Yes (until decay)	Yes	No	Optional
Knows size in a function	No	Yes	Yes	Yes
Owns memory	Yes	Yes	Yes	No
Resizable	No	No	Yes	No
Copyable / comparable	No	Yes	Yes	Shallow
Bounds-checked option	No	<code>.at()</code>	<code>.at()</code>	C++26
Object size	$N \cdot \text{sizeof}(T)$	$N \cdot \text{sizeof}(T)$	24 B + heap	16 B
Where it lives	Stack/BSS	Stack/BSS	Heap	Elsewhere

THE KEY INSIGHT

The decision procedure, in order: *size fixed at compile time and small?* `std::array`. *Size runtime or growing?* `vector`. *Function parameter?* `span`. *Talking to a C API?* `.data()`. A raw `T[]` in new code needs a written justification — usually "this is a fixed protocol frame" or "no heap exists on this device."

selection – layout identical across all four · the difference is metadata and ownership, never speed of `a[i]`

SECTION 7 — STRINGS & CHARACTER ARRAYS

35. Character Arrays

A character array is an array of bytes; everything in Sections 1-6 applies unchanged. The only new rule is the convention that a NUL byte (`'\0'`, value 0) marks the end — a convention, not a language feature, which is why it fails so loudly when broken.

```
char a[6] = "hello";           // 6 bytes: 'h','e','l','l','o','\0' - fits exactly
char b[5] = "hello";           // ERROR in C++: no room for the NUL
char c[] = "hello";             // size deduced 6
char d[10] = "hi";              // 'h','i','\0', then SEVEN zero bytes
char e[3] = {'a','b','c'};      // legal, NOT a string - no terminator

std::cout << sizeof(c) << " " << std::strlen(c);    // 6 5
```

```
char c[] = "hello";
+-----+-----+-----+
| h | e | l | l | o | \0 |   sizeof = 6 (bytes of storage)
+-----+-----+-----+   strlen = 5 (bytes before the NUL, 0(n) scan)
 0   1   2   3   4   5
```

THE TRAP

`sizeof` is a compile-time property of the array; `strlen` is a runtime $O(n)$ scan for a zero byte. Calling `strlen` inside a loop condition makes an $O(n)$ loop $O(n^2)$ — a real pattern in C code that processes megabyte buffers and appears to "hang".

char arrays – `sizeof` $O(1)$ compile-time · `strlen` $O(n)$ scan · 1 byte per element, 64 per cache line

36. C-Style Strings

The C string functions are the historical source of most buffer overflow vulnerabilities (Topic 127), because they take a destination pointer with no length and trust that the source has a NUL.

```
char dst[8];
std::strcpy(dst, "this is far too long");    // 21 bytes into 8: OVERFLOW
std::strncpy(dst, src, sizeof dst);          // bounded, but MAY NOT terminate
snprintf(dst, sizeof dst, "%s", src);        // bounded AND always terminates

// Best: do not use any of them.
std::string s = src;                         // owns, grows, always terminated
std::string_view v = src;                    // borrows; NOT NUL-terminated
```

Function	Cost	Failure mode
<code>strlen</code>	$O(n)$	Runs off the end if no NUL
<code>strcpy</code>	$O(n)$	Unbounded write — the classic CVE
<code>strncpy</code>	$O(n)$	Truncates <i>without</i> a NUL; also pads to n
<code>strcat</code>	$O(n+m)$	Rescans dst every call → $O(n^2)$ in a loop
<code>snprintf</code>	$O(n)$	Safe; returns the length it <i>wanted</i>

THE TRAP

`strncpy(dst, src, 8)` with an 8-character source leaves `dst` with no terminator. The next `printf("%s")` keeps reading into whatever follows — often the adjacent stack variable, which is how "my password appeared in the log line" happens. Prefer `snprintf`, or `std::string`.

C strings — every op $O(n)$ rescan · `strcat` in a loop $O(n^2)$ · no bounds carried: the caller is the only check

37. Arrays of Strings

Three layouts, with very different memory behaviour. Choosing badly here costs either memory or cache misses at scale.

```
const char* p[3] = {"ada", "grace", "alan"}; // 24 B of ptrs + .rodata bodies
char m[3][8] = {"ada", "grace", "alan"}; // 24 B FLAT, padded, contiguous
std::vector<std::string> v = {"ada", "grace", "alan"}; // 3 x 32 B + heap bodies
```

// argv is exactly layout 1:

```
int main(int argc, char* argv[]) { /* argv[i] is char*, argv[argc]==nullptr */ }
```

layout 1 (ptrs)	[p0][p1][p2] --> scattered bodies : 1 extra load per access
layout 2 (flat)	[ada\0\0\0\0\0][grace\0\0][alan\0\0\0\0] : 1 load, no chase wastes (8 - len - 1) bytes per row but scans at full speed
layout 3 (string)	32 B header each; SS0 keeps <=15 chars INSIDE the header

THE KEY INSIGHT

`libstdc++` `std::string` uses Small String Optimisation: strings up to 15 characters live inside the 32-byte object with no allocation at all. That is why a `vector<string>` of short words is far faster than the pointer-chasing picture suggests — and why crossing 15 characters can double your allocation count.

string arrays — flat: 1 load, padding waste · ptr array: +1 dependent load · `vector<string>`: 32 B each, SS0 ≤15 chars

PRACTICE SET — Topics 24-37

A. Conceptual

1. Why is `delete` on a `new[]` block undefined behaviour rather than merely a leak of the extra elements?
2. Explain, with the sum of a geometric series, why doubling makes `push_back` $O(1)$ amortised and growing by $+1$ makes it $O(n^2)$.
3. A vector's `clear()` leaves RSS unchanged. Why, and what actually returns the memory?
4. What exactly does `reserve(1000)` do to `size()`, `capacity()`, the number of allocations, and the number of constructor calls?
5. Give the peak memory multiplier during a vector reallocation, and explain why an 8 GB machine cannot always grow a 5 GB vector.
6. Why does `std::array<int,5>` not decay while `int[5]` does, given identical layout?
7. Name two things `std::array` can do that a C array cannot, and one thing it does *not* fix.
8. When is `span` the wrong type to store as a class member?
9. Give the difference between `sizeof(s)` and `strlen(s)` for `char s[] = "hi"`, and describe the loop bug that makes an $O(n)$ scan $O(n^2)$.
10. Why can `strncpy` leave you worse off than `strcpy` in a specific edge case?
11. Explain Small String Optimisation and the allocation cliff at 15 characters in `libstdc++`.
12. Someone proposes `new Base[n]` with a virtual destructor for a polymorphic collection. Give the mechanical reason this is broken and the correct replacement.

B. Tracing and analysis

1. Starting from an empty `libstdc++` vector, list capacity after each of 10 `push_back` calls, then give the total number of reallocations and the total number of element copies.
2. Repeat question 1 for a growth policy of $+2$ elements. Give reallocation count and total copies for $n = 10$ and the Big-O for general n .
3. `vector<int> v(1'000'000)`: how many bytes on the heap, how many zero stores, and roughly how long at 10 GB/s?
4. Give `sizeof` for: `std::array<int,5>`, `std::vector<int>` (`libstdc++`), `std::span<int>`, `std::unique_ptr<int[]>`, `std::shared_ptr<int[]>` and `std::string`.
5. Compare memory for 1000 words of average length 6, stored as `vector<string>` versus `char[1000][8]`. Count bytes for both, including SSO behaviour and heap headers.
6. `char d[10] = "hi";` — give the 10 byte values, `sizeof(d)` and `strlen(d)`.

C. Code tasks

1. **Watch the doubling.** Print `size()`, `capacity()` and `(void*)v.data()` after each of 20 `push_back` calls. Note every iteration where the pointer changes and confirm it matches the capacity jumps.
2. **Break it: the dangling reference across growth.** Take `int& r = v[0];` on a vector of size 4 with capacity 4, then `push_back`, then read `r`. Record the value. Repeat with `reserve(100)` beforehand and record it again. Then run the first version under ASan and record the error name and the allocation stack it prints.
3. **Break it: alloc/dealloc mismatch.** Allocate with `new int[10]` and free with `delete`. Run plain, note whether anything happens. Then run under ASan and paste the first line. Repeat with a class type that has a destructor printing its id, and count how many destructors ran.

4. **Break it: clear() does not free.** Push 10^7 ints, print `capacity()` and the process RSS from `/proc/self/status` (VmRSS), call `clear()`, print both again, then `shrink_to_fit()` and print again. Report the three RSS numbers.
5. **One function, four containers.** Write `long total(std::span<const int>)` and call it with a C array, `std::array`, `vector`, and a `subspan`. Then make `span` dangle deliberately: build it from a `vector`, `push_back`, and read through it under ASan.
6. **SSO cliff.** Instrument `operator new` with a global counter. Build a `vector<string>` of 1000 strings of length 15, then length 16. Report both allocation counts.
7. **strncpy termination.** Copy an 8-character source into `char dst[8]` with `strncpy`, then `printf("%s")` it. Record what prints. Fix with `snprintf` and compare.

ANSWER KEY — Topics 24-37

A. Conceptual

1. The two forms use different deallocation paths: `new[]` may store an element count in a cookie ahead of the returned pointer, so `delete` frees the *wrong address* and runs one destructor instead of `n`. Result is heap corruption, not a leak.
2. Doubling: copies are $n/2 + n/4 + \dots < n$, so **< 2 copies per element amortised**. Growth by `k`: reallocations n/k , copies $k + 2k + \dots + n \approx n^2/2k = \Theta(n^2)$.
3. `clear()` destroys objects and sets `size = 0`; **capacity is unchanged** so the block is still held. Only `shrink_to_fit()` or `vector<T>().swap(v)` returns it.
4. `size()` unchanged (0); `capacity()` becomes 1000; **one** allocation; **zero** constructor calls.
5. **~3×** at the instant of copy: old block (`n`) + new block (`2n`). A 5 GB vector needs $5 + 10 = 15$ GB to grow, so it fails on an 8 GB machine long before "memory is full".
6. Decay is a rule about the *array type*, not about layout. `std::array` is a class type, so no implicit conversion to `T*` exists; you must ask via `.data()`.
7. Can: be copied/assigned/returned, compared element-wise, used as a map key, report `size()` inside a function. Does not fix: **bounds checking on operator[]** (still UB) or stack size limits.
8. As a member, whenever the owner can outlive-or-predecease independently or can reallocate — a stored span is exactly as dangerous as a stored raw pointer, with none of the ownership.
9. `sizeof = 3` (compile-time storage), `strlen = 2` (runtime scan). Putting `strlen(s)` in the loop condition rescans every iteration: **$O(n^2)$** .
10. If the source is exactly `n` characters, `strncpy` copies `n` bytes and writes **no NUL**; the next read runs past the buffer. It also zero-pads short sources, which is $O(n)$ even for a 2-byte copy.
11. Strings up to **15 characters** are stored inside the 32-byte object; at 16 the string heap-allocates. 1000 short words = **0** allocations; 1000 sixteen-character words = **1000** allocations.
12. `delete[]` strides by `sizeof(Base)`, but the objects are `sizeof(Derived)` apart. Element 1 onward are computed at wrong addresses. Use `vector<unique_ptr<Base>>`.

B. Tracing and analysis

1. Capacities: **1, 2, 4, 4, 8, 8, 8, 8, 16, 16**. 5 reallocations (at sizes 1, 2, 4, 8, 16); copies **$1+2+4+8 = 15$** .
2. Capacities 2,2,4,4,6,6,8,8,10,10: 5 reallocations, copies $0+2+4+6+8 = 20$. General: **$\Theta(n^2)$** .
3. **4,000,000 bytes**; **10^6 zero stores**; about **0.4 ms** at 10 GB/s.
4. **20, 24, 16, 8, 16, 32** bytes respectively (libstdc++, x86-64).
5. `vector<string>`: $1000 \times 32 \text{ B} = \mathbf{32,000 \text{ B}}$ and **zero** body allocations ($6 \leq 15$, SSO), plus one 32 KB block. `char[1000][8]`: **8,000 B** flat, contiguous, no headers — 4× smaller and scannable, at the cost of an 7-character hard limit.
6. `'h','i','\0'` then **seven** zero bytes; `sizeof = 10`, `strlen = 2`.

C. What to verify

1. The data pointer changes exactly at sizes **1, 2, 3, 5, 9, 17** (the push that exceeds capacity). Between those points the pointer is stable — that stability is what `reserve` buys.

2. Without `reserve`, `r` reads freed memory: often the old value, sometimes garbage. With `reserve(100)` it is **correct and stays correct**. ASan: `heap-use-after-free READ of size 4`, plus a `freed by thread T0 here` stack that points at `push_back` — the line that "shouldn't" be involved.
3. Plain: usually **no visible symptom** for `int`. ASan: `ERROR: AddressSanitizer: alloc-dealloc-mismatch (operator new [] vs operator delete)`. With the destructor class, expect **1** destructor to run instead of 10 (and frequently a crash after).
4. Expect roughly: after push, capacity $\geq 10,000,000$ and VmRSS about **40 MB** above baseline; after `clear()`, capacity unchanged and RSS **unchanged**; after `shrink_to_fit()`, RSS drops back to near baseline.
5. All four calls compile and print the right totals. The dangling case gives `heap-use-after-free` — proving a span borrows and never extends lifetime.
6. **0** allocations at length 15; **1000** at length 16 (plus the vector's own growth allocations, which you should subtract by reserving).
7. `printf` prints the 8 characters **plus whatever follows in memory** until a zero byte — commonly other locals. `snprintf` prints 7 characters and terminates.

HANDNOTE SUMMARY CARD — Heap, Containers & Strings

DYNAMIC MEMORY + MODERN CONTAINERS (Topics 24-37)

RAW HEAP

pair every new[] with delete[]
new int[n] indeterminate new int[n]() zero-initialised
delete[] p n dtors + 1 free delete p UB: alloc-dealloc-mismatch
cost: 50-200 ns per allocation + 8-32 B invisible header per block

GROWTH (why push_back is O(1) amortised)

doubling: copies = $n/2 + n/4 + \dots < n$ -> under 2 moves per element
+k growth: copies = $n^2/2k$ -> $O(n^2)$. Never do this.
libstdc++ capacities: 1 2 4 4 8 8 8 8 16 16 ...
peak memory during realloc = 3x (old n + new 2n)
EVERY pointer/reference/iterator dies on realloc -> reserve() or use indices

CONTAINER TABLE

	sizeof	owns	size	resizable	where
int a[N]	N*T	yes	type	no	stack/BSS
std::array<T,N>	N*T	yes	type	no	stack/BSS
std::vector<T>	24	yes	yes	yes	heap
std::span<T>	16	NO	yes	no	borrows
std::unique_ptr<T[]>	8	yes	no	no	heap
std::shared_ptr<T[]>	16	shared	no	no	heap + atomics
std::string	32	yes	yes	yes	SSO <=15 chars

CHOOSE

N constant -> array runtime N -> vector parameter -> span
C API -> .data() no heap -> raw T[] polymorphic -> vector<unique_ptr>

VECTOR COSTS

v[i] back() O(1) | push_back O(1)* | pop_back O(1)
insert/erase at i O(n-i) | reserve O(n) once | clear O(n) dtors, capacity KEPT

C STRINGS

convention, not a language feature: trailing '\0'
sizeof compile-time storage strlen O(n) scan for the zero byte
strcpy unbounded write strncpy may NOT terminate; pads to n
strcat rescans dst -> $O(n^2)$ in a loop snprintf: bounded + terminated
char m[3][8] flat: 24 B, 1 load const char* p[3]: 24 B + chase + bodies

TRAPS

delete vs delete[] -> crash inside malloc LATER, unrelated code
clear() keeps capacity -> RSS never falls after a traffic spike
span/iterator after growth -> heap-use-after-free, no compiler warning
new Base[n] polymorphic -> wrong stride from element 1 on
shared_ptr by value -> atomic inc/dec 20-40 ns in the hot path
strlen in a loop condition -> $O(n)$ becomes $O(n^2)$ on MB buffers

PART II CHEATSHEET — C++ Array Toolkit

PART II AT A GLANCE: WHAT TO WRITE, AND WHY

THE FOUR DECISIONS, IN ORDER

- | | | |
|---------------------|--------------------------|-------------------------------|
| 1. WHO OWNS IT? | caller owns -> span | callee owns -> vector/array |
| 2. WHEN IS N KNOWN? | compile -> array<T,N> | runtime -> vector<T> |
| 3. DOES IT GROW? | yes -> vector (reserve!) | no -> array / unique_ptr<T[]> |
| 4. CROSSING AN API? | C -> .data()+size | C++ -> span<const T> |

THE SIX IDIOMS WORTH MEMORISING

```
void f(std::span<const int> s)           // one body, all containers
template<class T, size_t N> size_t len(const T(&)[N]) { return N; }
std::vector<int> v; v.reserve(n);        // then push_back
auto p = std::make_unique<int[]>(n);     // owning buffer, no size
std::fill(std::begin(a), std::end(a), value); // NOT = {value}
snprintf(dst, sizeof dst, "%s", src);    // bounded and terminated
```

COST MODEL FOR PART II

operation	cost	notes
a[i]	O(1) 1 instr	identical for all containers
push_back	O(1) amortised	O(n) on the realloc push
insert/erase middle	O(n-i)	memmove of the tail
vector copy (10 ⁶ ints)	~650 us + 1 alloc	pass by const& or span
new/delete	50-200 ns each	the alloc, not the memory
shared_ptr copy	20-40 ns	atomic RMW, blocks inlining
strlen(1 MB buffer)	~60 us	and O(n ²) if in a condition

WHEN A RAW ARRAY IS STILL RIGHT

- fixed protocol frames where the size IS the contract: uint8_t hdr[20]
- embedded targets with no heap and a linker-checked BSS budget
- constexpr lookup tables (Topic 158) baked into .rodata
- the innermost buffer of a container you are implementing yourself

WHAT PART II BUYS YOU LATER

span	-> every algorithm in Sections 8-11 takes one parameter
reserve	-> Topic 115 amortised analysis is now obvious
contiguity	-> Topics 116-122 (cache, SIMD) apply to vector, not to list
.data()	-> Topics 162-164 (mmap, GPU) need a raw contiguous pointer

CROSS-REFERENCES OUT OF PART II

growth/doubling	-> Topic 115	decay -> Topics 18, 90, 157
RAII	-> Topics 126-128 (leaks, dangling)	
SSO	-> Topic 37, 145 (feature vectors of strings)	
span	-> Topic 156 (ranges), 32 (subspan windows in sliding window)	

PART III

Operations, Patterns & Sorting

The working vocabulary: the five primitive operations, rotation, the eight interview patterns that cover most array problems, and the sorting algorithms they assume. Sections 8-11 · Topics 38-89.

SECTION 8 — BASIC ARRAY OPERATIONS

38. Basic Array Operations

Five operations, and one rule that explains all their costs: **anything that changes the position of elements is $O(n)$, because contiguity must be restored.** Nothing else in this section is new information.

Operation	Cost	Why
Access <code>a[i]</code>	$O(1)$	Address computed (Topic 8)
Update <code>a[i]=v</code>	$O(1)$	One store; no neighbours move
Search (unsorted)	$O(n)$	No ordering to exploit
Search (sorted)	$O(\log n)$	Halving (Topic 42)
Insert at <code>i</code>	$O(n-i)$	Tail slides right
Delete at <code>i</code>	$O(n-i)$	Tail slides left
Append (with capacity)	$O(1)$	Nothing moves

The asymmetry between "insert at the end" ($O(1)$) and "insert at the front" ($O(n)$) is the single most useful fact for choosing a structure. If your workload is front-insertion dominated, `std::deque` or a linked list wins outright; if it is index-dominated, nothing beats the array.

cheap		expensive
<code>a[i]</code> read/write	append (spare capacity)	insert/delete at 0
$O(1)$	$O(1)$	$O(n)$
1 instruction	1 store + size++	memmove of <code>n</code> elements

THE KEY INSIGHT

"Restoring contiguity" is the only reason middle operations are expensive. It is also why they are *fast in practice*: memmove of 1000 ints is one vectorised loop at ~ 10 GB/s, so an $O(n)$ shift can beat an $O(1)$ linked-list splice for `n` up to a few thousand. Big-O and wall clock disagree here, and the wall clock is what ships.

primitives – access/update $O(1)$ · search $O(n)/O(\log n)$ · insert/delete $O(n-i)$ · append $O(1)$
amortised

39. Insert at Any Index

```
// Insert v at index pos in a[0..len-1]; capacity must exceed len.
bool insertAt(int* a, int& len, int cap, int pos, int v) {
    if (pos < 0 || pos > len || len == cap) return false;
    for (int i = len; i > pos; --i)        // BACKWARDS: see the trap
        a[i] = a[i - 1];
    a[pos] = v;
    ++len;
    return true;
}
// std::vector equivalent: v.insert(v.begin() + pos, value);
```

```

insert 99 at pos 1, len 4, cap 6
start   [10, 20, 30, 40, _, _]
i=4     [10, 20, 30, 40, 40, _]    a[4] = a[3]
i=3     [10, 20, 30, 30, 40, _]    a[3] = a[2]
i=2     [10, 20, 20, 30, 40, _]    a[2] = a[1]
write   [10, 99, 20, 30, 40, _]    a[1] = 99      moves = len - pos = 3

```

THE TRAP

Shifting forwards (`for (i = pos; i < len; ++i) a[i+1] = a[i];`) copies the same element repeatedly, producing `[10,20,20,20,20]`. The symptom is a smeared array where every element after the insertion point equals the first one shifted. Always move from the far end towards `pos`, which is exactly what `memmove` does and `memcpy` does not.

insert – Time $O(n-pos)$, worst $O(n)$ at pos 0 · Space $O(1)$ · fails if `len == cap` (Topic 26)

40. Delete from Any Index

```

bool eraseAt(int* a, int& len, int pos) {
    if (pos < 0 || pos >= len) return false;
    for (int i = pos; i < len - 1; ++i) // FORWARDS this time
        a[i] = a[i + 1];
    --len;                               // a[len] is now stale, not cleared
    return true;
}

// O(1) unordered delete – legal only if order does not matter:
void eraseFast(int* a, int& len, int pos) { a[pos] = a[--len]; }

```

The unordered variant ("swap with the last, shrink") turns an $O(n)$ delete into $O(1)$ and appears constantly in game engines and particle systems, where the collection is a set, not a sequence. Recognising that your data has no meaningful order is worth a factor of n .

THE TRAP

`v.erase(it)` inside a loop over `v` invalidates `it` and everything after it. The classic wrong fix is to `++it` anyway, which skips an element — so deleting from `[2,2,3]` leaves `[2,3]`. Use `it = v.erase(it);` without incrementing, or the erase-remove idiom (Topic 136).

delete – ordered $O(n-pos)$ · unordered swap-with-last $O(1)$ · erase in a loop invalidates from pos onward

41. Linear Search

```

int linearSearch(const int* a, int n, int key) {
    for (int i = 0; i < n; ++i)
        if (a[i] == key) return i;    // first occurrence
    return -1;
}

// std::find(a, a+n, key) – same loop, vectorised for trivial types

```

Average comparisons on a hit: $(n+1)/2$. On a miss: n , always. For 10^6 ints this is about $250\ \mu\text{s}$ — and it is *memory-bandwidth bound*, not compare-bound, so it runs at roughly 10 GB/s regardless of how cheap the comparison is.

THE KEY INSIGHT

Linear search on a contiguous array beats a hash lookup for small n — typically up to $n \approx 8$ –30 for ints — because the hash, the modulo and the probe cost more than a fully prefetched scan of two cache lines. "Use a hash map" is wrong advice below roughly 16 elements.

linear search – Time $O(n)$, avg $n/2$ on hit, n on miss · Space $O(1)$ · ~ 10 GB/s, bandwidth-bound

42. Binary Search

Requires a sorted array and turns $O(n)$ into $O(\log_2 n)$: 20 comparisons for 10^6 elements, 30 for 10^9 . The whole difficulty is in the invariant, so write it once, correctly, and reuse it.

```
// Returns index of key, or -1. Half-open [lo, hi).
int binarySearch(const int* a, int n, int key) {
    int lo = 0, hi = n;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;    // NOT (lo+hi)/2 – overflow
        if (a[mid] == key) return mid;
        if (a[mid] < key) lo = mid + 1;  // discard left half INCLUSIVE
        else                hi = mid;    // discard right half EXCLUSIVE
    }
    return -1;
}
// Boundary versions: std::lower_bound (first >= key),
//                    std::upper_bound (first > key)
```

```
a = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]    key = 23    n = 10
lo=0 hi=10 mid=5 a[5]=23 -> FOUND at index 5    1 comparison

key = 40:
lo=0 hi=10 mid=5 23 < 40 -> lo=6
lo=6 hi=10 mid=8 72 > 40 -> hi=8
lo=6 hi=8 mid=7 56 > 40 -> hi=7
lo=6 hi=7 mid=6 38 < 40 -> lo=7 -> lo==hi, return -1    4 comparisons
```

THE TRAP

Three bugs, all classics. (1) `(lo+hi)/2` overflows for $n > 2^{30}$ — the bug that sat in the JDK for nine years. (2) Mixing `hi = mid` with `hi = n-1` initialisation gives an infinite loop when `lo == hi-1`. (3) Binary searching an *unsorted* array returns a plausible wrong index instead of failing. Symptom for (3): correct on sorted test data, wrong in production.

binary search – Time $O(\log n)$: 20 steps @ 10^6 · Space $O(1)$ · cache-hostile: $\sim \log n$ misses vs 1 per 16 in a scan

43. Update Element

The only $O(1)$ mutation: one store, no neighbours affected, no size change. It is worth stating explicitly because it is the operation that makes arrays the right structure for *lookup tables* (Topic 139), *frequency counts* (Topic 49) and *difference arrays* (Topic 62) — all of which are "update in place at a computed index."

```

a[i] = v;           // 0(1), one store
++freq[x];         // read-modify-write, still 0(1)
v.at(i) = v;        // 0(1) + one compare; throws instead of corrupting

```

update – Time $O(1)$, one store · Space $O(1)$ · no invalidation of any iterator

44. Reverse an Array

```

void reverse(int* a, int n) {
    for (int i = 0, j = n - 1; i < j; ++i, --j)
        std::swap(a[i], a[j]);    // n/2 swaps,  $O(1)$  space
}
// std::reverse(a, a + n) – identical, vectorised

```

[1, 2, 3, 4, 5]	i < j is the ONLY correct condition
i j	swap -> [5, 2, 3, 4, 1]
i j	swap -> [5, 4, 3, 2, 1]
i=j	stop. odd middle element needs no swap.

THE KEY INSIGHT

Reversal is the primitive behind rotation (Topic 57), palindrome checks, next permutation (Topic 73) and in-place matrix rotation (Topic 104). Three reversals rotate an array; one reversal of a suffix completes a next-permutation. Learn it as a building block, not as an exercise.

THE TRAP

Writing `i <= j` swaps the middle element with itself (harmless) — but writing `i != j` loops forever on even `n`, because the pointers cross without ever being equal. Symptom: a hang on even-length input only.

reverse – Time $O(n)$, $n/2$ swaps · Space $O(1)$ · sequential access both directions, fully prefetched

45. Merge Two Arrays

```

// Unsorted concatenation: copy both,  $O(n+m)$ .
std::vector<int> concat(const std::vector<int>& a,
                       const std::vector<int>& b) {
    std::vector<int> out;
    out.reserve(a.size() + b.size());    // one allocation, not log n
    out.insert(out.end(), a.begin(), a.end());
    out.insert(out.end(), b.begin(), b.end());
    return out;                          // elided, no copy
}

```

concat – Time $O(n+m)$ · Space $O(n+m)$ · with reserve: 1 allocation; without: $\sim \log_2(n+m)$ reallocations

46. Merge Two Sorted Arrays

The two-pointer merge is the core of merge sort (Topic 78) and of every "combine sorted streams" problem. One pass, no comparisons wasted.

```
std::vector<int> mergeSorted(const std::vector<int>& a,
                           const std::vector<int>& b) {
    std::vector<int> out;
    out.reserve(a.size() + b.size());
    std::size_t i = 0, j = 0;
    while (i < a.size() && j < b.size())
        out.push_back(a[i] <= b[j] ? a[i++] : b[j++]); // <= keeps STABILITY
    while (i < a.size()) out.push_back(a[i++]);
    while (j < b.size()) out.push_back(b[j++]);
    return out;
}
```

```
a = [1, 4, 9]    b = [2, 3, 10]
i=0 j=0  1 <= 2  -> take a[0]=1  out=[1]
i=1 j=0  4 > 2   -> take b[0]=2  out=[1,2]
i=1 j=1  4 > 3   -> take b[1]=3  out=[1,2,3]
i=1 j=2  4 <= 10 -> take a[1]=4  out=[1,2,3,4]
i=2 j=2  9 <= 10 -> take a[2]=9  out=[1,2,3,4,9]
a exhausted -> drain b           out=[1,2,3,4,9,10]  comparisons: 5
```

Merging in place, backwards is the variant interviewers ask for: if **a** has $m + n$ capacity with the first m used, fill from the back so nothing is overwritten before it is read — the same "shift from the far end" discipline as Topic 39.

THE TRAP

Using **<** instead of **<=** when values tie takes from **b** first and silently destroys stability. It passes every test with distinct values and fails on real records sorted by a secondary key.

merge — Time $O(n+m)$, $n+m-1$ comparisons worst · Space $O(n+m)$, or $O(1)$ merging backwards in place

47. Check if Array is Sorted

```
bool isSorted(const int* a, int n) {
    for (int i = 1; i < n; ++i)
        if (a[i - 1] > a[i]) return false; // > allows duplicates
    return true;                          // n <= 1 is sorted
}
// std::is_sorted(a, a+n)
```

Use **>** for non-decreasing (duplicates allowed) and **>=** for strictly increasing. Getting this backwards is how a "sorted" check rejects **[1,1,2]** — which is sorted by every definition that matters for binary search.

is_sorted — Time $O(n)$, early exit on first inversion · Space $O(1)$ · verifies binary search's precondition

48. Find Min and Max

```
// Naive: 2(n-1) comparisons. Paired: 3n/2 - 2 comparisons.
std::pair<int,int> minMax(const int* a, int n) {
    int lo = a[0], hi = a[0];
    int i = 1;
    if (n % 2 == 0) { lo = std::min(a[0], a[1]); hi = std::max(a[0], a[1]);
                     i = 2; }
    for (; i + 1 < n; i += 2) { // compare the PAIR first
        int s = std::min(a[i], a[i+1]), l = std::max(a[i], a[i+1]);
        lo = std::min(lo, s); hi = std::max(hi, l); // 3 per 2 elements
    }
    if (i < n) { lo = std::min(lo, a[i]); hi = std::max(hi, a[i]); }
    return {lo, hi};
}
```

THE KEY INSIGHT

Pairing saves 25% of comparisons ($3n/2$ versus $2n$) and is a classic interview follow-up. On modern hardware it rarely saves *time* — the loop is bandwidth-bound and `min/max` compile to branchless `cmov`. Know the theory; measure before deploying it.

THE TRAP

Initialising with `int lo = INT_MAX, hi = INT_MIN` and then returning them on an empty array reports a range of `[2147483647, -2147483648]` rather than failing. Handle `n == 0` explicitly — it is the first row of every test matrix in this book.

min/max — Time $O(n)$: $2n-2$ naive, $3n/2-2$ paired · Space $O(1)$ · empty input must be an explicit case

49. Count Frequency

The frequency array is the cheapest data structure in competitive programming: if the value range is small, indexing beats hashing by 5–10×.

```
// Values known to be in [0, K): direct indexing,  $O(n + K)$ 
int freq[256] = {};
for (int i = 0; i < n; ++i) ++freq[a[i]]; // one store per element

// Values sparse or unbounded: hash map,  $O(n)$  expected but ~5-10x slower
std::unordered_map<int,int> m;
for (int x : a) ++m[x]; // hash + probe per element
```

```
a = [3,1,3,3,0]   K = 4
      freq: index 0 1 2 3
              value 1 1 0 3   one pass, one store each, no comparisons
choose direct indexing when  $K \leq \sim 10^7$  ints (40 MB); hash map beyond that
```

THE TRAP

`++freq[a[i]]` with a negative or out-of-range value writes *before* the array or past it — silently, into other locals. This is how a counting sort (Topic 81) on data containing `-1` corrupts an unrelated variable. Offset the index (`freq[x + OFFSET]`) and validate the range first.

frequency – direct: Time $O(n+K)$, Space $O(K)$, ~1 ns/element · hash: Time $O(n)$ expected, ~5–10 ns/element

50. Remove Duplicates

```
// Sorted input, in place: the canonical two-pointer write-index pattern.
int dedupSorted(int* a, int n) {
    if (n == 0) return 0;
    int w = 1;                                // write index
    for (int r = 1; r < n; ++r)                // read index
        if (a[r] != a[w - 1]) a[w++] = a[r];
    return w;                                // new logical length
}
// Unsorted, order-preserving:  $O(n)$  time,  $O(n)$  space with a hash set.
// Unsorted, order-free:      sort first,  $O(n \log n)$ ,  $O(1)$  space.
```

```
a = [1,1,2,2,2,3]
r=1 a[1]=1 == a[0] skip      w=1 [1,...]
r=2 a[2]=2 != a[0] write a[1]=2 w=2 [1,2,...]
r=3,4 duplicates skip      w=2
r=5 a[5]=3 != a[1] write a[2]=3 w=3 [1,2,3,|2,2,3] -> length 3
tail beyond w is untouched garbage, not erased – that is the contract
```

THE KEY INSIGHT

The read-pointer/write-pointer pair is the same machinery as `std::remove`, move-zeros (Topic 51) and partitioning (Topic 58). None of these algorithms erase anything — they compact the survivors to the front and return a new length. The caller does the erasing. This is the erase-remove idiom (Topic 136).

dedup – sorted: Time $O(n)$, Space $O(1)$ · unsorted+order: $O(n)$ time, $O(n)$ space · sort-first: $O(n \log n)$, $O(1)$

51. Move Zeros

```
// Move all zeros to the end, preserving the order of non-zeros.  $O(n)$ ,  $O(1)$ .
void moveZeros(int* a, int n) {
    int w = 0;
    for (int r = 0; r < n; ++r)
        if (a[r] != 0) std::swap(a[w++], a[r]); // swap keeps values alive
    // Variant: assign a[w++] = a[r], then fill a[w..n) with 0 – same count.
}
```

```
[0, 1, 0, 3, 12]
r=0 zero, skip      w=0 [0,1,0,3,12]
r=1 swap a[0],a[1]  w=1 [1,0,0,3,12]
r=2 zero, skip      w=1
r=3 swap a[1],a[3]  w=2 [1,3,0,0,12]
r=4 swap a[2],a[4]  w=3 [1,3,12,0,0] done, order kept
```

THE KEY INSIGHT

Move-zeros *is* stable partition with the predicate "is non-zero". Once you see that, Topics 50, 51, 58 and 59 are one algorithm with four predicates, and you stop memorising them separately.

move zeros – Time $O(n)$, $\leq n$ swaps · Space $O(1)$ · stable for non-zeros; zeros' relative order is not preserved

PRACTICE SET — Topics 38-51

A. Conceptual

1. Give the one rule that explains why access is $O(1)$ and middle insertion is $O(n)$, in terms of the array's defining property.
2. Why must insertion shift from the far end towards the insertion point, and which standard library function encodes that rule?
3. When is `a[pos] = a[--len]` a legal deletion, and what does it cost you?
4. State the precondition of binary search and describe what happens — precisely — when it is violated.
5. Why is `lo + (hi-lo)/2` preferred over `(lo+hi)/2`, and at what array size does the difference first bite?
6. For which n does a linear scan beat an `unordered_map` lookup, and what makes the scan competitive?
7. Explain why `i != j` is a wrong loop condition in array reversal but `i < j` is right.
8. In a merge of two sorted arrays, why does `<=` preserve stability while `<` destroys it?
9. Why does `dedupSorted` return a length rather than shrinking the array, and what is the caller expected to do with it?
10. Explain how move-zeros, remove-duplicates and partition are the same algorithm. What varies?
11. Give the exact failure mode of `++freq[x]` when x can be -1 , and two ways to prevent it.
12. Binary search is $O(\log n)$ and linear scan is $O(n)$, yet a scan of 64 sorted ints is usually faster. Explain in terms of cache lines.

B. Tracing and analysis

1. Trace `insertAt` inserting 7 at pos 0 into `[1,2,3,4]` (cap 6). List the array after each iteration and give the number of element moves.
2. Binary search `[2,5,8,12,16,23,38,56,72,91]` for 12 and for 13. Give the `(lo,hi,mid)` triple at each step and the comparison count for each.
3. For $n = 10^6$, give the number of comparisons for linear search (hit average, miss) and for binary search (worst case). Give the ratio.
4. Merge `[1,3,5,7]` and `[2,3,6]`. Show the output after each comparison and count the comparisons.
5. `minMax` on 1001 elements: give the comparison count for the naive and paired versions, and the percentage saved.
6. Run `dedupSorted` on `[1,1,1,2,3,3]`. Give `w` after each read step, the returned length, and the full contents of the array including the tail.
7. Run `moveZeros` on `[0,0,1]`: give the array and `w` after each iteration, and the total number of swaps.

C. Code tasks

1. **Implement the five primitives** against a fixed `int a[100]` with an explicit `len`. Then write the test matrix from Card 1 ($n = 0, 1, 2$, insert at 0, insert at len, delete last) and make all cases pass.

2. **Break it: shift the wrong way.** Implement insertion with a *forward* shift. Insert 99 at index 1 of `[10,20,30,40]` and record the output exactly. Explain the smear, then fix it and diff the two outputs.
3. **Break it: binary search on unsorted data.** Run your correct binary search on `[5,2,9,1,7]` searching for 9 and for 2. Record what each returns. Then add `assert(std::is_sorted(a, a+n))` at the top and re-run — record the assertion message. The lesson is that the ungarded version returned a *plausible* index.
4. **Break it: frequency array underflow.** Build `int freq[256]` as a local, place a canary variable `int canary = 12345;` immediately before it, and count an array containing `-1` and `-2`. Print the canary. Then rebuild under `-fsanitize=address` and record the exact error. Fix with an offset and show the canary intact.
5. **Crossover measurement.** Benchmark linear search versus `unordered_map::find` on $n = 4, 8, 16, 32, 64, 256$ randomly chosen keys, 10^7 lookups each. Report the crossover n where the map wins.
6. **memmove versus a splice.** Time 10^5 insertions at index 0 into a `vector<int>` of size 1000, then into a `std::list<int>` of size 1000. Report both. Repeat at size 100,000 and report where the list finally wins.
7. **Stability check.** Merge two sorted arrays of `{key, tag}` pairs that tie on `key`, once with `<=` and once with `<`. Print the tag order for both and identify which is stable.

ANSWER KEY — Topics 38-51

A. Conceptual

1. **Contiguity must hold after every operation.** Reading does not disturb it ($O(1)$); inserting in the middle does, and restoring it costs one move per following element.
2. Because a forward shift overwrites `a[i+1]` before it has been copied. `memmove` handles overlapping ranges by choosing direction; `memcpy` does not and is UB on overlap.
3. Legal when the collection is a **set, not a sequence** — order carries no meaning. It costs you the order: the element that was last is now at `pos`.
4. Precondition: **non-decreasing order**. Violated, it does not error — it follows the halving rule against data that does not satisfy it and returns **-1 or a wrong-but-valid-looking index**.
5. `lo+hi` overflows `int` once both exceed 2^{30} ; the sum goes negative and `mid` becomes negative. First bites at $n \approx 2^{31}/2 = 1.07 \times 10^9$ elements (or smaller if `lo`, `hi` are byte offsets).
6. Roughly **$n \leq 8-30$** . The scan is branch-predictable, fully prefetched, and touches 1-2 cache lines; the map pays a hash, a modulo, a dependent load into a scattered bucket, and a key comparison.
7. Because the two indices **cross without meeting** for even n : $i = j$ never becomes true, so the loop runs past the ends. `i < j` terminates for both parities.
8. `<=` takes from the **left (earlier) array on a tie**, preserving original relative order. `<` takes from the right on a tie, reversing equal elements.
9. Because an array has no "shrink" — removing elements would mean moving them and the algorithm is defined to be $O(1)$ space. The caller **erases the tail** (the erase-remove idiom).
10. All three are **stable partition with a predicate**: keep-if. Only the predicate varies — "differs from the last kept", "is non-zero", "is less than the pivot".
11. It writes to `freq[-1]` — **4 bytes before the array**, into an adjacent local or a saved register. Prevent with an offset index (`freq[x + OFFSET]`) or an explicit range guard.
12. 64 ints = **4 cache lines** read at full bandwidth with a predictable stride. Binary search does ~ 6 *dependent* loads to scattered addresses, each potentially a miss — 6 misses cost more than 4 streamed lines.

B. Tracing and analysis

1. `[1,2,3,4,4,_]` $\rightarrow$ `[1,2,3,3,4,_]` $\rightarrow$ `[1,2,2,3,4,_]` $\rightarrow$ `[1,1,2,3,4,_]` $\rightarrow$ write: `[7,1,2,3,4,_]`. Moves = **4** (worst case, `pos 0`).
2. Key 12: (0,10,5) $23 > 12 \rightarrow hi=5$; (0,5,2) $8 < 12 \rightarrow lo=3$; (3,5,4) $a[4]=16 > 12 \rightarrow hi=4$; (3,4,3) $a[3]=12$ **found, 4 comparisons**. Key 13: (0,10,5), (0,5,2), (3,5,4), (3,4,3) $12 < 13 \rightarrow lo=4 = hi \rightarrow -1$, **4 comparisons**.
3. Linear: **500,000** average on a hit, **10^6** on a miss. Binary: **20**. Ratio on a miss: **$50,000\times$** in comparisons (about 300-1000 $\times$ in wall clock, because the scan streams).
4. Output evolves 1, 2, 3, 3, 5, then drains 7: **[1,2,3,3,5,6,7]**. Comparisons: **6**.
5. Naive **2000**; paired: $3 \cdot 1001/2 - 2 \approx$ **1499**. Saved **25%**.
6. `w` after each `r`: `r=1` $\rightarrow$ 1, `r=2` $\rightarrow$ 1, `r=3` $\rightarrow$ 2, `r=4` $\rightarrow$ 3, `r=5` $\rightarrow$ 3. Returns **3**; array is **[1,2,3,2,3,3]** — the tail past index 2 is stale, not cleared.
7. `r=0` zero (`w=0`), `r=1` zero (`w=0`), `r=2` swap `a[0],a[2]` $\rightarrow$ **[1,0,0]**, `w=1`. Swaps: **1**.

C. What to verify

1. All six cases pass, and inserting at `len` performs **zero** moves while inserting at 0 performs **len**.
2. Forward shift prints **[10,20,20,20]** (the smear); correct version prints **[10,99,20,30]**. Every element after the insertion point equals the one that was at `pos`.
3. On `[5,2,9,1,7]` searching 9, expect **-1** (`mid=2` holds 9 in this case, so you may get 2 — run both keys). Searching 2 returns **-1** although 2 is present at index 1. The assertion prints `Assertion 'std::is_sorted(a, a+n)' failed.` and aborts with signal 6. **The point: no wrong answer was reported without the assert.**
4. The canary prints a value other than 12345 (commonly 12344 or 12343, depending on layout and how many negatives you counted) — a decrement of an unrelated variable. ASan: `ERROR: AddressSanitizer: stack-buffer-underflow ... WRITE of size 4 with 4 bytes before.` After the offset fix the canary is **12345**.
5. Crossover typically **$n \approx 16-32$** . Below it the scan wins by 2-4×; above $n = 256$ the map wins by an order of magnitude.
6. At size 1000 the vector usually wins or ties (memmove of 4 KB ≈ 200 ns beats allocating a node ≈ 50 ns plus a guaranteed cache miss per traversal step). At size 100,000 the list wins on the insert loop alone — but loses any workload that also *reads* by index.
7. With `<=` the tags from the left array appear first among ties; with `<` they appear second. Only the first is stable.

HANDNOTE SUMMARY CARD — Basic Operations

BASIC ARRAY OPERATIONS (Topics 38-51)

THE RULE: anything that MOVES elements is $O(n)$. Everything below follows.

access $a[i]$	$O(1)$	1 instruction
update $a[i]=v$	$O(1)$	1 store, invalidates nothing
append (capacity)	$O(1)$	nothing moves
insert at pos	$O(n-pos)$	shift BACKWARDS from len down to pos+1
delete at pos	$O(n-pos)$	shift FORWARDS from pos to len-2
delete unordered	$O(1)$	$a[pos] = a[--len]$ (order is sacrificed)
linear search	$O(n)$	$n/2$ hit, n miss, ~ 10 GB/s bandwidth-bound
binary search	$O(\log n)$	20 steps @ 10^6 , 30 @ 10^9 ; needs SORTED
reverse	$O(n)$	$n/2$ swaps, $i < j$
merge sorted	$O(n+m)$	two pointers, $\leq$ for stability
is sorted	$O(n)$	early exit at first inversion
min+max	$O(n)$	$2n-2$ naive, $3n/2-2$ paired
frequency (K vals)	$O(n+K)$	direct index; hash is 5-10x slower per element
dedup sorted	$O(n)$	read/write pointers, returns new length
move zeros	$O(n)$	same pattern, predicate = "non-zero"

TEMPLATES WORTH MEMORISING

```
insert: for(i=len;i>pos;--i) a[i]=a[i-1]; a[pos]=v; ++len;
delete: for(i=pos;i<len-1;++i) a[i]=a[i+1]; --len;
bsearch: lo=0, hi=n; while(lo<hi){m=lo+(hi-lo)/2; a[m]<k?lo=m+1:hi=m;}
revers: for(i=0,j=n-1;i<j;++i,--j) swap(a[i],a[j]);
merge : while(i<n&& j<m) out.push_back(a[i]<=b[j]?a[i++]:b[j++]);
compct: w=0; for(r=0;r<n;++r) if(keep(a[r])) a[w++]=a[r]; return w;
```

ONE PATTERN, FOUR NAMES (read/write pointer = stable partition)

```
dedup sorted  keep if a[r] != a[w-1]
move zeros    keep if a[r] != 0
remove value  keep if a[r] != target    -> std::remove
partition     keep if a[r] < pivot      -> Topic 58
```

TRAPS

```
forward shift on insert -> smear: [10,20,20,20]
(lo+hi)/2               -> overflow above  $\sim 1.07e9$ ; JDK bug for 9 years
bsearch on unsorted     -> plausible WRONG index, never an error
i != j in reverse       -> infinite loop on even n
< instead of <= in merge -> silently unstable
++freq[-1]              -> stack-buffer-underflow into a neighbouring local
erase() inside a loop   -> skips an element; use it = v.erase(it)
compaction does NOT erase -> caller must erase the tail (erase-remove idiom)
```

BIG-0 vs WALL CLOCK

```
memmove 1000 ints  $\sim 200$  ns ->  $O(n)$  shift beats an  $O(1)$  list splice to  $n \sim 1000$ 
scan 64 sorted ints (4 lines) beats binary search (6 dependent misses)
```


SECTION 9 — ROTATION & REARRANGEMENT

52. Array Rotation

Rotation moves every element k positions cyclically: nothing is added or removed, only re-indexed. Formally, left rotation by k maps $b[i] = a[(i + k) \bmod n]$; right rotation maps $b[(i + k) \bmod n] = a[i]$. Every rotation algorithm in this section computes that same mapping — they differ only in space and in how cache-friendly the access pattern is.

Method	Time	Space	Notes
$k \times$ single rotation	$O(n \cdot k)$	$O(1)$	Only acceptable for $k = 1$
Temp buffer	$O(n)$	$O(k)$	Simplest correct version
Juggling (GCD cycles)	$O(n)$	$O(1)$	$\gcd(n, k)$ cycles; poor locality
Three reversals	$O(n)$	$O(1)$	The answer to give (Topic 57)
<code>std::rotate</code>	$O(n)$	$O(1)$	Use in production

rotation – best $O(n)$ time, $O(1)$ space · $3n/2$ element moves via reversal · `std::rotate` returns the new middle

53. Left Rotation

```
// Left-rotate by 1: [1,2,3,4,5] -> [2,3,4,5,1]
void rotateLeft1(int* a, int n) {
    if (n <= 1) return;
    int first = a[0];
    for (int i = 0; i < n - 1; ++i) a[i] = a[i + 1];    // n-1 moves
    a[n - 1] = first;
}
```

Repeating this k times is $O(n \cdot k)$ — for $n = 10^5$ and $k = 10^5$ that is 10^{10} operations, roughly ten seconds. This is the naive solution interviewers expect you to reject out loud before writing anything.

left rotate by 1 – Time $O(n)$, $n-1$ moves + 1 save · Space $O(1)$ · by k : $O(n \cdot k)$, unacceptable above $k \approx 3$

54. Right Rotation

```
// Right-rotate by 1: [1,2,3,4,5] -> [5,1,2,3,4]
void rotateRight1(int* a, int n) {
    if (n <= 1) return;
    int last = a[n - 1];
    for (int i = n - 1; i > 0; --i) a[i] = a[i - 1];    // BACKWARDS again
    a[0] = last;
}
```

THE KEY INSIGHT

Right rotation by k is left rotation by $n - k$. Implement one direction properly and express the other in terms of it; two implementations mean two places for an off-by-one to hide.

right rotate – Time $O(n)$ · Space $O(1)$ · $\text{rotateRight}(k) \equiv \text{rotateLeft}(n - k \bmod n)$

55. Rotation by K

```
// Extra-buffer version: clear,  $O(n)$  time,  $O(k)$  space.
void rotateLeftK(std::vector<int>& a, int k) {
    int n = a.size();
    if (n == 0) return;
    k %= n;                                // MANDATORY (Topic 56)
    if (k < 0) k += n;                     // handle negative k
    std::vector<int> tmp(a.begin(), a.begin() + k);
    std::copy(a.begin() + k, a.end(), a.begin()); // shift left
    std::copy(tmp.begin(), tmp.end(), a.end() - k); // append saved
}
```

```
a = [1,2,3,4,5,6,7]  k = 3
tmp  = [1,2,3]
shift = [4,5,6,7,_,_,_]
append= [4,5,6,7,1,2,3]      one pass, k ints of scratch space
```

rotate by k – Time $O(n)$, $2n$ copies · Space $O(k)$ · degenerates to $O(n)$ space when $k \approx n$

56. Rotation Using Mod

Rotating an array of length 7 by 7 is a no-op; by 10 it is the same as by 3. Without `k %= n`, a rotation by $k \geq n$ either does redundant work or reads out of bounds. With it, every rotation is $O(n)$ regardless of how large k is.

```
k %= n;                // 10 % 7 == 3
if (k < 0) k += n;      // C++ % keeps the sign of the dividend: -3 % 7 == -3
// so the mapping b[i] = a[(i + k) % n] is now always in range
```

THE TRAP

`%` in C++ is remainder, not mathematical modulo: `-3 % 7` is `-3`, not 4. Indexing with it reads before the array. Symptom: a rotation that works for positive k and segfaults or returns garbage for negative k . The two-line fix above is the whole solution, and the same fix applies to circular buffers (Topic 150) and hash bucket indices.

modulo – 1 division (~20–40 cycles) · turns unbounded k into $k \in [0, n)$ · negative k needs the `+n` correction

57. Rotation Using Reverse

The answer to give in an interview: three reversals, $O(n)$ time, $O(1)$ space, no gcd, no scratch buffer, and each reversal is a sequential two-pointer pass that the prefetcher handles well.

```
void rotateLeftK(int* a, int n, int k) {
    if (n <= 1) return;
    k %= n; if (k < 0) k += n;
    std::reverse(a, a + k);          // reverse the first k
    std::reverse(a + k, a + n);      // reverse the rest
    std::reverse(a, a + n);          // reverse the whole thing
}
```

```
a = [1,2,3,4,5,6,7]  k = 3  (left rotation)
reverse [0,3)  -> [3,2,1, 4,5,6,7]
reverse [3,7)  -> [3,2,1, 7,6,5,4]
reverse [0,7)  -> [4,5,6,7, 1,2,3]    total swaps = 3n/2 = 10
```

THE KEY INSIGHT

Reversal is an involution that turns "move a block from the front to the back" into three local passes. It is the same trick as in-place matrix rotation (Topic 104 — transpose then reverse rows) and next permutation (Topic 73 — reverse the suffix). One primitive, three famous problems.

reversal rotation — Time $O(n)$, $\sim 3n/2$ swaps · Space $O(1)$ · 3 sequential passes, fully prefetched

58. Partitioning Arrays

Partitioning rearranges elements so that everything satisfying a predicate comes first. It is the engine of quicksort (Topic 79), quickselect, and every "move all X to one side" problem.

```
// Lomuto: simple, one pass, ~n swaps. Returns the pivot's final index.
int partition(int* a, int lo, int hi) {          // pivot = a[hi]
    int pivot = a[hi], i = lo - 1;
    for (int j = lo; j < hi; ++j)
        if (a[j] < pivot) std::swap(a[++i], a[j]);
    std::swap(a[i + 1], a[hi]);
    return i + 1;
}
// std::partition (unstable,  $O(1)$  space) / std::stable_partition ( $O(n)$  space)
```

```
a = [7,2,9,1,5]  pivot = 5 (last)
j=0 7<5? no           [7,2,9,1|5]  i=-1
j=1 2<5? yes swap a[0],a[1]  [2,7,9,1|5]  i=0
j=2 9<5? no           [2,7,9,1|5]  i=0
j=3 1<5? yes swap a[1],a[3]  [2,1,9,7|5]  i=1
place pivot: swap a[2],a[4]  [2,1,5,7,9]  return 2
```

THE TRAP

Lomuto partition on an array where *all elements are equal* makes every comparison false and returns `hi`, splitting n into $n-1$ and 0 — which turns quicksort into $O(n^2)$ and overflows the stack at $n \approx 10^5$. Hoare partition or three-way partitioning (Topic 59) fixes it. Symptom: a sort that is fast on random data and stack-overflows on real data with many duplicates.

partition — Time $O(n)$, $\leq n$ swaps · Space $O(1)$ unstable / $O(n)$ stable · degenerate on all-equal input

59. Dutch National Flag

Three-way partitioning in one pass: sort an array of 0s, 1s and 2s without counting, using three pointers. Dijkstra's problem, and the fix for quicksort's duplicate-key blow-up.

```
void dutchFlag(int* a, int n) {
    int low = 0, mid = 0, high = n - 1;
    while (mid <= high) {
        if (a[mid] == 0) std::swap(a[low++], a[mid++]);
        else if (a[mid] == 1) ++mid;
        else std::swap(a[mid], a[high--]); // do NOT ++mid
    }
}
```

```
invariant:  [0 0 0 | 1 1 1 | ? ? ? | 2 2 2]
              ^low   ^mid   ^high
a[0..low)   all 0      a[low..mid) all 1
a[mid..high] unknown  a[high..n)   all 2
[2,0,2,1,1,0] -> swap mid,high -> [0,0,2,1,1,2] -> ... -> [0,0,1,1,2,2]
```

THE TRAP

Incrementing **mid** after swapping with **high** is the canonical bug: the value you just pulled from the back has not been examined yet. Symptom: a stray 2 stranded in the middle, on roughly one input in three — it passes casual testing.

DNF — Time $O(n)$, one pass, $\leq n$ swaps · Space $O(1)$ · alternative: 2 passes counting, also $O(n)$ but not in-place-stable

SECTION 10 — ARRAY PROBLEM-SOLVING PATTERNS

60. Prefix Sum

Precompute cumulative sums once in $O(n)$, then answer any range-sum query in $O(1)$. The trade is n extra integers for unlimited free queries — the right trade whenever queries outnumber updates.

```
// p[i] = a[0] + ... + a[i-1], so p[0] = 0 and sum(l, r) = p[r+1] - p[l].
std::vector<long long> prefix(const std::vector<int>& a) {
    std::vector<long long> p(a.size() + 1, 0);
    for (std::size_t i = 0; i < a.size(); ++i) p[i + 1] = p[i] + a[i];
    return p;
}
long long rangeSum(const std::vector<long long>& p, int l, int r) {
    return p[r + 1] - p[l];           // inclusive [l, r]
}
```

```
a =      [ 3,  1,  4,  1,  5]
p = [0,   3,  4,  8,  9, 14]    p has n+1 entries; p[0] = 0 is the trick
sum(1,3) = p[4] - p[1] = 9 - 3 = 6      = 1 + 4 + 1    one subtraction
```

THE KEY INSIGHT

The $n+1$ sized array with $p[0] = 0$ eliminates the $l == 0$ special case entirely. Every off-by-one bug in prefix sums comes from using an n -sized array and branching. Never branch; pad.

THE TRAP

Prefix sums overflow silently. 10^5 elements of up to 10^9 sum to 10^{14} — well past `int`'s 2.1×10^9 . Symptom: correct answers on small tests, negative answers on large ones. Use `long long` for the prefix array even when the input is `int`.

prefix sum — build $O(n)$ time / $O(n)$ space · query $O(1)$ · point update invalidates the suffix: $O(n)$ to rebuild

61. Suffix Sum

The mirror image: $s[i]$ is the sum from i to the end. Used whenever a problem needs "everything to my right" — equilibrium index, product-except-self, maximum-of-suffix comparisons, and stock-profit variants.

```
std::vector<long long> suffix(const std::vector<int>& a) {
    int n = a.size();
    std::vector<long long> s(n + 1, 0);
    for (int i = n - 1; i >= 0; --i) s[i] = s[i + 1] + a[i];
    return s;
}
// Equilibrium index: the i where prefix[i] == suffix[i+1].
```

THE KEY INSIGHT

Prefix and suffix arrays together answer "left of i" and "right of i" in $O(1)$, which converts a large family of $O(n^2)$ two-loop problems into two linear passes. Product-except-self is the canonical example: left products, right products, multiply — no division, handles zeros correctly.

suffix sum — build $O(n)$ / $O(n)$ · query $O(1)$ · with prefix: any split-point problem becomes 2 passes

62. Difference Array

The dual of prefix sums: prefix sums make *queries* $O(1)$ at the cost of $O(n)$ updates; difference arrays make *range updates* $O(1)$ at the cost of an $O(n)$ finalisation. Use when all updates arrive before all reads.

```
// Add v to every element in [l, r], in  $O(1)$  per update.
void rangeAdd(std::vector<long long>& d, int l, int r, long long v) {
    d[l] += v;
    d[r + 1] -= v; // d must have n+1 entries
}
// One final pass turns the difference array back into the real array:
void finalise(std::vector<long long>& d) {
    for (std::size_t i = 1; i < d.size(); ++i) d[i] += d[i - 1];
}
```

```
n=5, add 10 to [1,3]      d = [0, 10, 0, 0, -10, 0]
prefix-sum d ->          [0, 10,10,10,  0, 0]
                        ^^^^^^^^^^ exactly [1,3]
q updates + 1 pass:  $O(q + n)$  instead of  $O(q \cdot n)$ 
```

THE KEY INSIGHT

A difference array is "prefix sum run backwards". The pair is a general technique: whichever of update/query is rarer should be the $O(n)$ one. When both are frequent and interleaved, you need a Fenwick or segment tree — the point where plain arrays stop being enough.

difference array — update $O(1)$ · finalise $O(n)$ · q updates: $O(q+n)$ vs $O(q \cdot n)$ naive · needs $n+1$ slots

63. Kadane's Algorithm

Maximum sum of a contiguous subarray, in one pass and $O(1)$ space. The insight is local: at each element, either extend the previous best subarray or start fresh here.

```
long long kadane(const std::vector<int>& a) {
    long long best = a[0], cur = a[0];
    for (std::size_t i = 1; i < a.size(); ++i) {
        cur = std::max<long long>(a[i], cur + a[i]); // extend or restart
        best = std::max(best, cur);
    }
    return best; // handles all-negative correctly
}
```

```

a = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
cur = -2 1 -2 4 3 5 6 1 5
best= -2 1 1 4 4 5 6 6 6      answer 6 = subarray [4,-1,2,1]

```

THE TRAP

Initialising `best = 0` returns 0 for an all-negative array instead of the largest (least negative) element. This is the most common Kadane bug and the test case interviewers use: `[-3,-1,-2]` must return `-1`, not 0. Initialise both variables to `a[0]` and start the loop at index 1.

Kadane – Time $O(n)$, one pass · Space $O(1)$ · handles negatives iff initialised from `a[0]`, not 0

64. Sliding Window

A window over a contiguous range, moved one step at a time, with an incrementally maintained aggregate. Converts the $O(n \cdot k)$ "recompute each window" loop into $O(n)$ total.

```

// FIXED window of size k: max sum.
long long maxSumK(const std::vector<int>& a, int k) {
    long long sum = 0;
    for (int i = 0; i < k; ++i) sum += a[i];
    long long best = sum;
    for (std::size_t i = k; i < a.size(); ++i) {
        sum += a[i] - a[i - k];          // add entering, remove leaving:  $O(1)$ 
        best = std::max(best, sum);
    }
    return best;
}

// VARIABLE window: longest subarray with sum <= S (non-negative values).
int longestAtMost(const std::vector<int>& a, long long S) {
    long long sum = 0; int lo = 0, best = 0;
    for (int hi = 0; hi < (int)a.size(); ++hi) {
        sum += a[hi];
        while (sum > S) sum -= a[lo++];    // shrink from the left
        best = std::max(best, hi - lo + 1);
    }
    return best;
}

```

```

fixed k=3:  [1, 2, 3, 4, 5]
            [1 2 3]      sum 6
            [2 3 4]      sum 6 + 4 - 1 = 9      one add, one subtract per step
            [3 4 5]      sum 9 + 5 - 2 = 12      total work  $O(n)$ , not  $O(n \cdot k)$ 

```

THE TRAP

The variable-size window requires that shrinking always *helps* — true for non-negative values, false the moment negatives appear, because removing a negative *increases* the sum. With negatives you need prefix sums plus a hash map or a monotonic deque instead. Symptom: correct on every test with positive numbers, wrong on one with a `-5` in it.

sliding window – Time $O(n)$, each element enters and leaves once · Space $O(1)$ fixed, $O(k)$ with a deque

65. Two Pointers

Two indices moving under a rule that guarantees each only advances, giving $O(n)$ total. Three shapes cover nearly every use: opposite ends converging, same-side fast/slow (Topic 66), and read/write compaction (Topic 50).

```
// Opposite ends: pair summing to target in a SORTED array.  $O(n)$ ,  $O(1)$ .
std::pair<int,int> twoSum(const std::vector<int>& a, int target) {
    int i = 0, j = (int)a.size() - 1;
    while (i < j) {
        long long s = (long long)a[i] + a[j];
        if (s == target) return {i, j};
        if (s < target) ++i;          // need bigger: only the left can grow
        else --j;                     // need smaller: only the right can shrink
    }
    return {-1, -1};
}
```

```
a = [1,3,4,6,8,11] target = 10
i=0 j=5 1+11=12 > 10 -> --j
i=0 j=4 1+ 8= 9 < 10 -> ++i
i=1 j=4 3+ 8=11 > 10 -> --j
i=1 j=3 3+ 6= 9 < 10 -> ++i
i=2 j=3 4+ 6=10 -> FOUND total steps <= n
```

THE KEY INSIGHT

The correctness argument is always the same: when you discard a candidate, prove it cannot be part of *any* solution. Here, if $a[i]+a[j]$ is too big, then $a[j]$ paired with anything $\geq a[i]$ is also too big, so $a[j]$ is dead and j may move. If you cannot make that argument, two pointers is the wrong tool.

two pointers – Time $O(n)$ after sorting ($O(n \log n)$ if unsorted) · Space $O(1)$ · beats the $O(n^2)$ double loop

66. Fast and Slow Pointers

One pointer advances by 1, the other by 2. On arrays the pattern finds the midpoint in a single pass; on index-as-pointer arrays it detects cycles (Topic 69) in $O(1)$ space.

```
// Midpoint in one pass (useful when n is not known in advance).
int midpoint(const std::vector<int>& a) {
    std::size_t slow = 0, fast = 0;
    while (fast + 1 < a.size()) { ++slow; fast += 2; }
    return (int)slow;          // n=5 -> 2 ; n=6 -> 3
}
```

```
n = 6: index 0 1 2 3 4 5
slow      0 1 2 3      slow ends at floor(n/2) = 3
fast      0 2 4 (stop) fast covers 2x the ground
"fast reaches the end when slow reaches the middle" – one pass,  $O(1)$  space
```

THE TRAP

`fast + 1 < n` versus `fast < n` selects the lower or upper midpoint on even `n`, and `fast += 2` without the bound check reads past the end when `n` is even. Decide which midpoint your problem needs and write the test for `n = 0, 1, 2` before anything else.

fast/slow – Time $O(n)$ · Space $O(1)$ · finds midpoint, cycle entry, and `k`-from-end in one pass

67. Majority Element

Boyer–Moore voting finds an element occurring more than $n/2$ times in one pass and $O(1)$ space, without sorting or hashing.

```
int majority(const std::vector<int>& a) {
    int cand = a[0], count = 0;
    for (int x : a) {
        if (count == 0) cand = x;
        count += (x == cand) ? 1 : -1;
    }
    int occurrences = std::count(a.begin(), a.end(), cand); // pass 2: VERIFY
    return occurrences > (int)a.size() / 2 ? cand : -1;
}
```

```
a = [2,2,1,1,1,2,2]
x=2 c=0 -> cand=2 c=1    x=2 c=2    x=1 c=1    x=1 c=0
x=1 c=0 -> cand=1 c=1    x=2 c=0    x=2 c=0 -> cand=2 c=1 -> candidate 2
verify: count(2) = 4 > 7/2 = 3 -> answer 2
intuition: each non-majority element cancels one majority element; a strict
majority has more than half the votes, so at least one survives.
```

THE TRAP

Skipping the verification pass returns a confident wrong answer whenever no majority exists: `[1,2,3]` yields candidate 3. The algorithm is only valid *given* that a majority exists; the second pass is what makes it correct in general.

Boyer–Moore – Time $O(n)$, 2 passes · Space $O(1)$ · vs hash map $O(n)$ space · $n/3$ variant keeps 2 candidates

68. Missing Number

Given `n` distinct values from `0..n` with one missing, find it. Two $O(1)$ -space methods; one of them overflows and one does not.

```
// Sum method: clear, but n*(n+1)/2 overflows int at n ≈ 65,536.
int missingSum(const std::vector<int>& a) {
    long long n = a.size();
    long long expected = n * (n + 1) / 2;
    long long actual = std::accumulate(a.begin(), a.end(), 0LL);
    return (int)(expected - actual);
}

// XOR method: cannot overflow, same O(n)/O(1).
int missingXor(const std::vector<int>& a) {
    int x = (int)a.size();
    for (std::size_t i = 0; i < a.size(); ++i) x ^= (int)i ^ a[i];
    return x;
    // every present value cancels itself
}
```

THE KEY INSIGHT

XOR is its own inverse ($x \oplus x == 0$) and is associative, so pairing each index with each value cancels everything except the unpaired one. The same identity solves "single number among pairs" and, with bit-position counting, "single number among triples".

missing number – Time $O(n)$ · Space $O(1)$ · sum: overflows past $n \approx 65k$ in int; XOR: never overflows

69. Duplicate Number

```
// Values in [1, n] with exactly one duplicate: Floyd cycle detection.
// Treat a[i] as "next index". The duplicate is the cycle entry. O(1) space,
// and it does not modify the array.
int findDuplicate(const std::vector<int>& a) {
    int slow = a[0], fast = a[a[0]];
    while (slow != fast) { slow = a[slow]; fast = a[a[fast]]; } // meet
    slow = 0;
    while (slow != fast) { slow = a[slow]; fast = a[fast]; } // entry
    return slow;
}
```

a = [1,3,4,2,2] index: 0 1 2 3 4
 0 -> 1 -> 3 -> 2 -> 4 -> 2 -> 4 -> ... cycle entered at value 2
 two pointers meet inside the cycle; restarting one at 0 and stepping both
 at speed 1 makes them meet exactly at the entry – that entry IS the duplicate

Method	Time	Space	Modifies input
Hash set	$O(n)$	$O(n)$	No
Sort then scan	$O(n \log n)$	$O(1)$	Yes
Negative marking	$O(n)$	$O(1)$	Yes
Floyd cycle	$O(n)$	$O(1)$	No

duplicate – Floyd: Time $O(n)$, 2 phases · Space $O(1)$ · requires values in $[1, n]$ and read-only access

70. Subarray Concepts

A **subarray** is a contiguous slice: `a[l..r]`. There are $n(n+1)/2$ of them — about 5×10^9 for $n = 10^5$, which is why "check every subarray" is never the intended solution above $n \approx 3000$.

```
a = [1,2,3]      subarrays (6 = 3*4/2):
  [1] [2] [3] [1,2] [2,3] [1,2,3]
NOT subarrays: [1,3] (not contiguous — that is a subsequence, Topic 71)
```

The standard escapes from $O(n^2)$ are: prefix sums (Topic 60) when the property is additive, sliding window (Topic 64) when it is monotone, and Kadane (Topic 63) when it is a running optimum.

subarrays — count $n(n+1)/2$ · enumerate all: $O(n^2)$ pairs, $O(n^3)$ if you also sum · $O(n)$ with prefix/window

71. Subsequence Concepts

A **subsequence** keeps order but not adjacency, so there are 2^n of them — 10^{30} for $n = 100$. Any subsequence problem you can solve is solved by dynamic programming or greedy, never by enumeration.

Concept	Contiguous?	Count for n	Typical technique
Subarray	Yes	$n(n+1)/2$	Window, prefix, Kadane
Subsequence	No	2^n	DP (LIS, LCS), greedy
Subset	No, order-free	2^n	Bitmask, DP
Permutation	Rearrangement	$n!$	Backtracking (Topic 72)

THE TRAP

Misreading "subarray" as "subsequence" changes the problem class entirely — maximum subarray sum is $O(n)$ Kadane; maximum increasing *subsequence* is $O(n \log n)$ DP. Interviewers use the words precisely. Restate the definition out loud before you start.

counts — subarray $n(n+1)/2$ · subsequence 2^n · permutation $n!$ · $n=20$: 210 vs 1.05M vs 2.4×10^{18}

72. Permutations

```
// Backtracking: swap each candidate into position, recurse, swap back.
void permute(std::vector<int>& a, int k,
             std::vector<std::vector<int>>& out) {
    if (k == (int)a.size()) { out.push_back(a); return; }
    for (int i = k; i < (int)a.size(); ++i) {
        std::swap(a[k], a[i]);
        permute(a, k + 1, out);
        std::swap(a[k], a[i]);           // UNDO — this is the backtrack
    }
}
```

$n!$ growth: $5! = 120$ $8! = 40,320$ $10! = 3.6\text{M}$ $12! = 479\text{M}$ $20! = 2.4\text{e}18$
practical ceiling for full enumeration: $n = 10$ or 11 . Beyond that the answer is never "generate them all".

THE TRAP

Forgetting the undo swap corrupts the array for every sibling branch: you get the right *number* of permutations with wrong contents, and the bug looks like a duplicate-output problem. Also note this method emits duplicates when the input has repeated values — sort and skip equal candidates to fix.

permutations – Time $O(n \cdot n!)$ to emit all · Space $O(n)$ recursion · ceiling $n \approx 11$ in practice

73. Next Permutation

Produce the next lexicographically larger arrangement in $O(n)$, in place — the algorithm behind `std::next_permutation`. Four steps, each of which is a primitive from earlier topics.

```
bool nextPermutation(std::vector<int>& a) {
    int n = a.size(), i = n - 2;
    while (i >= 0 && a[i] >= a[i + 1]) --i;      // 1. rightmost ascent
    if (i < 0) { std::reverse(a.begin(), a.end()); return false; } // last
    int j = n - 1;
    while (a[j] <= a[i]) --j;                    // 2. rightmost value > a[i]
    std::swap(a[i], a[j]);                       // 3. swap
    std::reverse(a.begin() + i + 1, a.end());    // 4. sort the suffix
    return true;
}
```

```
a = [1, 3, 5, 4, 2]
1. scan from right for a[i] < a[i+1]: i = 1 (value 3)
2. rightmost value > 3 from the right: j = 3 (value 4)
3. swap:                                [1, 4, 5, 3, 2]
4. reverse suffix after i:               [1, 4, 2, 3, 5] <- the answer
```

THE KEY INSIGHT

The suffix after the ascent is always non-increasing, so "sort it ascending" is just *reverse it* — $O(n)$ instead of $O(n \log n)$. This is Topic 44 doing the work again, exactly as it did in Topic 57 and will in Topic 104.

next permutation – Time $O(n)$ worst, $O(1)$ amortised over a full cycle · Space $O(1)$ · returns false and wraps at the last

PRACTICE SET — Topics 52-73

A. Conceptual

- Express right rotation by k in terms of left rotation, and say why implementing only one direction reduces bugs.
- Why is `k %= n` mandatory before any rotation, and what additional line is needed when k may be negative?
- Explain why three reversals rotate an array, in terms of what each reversal achieves.
- State the loop invariant of Dutch national flag, and explain why `mid` must not advance after a swap with `high`.
- Why does Lomuto partition degrade to $O(n^2)$ on all-equal input, and which two alternatives fix it?
- Prefix sums make queries $O(1)$ and updates $O(n)$; difference arrays invert that. State the rule for choosing between them, and what to use when both are frequent.
- Why must a prefix-sum array be `long long` even when the input is `int`? Give the threshold with numbers.
- Why does Kadane initialised to 0 fail on all-negative input, and what is the correct initialisation?
- The variable-size sliding window requires a specific property of the data. Name it, and give a concrete input that breaks the technique.
- State the general correctness argument for opposite-end two pointers. What must be provable before you may move a pointer?
- Why does Boyer-Moore need a second pass, and what does it return without one?
- Give the counts of subarrays, subsequences and permutations for $n = 20$, and say which of the three can be enumerated.
- In next-permutation, why is reversing the suffix equivalent to sorting it ascending?

B. Tracing and analysis

- Left-rotate `[1,2,3,4,5,6,7]` by $k = 10$ using three reversals. Show the array after each reversal, and give the effective k .
- Run Dutch national flag on `[2,0,2,1,1,0]`. Give `(low, mid, high)` and the array after every iteration, plus the total swap count.
- Build the prefix array for `[4,-2,7,1,-5]` and answer `sum(1,3)` and `sum(0,4)` using only subtraction.
- Apply range updates $+5$ on $[0,2]$ and -3 on $[1,4]$ to a difference array of length 6 ($n = 5$). Show the difference array, then the finalised array.
- Trace Kadane on `[-2,-3,4,-1,-2,1,5,-3]`: give `cur` and `best` at every index and identify the winning subarray.
- Fixed sliding window of size 3 on `[2,1,5,1,3,2]`: give every window sum, the number of additions performed, and compare with the naive recomputation count.
- Run Boyer-Moore on `[1,2,3,2,2]` and on `[1,2,3]`. Give the candidate and the final answer for both.
- Apply `nextPermutation` to `[1,2,3]` five times in succession. List all five results and state what the sixth call returns.

C. Code tasks

1. **Three rotations, one benchmark.** Implement rotation by $k \times$ single-shift, by temp buffer, and by three reversals. Verify all three agree for $n = 1000$ and every k in $0..1000$, then time all three at $n = 10^6$, $k = 500,000$.
2. **Break it: forget the modulo.** Call your reversal rotation with $k = n + 3$ without `k %= n`. Record the output or crash, then run under ASan and record the error and the offset it reports. Add the modulo and confirm the result equals rotation by 3.
3. **Break it: Dutch flag with the wrong increment.** Write the version that does `++mid` after swapping with `high`. Run it over 1000 random arrays of 0/1/2 and count how many outputs fail `is_sorted`. Print one failing case and locate the stranded 2.
4. **Break it: prefix sum overflow.** Build a prefix array of `int` over 10^5 elements each equal to 10^6 . Print the last entry, then rebuild with `long long` and print it again. Compile with `-fsanitize=signed-integer-overflow` and record the exact runtime diagnostic.
5. **Break it: Kadane initialised to zero.** Implement both versions and run them on `[-3,-1,-2]` and `[-5]`. Record all four outputs and state which two are wrong.
6. **Window versus recompute.** Implement max-sum-of-k both ways for $n = 10^6$, $k = 1000$. Report both times and the ratio, and confirm the ratio is approximately k .
7. **Duplicate detection, four ways.** Implement hash set, sort, negative marking and Floyd on the same input of 10^7 values. Report time, peak RSS, and whether the input array survived intact.

ANSWER KEY — Topics 52-73

A. Conceptual

1. `rotateRight(k) == rotateLeft(n - k mod n)`. One implementation means one place for an off-by-one; the wrapper is a single line that cannot be wrong in a new way.
2. Because rotation is periodic with period n ; without it, $k = n + 3$ does redundant work or indexes out of range. Negative k needs `if (k < 0) k += n`; since C++ `%` keeps the dividend's sign.
3. Reversing the first k and the remaining $n-k$ puts each *block* in reversed order internally but in the right relative position; the final full reversal un-reverses both blocks while swapping their order. Net effect: the blocks are exchanged.
4. Invariant: `[0,low)` all 0s, `[low,mid)` all 1s, `[mid,high]` unknown, `(high,n)` all 2s. The value swapped in from `high` is **unexamined**, so advancing `mid` would skip it.
5. All-equal input makes `a[j] < pivot` false for every j , so the pivot lands at `hi` and the split is $(n-1, 0)$, recursing n deep. Fixes: **Hoare partition** or **three-way (Dutch flag) partition**.
6. Make the **rarer** operation the $O(n)$ one: many queries, few updates $\rightarrow$ prefix sums; many range updates, one read $\rightarrow$ difference array. Both frequent and interleaved $\rightarrow$ **Fenwick or segment tree**.
7. $10^5 \times 10^9 = 10^{14}$, versus `int`'s max 2.147×10^9 . Overflow is UB and typically shows as a negative sum.
8. With `best = 0` the answer can never be negative, so `[-3,-1,-2]` returns 0 instead of -1 . Initialise both `cur` and `best` to `a[0]` and start at index 1.
9. It requires that shrinking the window is **monotone** — removing an element must move the aggregate towards feasibility. Negative values break it: in `[3, -5, 4]` with $S = 3$, removing -5 *increases* the sum.
10. You may move a pointer only if you can prove the element it abandons **cannot participate in any solution**. Sortedness supplies that proof; without it, two pointers is invalid.
11. The voting phase only guarantees "if a majority exists, it is the candidate". Without verification it returns the last surviving candidate — for `[1,2,3]`, **3**, which is not a majority.
12. Subarrays **210**; subsequences **1,048,576**; permutations 2.43×10^{18} . Only the first two can be enumerated (and the second only barely).
13. Because the suffix after the pivot position is by construction **non-increasing**; reversing a non-increasing sequence yields a non-decreasing one, in $O(n)$ rather than $O(n \log n)$.

B. Tracing and analysis

1. Effective $k = 10 \% 7 = 3$. After `reverse[0,3)`: `[3,2,1,4,5,6,7]`; after `reverse[3,7)`: `[3,2,1,7,6,5,4]`; after full reverse: **[4,5,6,7,1,2,3]**.
2. Start (0,0,5) `[2,0,2,1,1,0]`. `a[0]=2` $\rightarrow$ swap with high: (0,0,4) `[0,0,2,1,1,2]`. `a[0]=0` $\rightarrow$ swap low,mid (self): (1,1,4) unchanged. `a[1]=0` $\rightarrow$ (2,2,4). `a[2]=2` $\rightarrow$ swap with high: (2,2,3) `[0,0,1,1,2,2]`. `a[2]=1` $\rightarrow$ (2,3,3). `a[3]=1` $\rightarrow$ (2,4,3), loop ends. Result **[0,0,1,1,2,2]**, **4** swaps (two of them self-swaps).
3. `p = [0,4,2,9,10,5]`. `sum(1,3) = p[4]-p[1] = 10-4 = 6`; `sum(0,4) = p[5]-p[0] = 5`.
4. `d = [5,-3,0,-5,0,3]` (indices 0..5). Finalised prefix: **[5,2,2,-3,-3]** for indices 0..4.
5. `cur: -2,-3,4,3,1,2,7,4`. `best: -2,-2,4,4,4,7,7`. Winning subarray **[4,-1,-2,1,5]**, sum 7.
6. Windows: $2+1+5=8$, $1+5+1=7$, $5+1+3=9$, $1+3+2=6$. Sliding does **3 additions + 3 subtractions** after the initial 3; naive does $4 \times 3 = 12$ additions. Ratio approaches k for large k .

7. `[1,2,3,2,2]`: candidate **2**, count $3 > 2 \rightarrow 2$. `[1,2,3]`: candidate **3**, count 1 is not $> 1 \rightarrow -1$.
8. `[1,3,2]`, `[2,1,3]`, `[2,3,1]`, `[3,1,2]`, `[3,2,1]`. The sixth call returns **false** and leaves the array reset to `[1,2,3]`.

C. What to verify

1. All three agree on all 1001 values of k , including $k = 0$ and $k = n$. At $n = 10^6$, $k = 500,000$ expect roughly: reversal **~2-4 ms**, temp buffer **~2-3 ms** (but 2 MB of scratch), $k \times$ single-shift **> 100 s** — do not wait for it, compute 5×10^{11} operations and stop.
2. Without the modulo, `reverse(a, a+k)` with $k > n$ walks past the end: ASan reports **heap-buffer-overflow ... WRITE of size 4** with an offset of **12 bytes after** the block. With the modulo the output equals rotation by 3 exactly.
3. Roughly **60-75%** of random 0/1/2 arrays fail. A typical failure is a lone 2 left before the final 2-block, e.g. `[0,0,1,2,1,2]` — the stranded element is the one swapped in from **high** and never re-examined.
4. The `int` version prints a wrapped, often negative value ($10^{11} \bmod 2^{32}$); the correct value is **100,000,000,000**. UBSan prints **runtime error: signed integer overflow: ... cannot be represented in type 'int'**.
5. `[-3,-1,-2]`: correct **-1**, zero-initialised **0** (wrong). `[-5]`: correct **-5**, zero-initialised **0** (wrong).
6. Recompute $\approx 10^9$ **additions** (seconds); window $\approx 2 \times 10^6$ (a few ms). Ratio $\approx k/2$ to k , i.e. several hundred fold.
7. Expect: hash set fastest to write but **~400 MB+** peak RSS; sort **~1 s** and destroys order; negative marking **~30 ms** and mutates the array; Floyd **~40-80 ms**, $O(1)$ space, array **intact** — the only method that satisfies all three constraints at once.

HANDNOTE SUMMARY CARD — Rotation & Patterns

ROTATION AND THE EIGHT PATTERNS (Topics 52-73)

ROTATION left by k == right by $(n-k)$; ALWAYS $k \% n$, then $k += n$ if $k < 0$
k x shift $O(n*k)$ never temp buffer $O(n)$ time / $O(k)$ space
juggling $O(n)/O(1)$, gcd(n,k) cycles, poor locality
3 REVERSALS $O(n)/O(1)$, $\sim 3n/2$ swaps, sequential <- give this answer
reverse(0,k); reverse(k,n); reverse(0,n);
std::rotate(first, middle, last) in production

PARTITIONING one read pointer, one write pointer, one predicate
Lomuto pivot=a[hi]; i=lo-1; for j: if(a[j]<p) swap(a[++i],a[j]); swap(a[i+1],a[hi])
DNF low/mid/high; 0->swap(low++,mid++) 1->mid++ 2->swap(mid,high--) NO mid++

PREFIX / DIFFERENCE (duals)
prefix p[0]=0, p[i+1]=p[i]+a[i] sum(l,r) = p[r+1]-p[l] query $O(1)$
suffix s[n]=0, s[i]=s[i+1]+a[i] "everything to my right"
diff d[l]+=v; d[r+1]-=v; then one prefix pass update $O(1)$
choose: rare updates -> prefix | rare reads -> diff | both -> Fenwick/segtree
ALWAYS long long: $10^5 * 10^9 = 10^{14} >> \text{INT_MAX } 2.1e9$

KADANE cur = max(a[i], cur+a[i]); best = max(best, cur)
init BOTH from a[0], loop from 1 -> all-negative works; init 0 returns 0 (WRONG)

SLIDING WINDOW
fixed k: sum += a[i] - a[i-k] $O(n)$ not $O(n*k)$
variable: expand hi always; while(bad) shrink lo needs MONOTONE shrink
negatives present -> window invalid; use prefix + hash map or monotonic deque

TWO POINTERS discard only what cannot be in ANY solution
opposite ends sorted pair-sum: s<t -> ++i ; s>t -> --j
fast/slow midpoint, cycle entry, k-from-end
read/write compaction: dedup, move zeros, remove, partition

COUNTING TRICKS
majority Boyer-Moore vote, then VERIFY (without it, [1,2,3] -> 3)
missing XOR all indices and values (sum method overflows past $n \sim 65k$)
duplicate Floyd cycle on a[i] as next-index: $O(1)$ space, input unmodified
frequency direct index when range $\leq \sim 10^7$, else hash

COUNTS subarray $n(n+1)/2$ | subsequence 2^n | permutation $n!$
n=20: 210 | 1,048,576 | 2.4e18
enumerate all subarrays only to $n \sim 3000$; all permutations only to $n \sim 11$

NEXT PERMUTATION 1 find rightmost i with $a[i] < a[i+1]$ 2 rightmost j with $a[j] > a[i]$
3 swap(i,j) 4 reverse suffix after i (suffix is non-increasing)

TRAPS
no $k \% n$ -> heap-buffer-overflow | ++mid after high-swap -> stranded 2
Lomuto on all-equal -> $O(n^2)$, stack overflow at $n \sim 1e5$
prefix in int -> negative sums | Kadane best=0 -> 0 on all-negative
window with negatives -> silently wrong on one test in ten
missing undo swap in permute -> right count, wrong contents

SECTION 11 — SORTING

74. Sorting Arrays

Sorting is the precondition for binary search (Topic 42), for two-pointer techniques (Topic 65), for deduplication (Topic 50) and for most greedy algorithms. In production you call `std::sort`; the algorithms below matter because interviewers ask, because the constants differ by 100×, and because knowing *which* one your library uses explains its behaviour on adversarial input.

Algorithm	Best	Average	Worst	Space	Stable
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting	$O(n+K)$	$O(n+K)$	$O(n+K)$	$O(K)$	Yes
Radix	$O(d(n+K))$	$O(d(n+K))$	$O(d(n+K))$	$O(n+K)$	Yes

Measured on 10^6 random ints, `std::sort` takes roughly **10 minutes**, `std::sort` about **70 ms**, and a radix sort about **12 ms**. The Big-O table understates the gap; the constants are the story.

THE KEY INSIGHT

$O(n \log n)$ is the proven lower bound for *comparison* sorts — $n!$ possible orders need $\log_2(n!) \approx n \log_2 n$ comparisons to distinguish. Counting and radix beat it only because they never compare: they use the value as an index, which is the array trick from Topic 49 again.

sorting – comparison lower bound $\Omega(n \log n) \cdot 10^6$ ints: insertion ~600 s, `std::sort` ~70 ms, radix ~12 ms

75. Bubble Sort

```
void bubbleSort(int* a, int n) {
    for (int i = 0; i < n - 1; ++i) {
        bool swapped = false;                // the early-exit flag
        for (int j = 0; j < n - 1 - i; ++j)    // -i: tail is already sorted
            if (a[j] > a[j + 1]) { std::swap(a[j], a[j + 1]); swapped = true; }
        if (!swapped) return;                 // sorted input ->  $O(n)$ 
    }
}
```

[5,1,4,2] pass 1: (5,1)swap (5,4)swap (5,2)swap -> [1,4,2,5] largest sunk
pass 2: (1,4)ok (4,2)swap -> [1,2,4,5]
pass 3: (1,2)ok -> no swaps, exit early
comparisons $n(n-1)/2 = 6$ worst; swaps = number of INVERSIONS in the input

THE KEY INSIGHT

Bubble sort's swap count equals the number of inversions in the input — which is why it is $O(n)$ on already-sorted data and why "nearly sorted" matters. It is still the worst practical choice: insertion sort has the same properties and $2\text{--}3\times$ fewer writes.

bubble – Time $O(n^2)$, $O(n)$ if sorted with the flag · Space $O(1)$ · stable · never use in production

76. Selection Sort

```
void selectionSort(int* a, int n) {
    for (int i = 0; i < n - 1; ++i) {
        int m = i;
        for (int j = i + 1; j < n; ++j) if (a[j] < a[m]) m = j;
        if (m != i) std::swap(a[i], a[m]);    // at most n-1 swaps TOTAL
    }
}
```

Selection sort is the algorithm to choose when *writes are expensive and reads are cheap* — EEPROM, flash memory with limited erase cycles. It performs exactly $n-1$ swaps regardless of input, versus up to $n^2/2$ for bubble.

THE TRAP

Selection sort is **not stable**: swapping a distant minimum into place jumps it over equal elements. `[2a, 2b, 1]` becomes `[1, 2b, 2a]`. If you sort records by a secondary key first and then by a primary key with selection sort, the first sort is destroyed silently.

selection – Time $O(n^2)$ always, $n^2/2$ comparisons · Space $O(1)$ · exactly $n-1$ swaps · not stable

77. Insertion Sort

```
void insertionSort(int* a, int n) {
    for (int i = 1; i < n; ++i) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) { a[j + 1] = a[j]; --j; } // shift, Topic 39
        a[j + 1] = key;
    }
}
```

```
[5,2,4,6,1]  sorted prefix grows left to right
i=1 key=2  shift 5      -> [2,5,4,6,1]
i=2 key=4  shift 5      -> [2,4,5,6,1]
i=3 key=6  no shift     -> [2,4,5,6,1]
i=4 key=1  shift 6,5,4,2-> [1,2,4,5,6]    total shifts = inversions
```

THE KEY INSIGHT

Insertion sort is the fastest algorithm known for $n \leq \sim 16$ and for nearly-sorted input — which is why every industrial `std::sort` switches to it below a threshold (libstdc++ uses 16) rather than recursing further. It is not a toy; it is the base case of the real thing.

insertion – Time $O(n)$ sorted / $O(n^2)$ reversed · Space $O(1)$ · stable · optimal for $n \leq 16$ and near-sorted data

78. Merge Sort

```
void mergeSort(std::vector<int>& a, int lo, int hi,
               std::vector<int>& buf) {           // buf allocated ONCE
    if (hi - lo <= 1) return;
    int mid = lo + (hi - lo) / 2;
    mergeSort(a, lo, mid, buf);
    mergeSort(a, mid, hi, buf);
    int i = lo, j = mid, k = lo;
    while (i < mid && j < hi) buf[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i < mid) buf[k++] = a[i++];
    while (j < hi)  buf[k++] = a[j++];
    std::copy(buf.begin() + lo, buf.begin() + hi, a.begin() + lo);
}
```

```
[38,27,43,3,9,82,10]
split into single elements (log2 n = 3 levels)
merge pairwise upward: each LEVEL touches all n elements
levels = log2(n), work per level =  $O(n)$  ->  $O(n \log n)$  always
n =  $10^6$ : 20 levels x  $10^6$  =  $2 \times 10^7$  merge steps, ~90 ms
```

THE TRAP

Allocating the scratch buffer inside the recursion (`vector tmp` per call) turns $2n-1$ calls into $2n-1$ allocations — typically 3–5× slower than the sort itself and a textbook profiler surprise. Allocate once and pass it down, as above.

merge sort – Time $\Theta(n \log n)$ always · Space $O(n)$ buffer + $O(\log n)$ stack · stable · the choice for linked lists and external sorts

79. Quick Sort

```
void quickSort(int* a, int lo, int hi) {           // [lo, hi] inclusive
    while (lo < hi) {
        int p = partition(a, lo, hi);             // Topic 58
        if (p - lo < hi - p) {                     // recurse on the SMALLER
            quickSort(a, lo, p - 1);               // side: stack depth  $O(\log n)$ 
            lo = p + 1;                             // tail-call the larger side
        } else {
            quickSort(a, p + 1, hi);
            hi = p - 1;
        }
    }
}
```

Quicksort is the fastest general comparison sort in practice — roughly 2–3× faster than merge sort on ints — because it is in-place, cache-friendly, and its inner loop is a single comparison and a conditional swap. Its worst case is genuinely $O(n^2)$, which is why libstdc++ uses *introsort*: quicksort that switches to heapsort once recursion depth exceeds $2 \cdot \log_2 n$.

THE TRAP

Naive quicksort with the first or last element as pivot is $O(n^2)$ on *already sorted* input — the most common real-world case. Combined with recursion on both sides, it stack-overflows at $n \approx 10^5$. Symptom: a sort that is fast on random test data and crashes on production data that arrives sorted. Fix: median-of-three pivot, recurse on the smaller side, and a depth limit.

quicksort — Time $O(n \log n)$ avg / $O(n^2)$ worst · Space $O(\log n)$ with smaller-side recursion · not stable · fastest in practice

80. Heap Sort

```
void siftDown(int* a, int n, int i) {
    while (true) {
        int largest = i, l = 2*i + 1, r = 2*i + 2;    // implicit tree, Topic 153
        if (l < n && a[l] > a[largest]) largest = l;
        if (r < n && a[r] > a[largest]) largest = r;
        if (largest == i) return;
        std::swap(a[i], a[largest]); i = largest;
    }
}

void heapSort(int* a, int n) {
    for (int i = n/2 - 1; i >= 0; --i) siftDown(a, n, i);    // build:  $O(n)$ 
    for (int i = n - 1; i > 0; --i) {                      // extract max
        std::swap(a[0], a[i]);
        siftDown(a, i, 0);
    }
}
```

array [9,5,6,2,3] AS a binary heap (no pointers, Topic 153):

9(0)		parent(i) = (i-1)/2
/	\	left(i) = 2i+1
5(1)	6(2)	right(i) = 2i+2
/	\	a complete tree stored in an array with ZERO overhead
2(3)	3(4)	

THE KEY INSIGHT

Heap sort is the only $O(n \log n)$ sort with $O(1)$ space and no worst case — which is exactly why it is introsort's fallback. It loses in practice (2-3× slower than quicksort) because $2i+1$ jumps around memory: every sift-down step is a cache miss at large n .

heapsort — Time $\Theta(n \log n)$ always · Space $O(1)$ · not stable · poor locality: $\sim \log n$ misses per extract

81. Counting Sort

```
// Values in [0, K). Stable version – the one that composes into radix sort.
void countingSort(std::vector<int>& a, int K) {
    std::vector<int> cnt(K, 0), out(a.size());
    for (int x : a) ++cnt[x];                // 1. tally
    for (int i = 1; i < K; ++i) cnt[i] += cnt[i - 1]; // 2. prefix sums!
    for (int i = (int)a.size() - 1; i >= 0; --i)    // 3. BACKWARDS = stable
        out[--cnt[a[i]]] = a[i];
    a = out;
}
```

```
a = [2,5,3,0,2,3]    K = 6
tally   cnt = [1,0,2,2,0,1]
prefix  cnt = [1,1,3,5,5,6]    cnt[v] = number of elements <= v
place backwards: a[5]=3 -> cnt[3]-- = 4 -> out[4]=3 ... stable placement
```

THE TRAP

Counting sort is $O(n + K)$, so sorting 10 values in the range $[0, 10^9)$ allocates 4 GB and dies. The break-even is roughly $K \leq n \log n$; beyond it, use a comparison sort. Also: iterate *backwards* in step 3 — forwards produces a correct-looking but unstable result, which silently breaks radix sort.

counting sort – Time $O(n+K)$ · Space $O(K+n)$ · stable if placed backwards · unusable when $K \gg n$

82. Radix Sort

Sort by digit, least significant first, using a stable counting sort per digit. Sorting 10^6 32-bit ints as four 8-bit digits takes four passes — about 12 ms, roughly $6\times$ faster than `std::sort`.

```
void radixSort(std::vector<unsigned>& a) {
    std::vector<unsigned> buf(a.size());
    for (int shift = 0; shift < 32; shift += 8) {        // 4 passes of 8 bits
        int cnt[256] = {};
        for (unsigned x : a) ++cnt[(x >> shift) & 255];
        for (int i = 1; i < 256; ++i) cnt[i] += cnt[i - 1];
        for (int i = (int)a.size() - 1; i >= 0; --i)
            buf[--cnt[(a[i] >> shift) & 255]] = a[i];
        a.swap(buf);
    }
}
```

THE KEY INSIGHT

Radix sort works *only* because each pass is stable — earlier passes' ordering must survive later ones. This is the concrete reason stability is worth caring about (Topic 86), not an abstract property.

THE TRAP

Radix sorting signed ints with a naive byte extraction puts negatives after positives, because the sign bit makes them numerically larger as unsigned. Fix: XOR the top bit (`x ^ 0x80000000`) on the way in and out. Symptom: perfectly sorted output that begins at 0 and ends with all the negative numbers.

radix – Time $O(d(n+K))$: 4 passes for 32-bit · Space $O(n+256)$ · stable · ints/strings only, no custom comparator

83. Bucket Sort

Scatter elements into K buckets by value range, sort each bucket, concatenate. Expected $O(n)$ when the input is uniformly distributed and $K \approx n$; degenerates to the inner sort's cost when it is not.

```
void bucketSort(std::vector<double>& a) {           // values in [0, 1)
    int n = a.size();
    std::vector<std::vector<double>>> b(n);
    for (double x : a) b[(int)(x * n)].push_back(x); // scatter
    for (auto& bucket : b) std::sort(bucket.begin(), bucket.end());
    int k = 0;
    for (auto& bucket : b) for (double x : bucket) a[k++] = x;
}
```

THE TRAP

Skewed input puts everything in one bucket and the cost becomes the inner sort's $O(n \log n)$ — plus n vector allocations you did not need. Bucket sort is a distribution assumption wearing an algorithm's clothes; verify the distribution before choosing it.

bucket sort – Time $O(n)$ expected uniform / $O(n \log n)$ skewed · Space $O(n + K)$ · n allocations unless preallocated

84. C++ Built-in Sorting

```
std::sort(v.begin(), v.end());           // introsort,  $O(n \log n)$ , unstable
std::stable_sort(v.begin(), v.end());    // merge-based,  $O(n \log n)$ ,
                                         //  $O(n)$  extra memory
std::partial_sort(v.begin(), v.begin()+k, v.end()); // top k,  $O(n \log k)$ 
std::nth_element(v.begin(), v.begin()+k, v.end());  // k-th,  $O(n)$  EXPECTED
std::sort(v.begin(), v.end(), std::greater<int>{}); // descending
std::is_sorted(v.begin(), v.end());
```

Need	Call	Cost
Full order	<code>sort</code>	$O(n \log n)$, in place
Ties keep order	<code>stable_sort</code>	$O(n \log n) + O(n)$ memory
Top k	<code>partial_sort</code>	$O(n \log k)$
Median / k-th only	<code>nth_element</code>	$O(n)$ expected
Group by predicate	<code>partition</code>	$O(n)$

THE KEY INSIGHT

Finding the median of 10^6 elements with `sort` costs ~70 ms; with `nth_element` it costs ~6 ms, because you asked for less. "Sort then take element k " is the single most common needless $O(n \log n)$ in production code.

```
std::sort – introsort: quick + heap fallback + insertion below 16 · ~70 ms / 106 ints ·  
stable_sort: +O(n) memory, ~1.4× slower
```

85. Custom Comparator

```
struct Rec { std::string name; int age; };  
std::vector<Rec> v;  
  
// Sort by age ascending, then name ascending. Tuples do the tie-breaking.  
std::sort(v.begin(), v.end(), [](const Rec& a, const Rec& b) {  
    return std::tie(a.age, a.name) < std::tie(b.age, b.name);  
});  
  
// Sort indices instead of records when records are expensive to move:  
std::vector<int> idx(v.size());  
std::iota(idx.begin(), idx.end(), 0);  
std::sort(idx.begin(), idx.end(),  
    [&](int i, int j) { return v[i].age < v[j].age; });
```

THE TRAP

The comparator must be a *strict weak ordering*: `comp(a,a)` must be false. Writing `<=` instead of `<` violates it and `std::sort` then reads past the end of the range — a genuine segfault inside the standard library, not a wrong answer. It typically only triggers above the insertion-sort threshold, so it passes every small test. Compile with `-D_GLIBCXX_DEBUG` to get `Error: comparison doesn't meet irreflexive requirements` instead.

```
comparator – 1 call per comparison, ~n log n calls · lambda inlines to zero cost · <= in a  
comparator = UB, segfault
```

86. Stable vs Unstable Sorting

A stable sort preserves the relative order of elements that compare equal. That matters in exactly two situations, and in both it matters completely.

```
records sorted by NAME, then sorted by AGE:  
stable      (Ann,30) (Bob,30) (Cid,25) -> (Cid,25) (Ann,30) (Bob,30) names kept  
unstable    same input                -> (Cid,25) (Bob,30) (Ann,30) names LOST  
use 1: multi-key sorting by repeated passes (last key first)  
use 2: radix sort, which is stable counting sorts stacked (Topic 82)
```

THE KEY INSIGHT

Stability lets you sort by multiple keys with repeated single-key passes, applying the *least* significant key first. Without it you need one comparator that handles all keys at once. Both work; only one composes.

```
stability – stable_sort costs O(n) memory and ~1.4× time · sort free but reorders ties arbitrarily
```

87. Sorting Best Practices

Situation	Use
General case	<code>std::sort</code>
Ties must keep order	<code>std::stable_sort</code>
Only need the k smallest	<code>partial_sort</code> / <code>nth_element</code>
Small integers, $K \leq n \log n$	Counting sort
Fixed-width ints, $n \geq 10^5$	Radix sort (6× faster)
$n \leq 16$ or nearly sorted	Insertion sort
Data bigger than RAM	External merge sort (Topic 165)
Records are large	Sort an index array (Topic 85)
Already sorted, need to insert	Do not re-sort: <code>lower_bound</code> + insert

THE TRAP

Re-sorting an already-sorted vector after each insertion is the most common accidental $O(n^2 \log n)$ in application code. Inserting into the correct position with `lower_bound` is $O(n)$ per insert and usually 100× faster in aggregate. Symptom: latency that grows quadratically with the size of a list that "only gets a few new items."

selection – sort once $O(n \log n)$ · maintain sorted $O(n)$ per insert · re-sort per insert $O(n^2 \log n)$: the bug

88. PHP Array Functions

PHP's "array" is not an array: it is an ordered hash map, so every C++ intuition about contiguity is wrong there. This matters if you move between the two languages, because the same word describes structures with different asymptotics and 5-10× different memory footprints.

```
<?php
$a = [5, 3, 8, 1];
sort($a);           // values ascending, KEYS ARE REINDEXED 0..n-1
rsort($a);          // descending
asort($a);          // sort by value, PRESERVE key association
ksort($a);          // sort by key
usort($a, fn($x, $y) => $x <=> $y); // custom comparator (spaceship)
uasort($a, $cmp);   // custom, keys preserved
array_multisort($a, $b); // parallel arrays
```

PHP	C++ equivalent	Note
<code>sort()</code>	<code>std::sort</code> + reindex	Destroys keys
<code>asort()</code>	Sort a vector of pairs by <code>.second</code>	Keeps keys
<code>usort()</code>	<code>std::sort</code> with comparator	Comparator returns int, not bool
<code>array_push</code>	<code>push_back</code>	Amortised $O(1)$ in both
<code>in_array</code>	<code>std::find</code>	$O(n)$ in both
<code>isset(\$a[\$k])</code>	<code>map::contains</code>	$O(1)$ in PHP, $O(1)/O(\log n)$ in C++

THE TRAP

PHP comparators return `-1/0/1`, C++ comparators return a `bool` meaning "strictly less". Porting a PHP `usort` callback to C++ verbatim returns 1 for "greater" — which is `true` — producing a reversed-and-invalid ordering that can crash `std::sort` (Topic 85).

PHP sorts — all $O(n \log n)$ · plus hash-table overhead ~80 bytes/element vs 4 for a C++ int array

89. C++ vs PHP Arrays

Property	C++ <code>vector<int></code>	PHP <code>array</code>
Underlying structure	Contiguous block	Ordered hash table
Bytes per int element	4	~36–80 (bucket + zval)
Index access	$O(1)$, one instruction	$O(1)$ expected, hash + probe
Keys	0..n–1 only	Any int or string, sparse
Mixed types	No	Yes
Insert at front	$O(n)$	$O(n)$ (<code>array_unshift</code> reindexes)
Iteration order	Memory order	Insertion order, guaranteed
Cache behaviour	1 miss per 16 ints	Effectively 1 miss per element

THE KEY INSIGHT

A PHP array is closer to C++'s `std::unordered_map` with insertion order than to `std::vector`. When you port PHP logic that indexes `$a[1000000]` on a 3-element array, the PHP version costs nothing and the C++ version needs 4 MB — the sparse-key assumption does not survive translation. Reach for `unordered_map`, not `vector`.

C++ vs PHP — memory 4 B vs ~36–80 B per int · both $O(1)$ index · PHP: sparse keys free, C++: dense keys required

PRACTICE SET — Topics 74-89

A. Conceptual

1. Why is $\Omega(n \log n)$ a lower bound for comparison sorts, and by what mechanism do counting and radix sort escape it?
2. Bubble sort and insertion sort share the same complexity profile. Give the concrete reason to prefer insertion sort.
3. In what hardware situation is selection sort the correct choice, and what property makes it so?
4. Why is quicksort faster than merge sort in practice despite a worse worst case? Give two mechanisms.
5. Explain introsort's depth limit: what triggers the switch, what does it switch to, and what guarantee does that restore?
6. Why must counting sort place elements by iterating the input backwards?
7. Give the concrete reason stability matters for radix sort — not the definition, the failure that occurs without it.
8. What exactly goes wrong when a comparator uses `<=`, and why does it usually pass small tests?
9. When should you sort an array of indices instead of the records themselves?
10. Give the cost of "sort then take element k" versus `nth_element` for $n = 10^6$, with numbers.
11. Why is a PHP array not comparable to a C++ `vector`, and which C++ container is the honest equivalent?
12. A PHP `usort` callback is ported verbatim to C++. Describe the failure and its mechanism.

B. Tracing and analysis

1. Bubble sort `[5,1,4,2,8]`. Show the array after each pass, the swap count per pass, and state which pass triggers the early exit.
2. Insertion sort `[5,2,4,6,1,3]`: give the array after each `i` and the total number of shifts. Compare that total with the number of inversions in the input.
3. Merge sort `[38,27,43,3]`: draw the recursion tree, list every merge and the comparisons each performs, and total them.
4. Lomuto-partition `[7,2,9,1,5]` with pivot 5. Give `(i, j)` and the array at each step and the returned index.
5. Counting sort `[2,5,3,0,2,3]` with $K = 6$: give the tally array, the prefix array, and the output placement order for the last three elements.
6. Radix sort 4 unsigned values `[0x0102, 0x0201, 0x0101, 0x0202]` with 8-bit digits: show the array after the low pass and after the high pass.
7. Compute the number of comparisons for `std::sort` on $n = 10^6$ (use $n \log_2 n$) and the time at 5 ns per comparison. Compare with a measured 70 ms and explain the discrepancy.
8. An array of 10^6 records of 200 bytes each is sorted directly versus by index. Compute bytes moved in each case, assuming $n \log_2 n / 2$ swaps.

C. Code tasks

1. **Six sorts, one harness.** Implement bubble, selection, insertion, merge, quick and heap. Verify each against `std::sort` on 10^4 random arrays including all-equal, sorted, and reversed input. Report times at $n = 10^4$ and $n = 10^5$.

2. **Break it: quicksort on sorted input.** Implement quicksort with `pivot = a[hi]` and recursion on both sides. Run on random input at $n = 10^5$ and record the time; then run on *already sorted* input of the same size and record what happens. Add median-of-three and smaller-side recursion and re-run both.
3. **Break it: an invalid comparator.** Sort 10^5 ints with `[](int a, int b){ return a <= b; }`. Record the behaviour, then rebuild with `-D_GLIBCXX_DEBUG` and record the exact message. Then run the same comparator on 10 elements and explain why it appeared to work.
4. **Break it: unstable multi-key sort.** Build 10^4 records of `{name, age}`. Sort by name, then by age with `std::sort`; check whether names remain ordered within each age group and count the violations. Repeat with `stable_sort` and confirm zero violations.
5. **Break it: counting sort with a huge range.** Counting-sort 10 values in the range $[0, 10^9)$. Record what happens (allocation failure or OOM kill), then compute the memory the algorithm requested. Fix by switching to `std::sort` and state the break-even K .
6. **Radix versus std::sort.** Sort 10^7 random `unsigned` with both. Report both times and the ratio. Then repeat with signed ints in a naive radix and record where the negatives land.
7. **nth_element.** Find the median of 10^6 ints with `sort` and with `nth_element`. Report both times and confirm the medians match.

ANSWER KEY — Topics 74-89

A. Conceptual

1. A comparison sort's decision tree must have $n!$ leaves; a binary tree with $n!$ leaves has depth $\geq \log_2(n!) \approx n \log_2 n - 1.44n$. Counting/radix never compare — they **use the value as an index**, which is information the comparison model does not have.
2. Insertion sort **shifts** (one write per moved element) where bubble **swaps** (three writes). Same comparison count, **2-3× fewer writes**, and it is the standard base case of introsort.
3. When **writes are far more expensive than reads** — EEPROM/flash with limited erase cycles. Selection sort performs exactly $n-1$ swaps regardless of input.
4. (1) It is **in place** — no $O(n)$ buffer and no copy-back pass. (2) Its inner loop is **sequential and cache-friendly**, with a branchless conditional swap.
5. Recursion depth exceeding $2 \cdot \log_2 n$ triggers a switch to **heapsort**, restoring a hard $O(n \log n)$ worst case while keeping quicksort's average speed. Below 16 elements it switches to insertion sort.
6. Because `cnt[v]` holds the *end* position of value *v*'s block; decrementing it as you walk backwards assigns later-appearing equal elements to later slots, which is exactly **stability**.
7. Radix processes digits least-significant first. If a later pass reorders elements that tie on the current digit, it **destroys the ordering established by all previous digits**, and the final output is simply wrong.
8. `<=` makes `comp(a,a)` true, violating irreflexivity. `std::sort`'s inner loop relies on the pivot acting as a sentinel; with an invalid comparator the scan **runs past the end of the range** — a segfault inside the library. Small tests pass because arrays under **16** elements take the insertion-sort path, which does not use the sentinel.
9. When `sizeof(record)` is large. Sorting indices moves 4-8 bytes per swap instead of the whole record, at the cost of one indirection per comparison.
10. `sort` $\approx$ **70 ms**; `nth_element` $\approx$ **6 ms** (expected $O(n)$, quickselect). Roughly **10×**.
11. A PHP array is an **ordered hash table**: arbitrary and sparse keys, insertion-order iteration, ~36-80 bytes per element. The honest C++ equivalent is `unordered_map` **plus an insertion-order list**, not `vector`.
12. PHP callbacks return **-1/0/1**; C++ expects **bool "strictly less"**. Returning 1 for "greater" is **true**, so the comparator claims $a < b$ in both directions — an invalid strict weak ordering, hence UB and a possible crash.

B. Tracing and analysis

1. P1 `[1,4,2,5,8]` (3 swaps); P2 `[1,2,4,5,8]` (1 swap); P3 `[1,2,4,5,8]` (**0 swaps → early exit**). Total swaps **4** = the number of inversions.
2. `[2,5,4,6,1,3]`, `[2,4,5,6,1,3]`, `[2,4,5,6,1,3]`, `[1,2,4,5,6,3]`, `[1,2,3,4,5,6]`. Shifts: $1+1+0+4+3 = 9$, exactly the inversion count.
3. Splits: `[38,27]` `[43,3]` → `[38][27]` `[43][3]`. Merges: (38,27) 1 comparison; (43,3) 1; (`[27,38]`, `[3,43]`) 3. Total **5** comparisons; output `[3,27,38,43]`.
4. $j=0$ $7 < 5$ no ($i=-1$); $j=1$ $2 < 5$ yes swap `a[0],a[1]` → `[2,7,9,1,5]` ($i=0$); $j=2$ $9 < 5$ no; $j=3$ $1 < 5$ yes swap `a[1],a[3]` → `[2,1,9,7,5]` ($i=1$); place pivot swap `a[2],a[4]` → `[2,1,5,7,9]`, return **2**.
5. Tally `[1,0,2,2,0,1]`; prefix `[1,1,3,5,5,6]`. Backwards: `a[5]=3` → `out[4]`; `a[4]=2` → `out[2]`; `a[3]=0` → `out[0]`.
6. Low bytes are 02, 01, 01, 02, so the stable low pass gives **[0x0201, 0x0101, 0x0102, 0x0202]** (the two 01s keep their relative order). High bytes are then 02, 01, 01, 02, and the

high pass gives **[0x0101, 0x0102, 0x0201, 0x0202]** — correct only because the second pass preserved the first pass's ordering among ties.

7. $n \log_2 n = 10^6 \times 20 = 2 \times 10^7$ comparisons $\rightarrow$ **100 ms** at 5 ns. Measured 70 ms is *faster* because comparisons are pipelined and mostly branch-predicted, and the constant in introsort is below $1 \cdot n \log_2 n$.
8. Direct: 10^7 swaps $\times$ 200 bytes $\times$ 2 (read+write) $\approx$ **4 GB** moved. By index: $10^7 \times 4$ bytes $\times$ 2 = **80 MB** — **50 $\times$ less traffic**.

C. What to verify

1. At $n = 10^5$ expect roughly: bubble **10-20 s**, selection **4-8 s**, insertion **2-4 s**, merge **10 ms**, quick **7 ms**, heap **12 ms**, `std::sort` **6 ms**.
2. Random: **~7 ms**. Sorted: **segmentation fault** from stack exhaustion at depth 10^5 (or minutes of runtime with an explicit stack). With median-of-three plus smaller-side recursion, sorted input runs in **~4 ms** — faster than random, because the partitions are balanced.
3. Typically a **segfault or an infinite loop** inside `__introsort_loop`. With `-D_GLIBCXX_DEBUG: Error: comparison doesn't meet irreflexive requirements, assert(!(a < a))`. Ten elements take the insertion-sort path, which never runs off the end.
4. `std::sort` typically produces **thousands** of ordering violations within age groups; `stable_sort` gives **0**.
5. It requests $10^9 \times 4$ bytes = **4 GB** and either throws `std::bad_alloc` or is killed by the OOM killer (exit 137). Break-even is roughly **$K \leq n \log_2 n$** .
6. Radix **~120 ms** versus `std::sort` **~800 ms** at $n = 10^7$ — about **6 $\times$** . Naive signed radix puts **all negatives after all positives**; the fix is to flip the sign bit.
7. `sort` **~70 ms**, `nth_element` **~6 ms**, identical medians.

HANDNOTE SUMMARY CARD — Sorting

SORTING (Topics 74-89)

	best	average	worst	space	stable	in place
bubble	$O(n)^2$	$O(n^2)$	$O(n^2)$	$O(1)$	yes	yes
selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	NO	yes
insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	yes	yes
merge	$n \log n$	$n \log n$	$n \log n$	$O(n)$	yes	no
quick	$n \log n$	$n \log n$	$O(n^2)$	$O(\log n)$	NO	yes
heap	$n \log n$	$n \log n$	$n \log n$	$O(1)$	NO	yes
counting	$O(n+K)$	$O(n+K)$	$O(n+K)$	$O(K)$	yes*	no
radix	$d(n+K)$	$d(n+K)$	$d(n+K)$	$O(n+K)$	yes	no

* with the swapped/early-exit flag ; counting stable only if placed BACKWARDS

MEASURED, 10^6 random ints, -O2

insertion ~600 s | heap ~120 ms | merge ~90 ms | std::sort ~70 ms | radix ~12 ms
nth_element (median only) ~6 ms partial_sort top-100 ~15 ms

STD LIBRARY

sort introsort = quick + heapsort past depth $2\log_2(n)$ + insertion <16
stable_sort merge based, $O(n)$ memory, ~1.4x slower
partial_sort top k, $O(n \log k)$ nth_element k-th only, $O(n)$ expected
lower_bound first >= key upper_bound first > key (sorted only)

COMPARATOR RULES

return a STRICT weak ordering: comp(a,a) MUST be false
multi-key: return std::tie(a.k1,a.k2) < std::tie(b.k1,b.k2);
large records: sort an index array, not the records (50x less byte traffic)
<= in a comparator -> UB -> segfault inside std::sort above n=16

STABILITY: WHY IT EXISTS

multi-key by repeated passes, LEAST significant key first
radix sort is stacked stable counting sorts - unstable = wrong output

CHOOSING

general -> std::sort ties matter -> stable_sort
k-th / median -> nth_element top k -> partial_sort
small K ints -> counting fixed-width ints, $n \geq 1e5$ -> radix (6x)
 $n \leq 16$ or nearly sorted -> insertion bigger than RAM -> external merge
maintaining a sorted list -> lower_bound + insert, NEVER re-sort per insert

PHP CONTRAST (Topics 88-89)

PHP array = ordered hash map: sparse keys free, ~36-80 B/element, 1 miss each
sort() reindexes keys | asort() keeps them | usort() callback returns -1/0/1
C++ comparator returns bool -> porting a PHP callback verbatim is UB

TRAPS

quicksort pivot=last on SORTED input -> $O(n^2)$ + stack overflow at $n=1e5$
merge sort allocating the buffer per call -> 3-5x slowdown
counting sort with $K=1e9$ -> 4 GB request, OOM kill (exit 137)
naive radix on signed ints -> all negatives sorted AFTER positives
sort-then-index-k when nth_element would do -> 10x wasted

PART III CHEATSHEET — Operations & Patterns

PART III: PATTERN RECOGNITION IN ONE PAGE

THE PHRASE IN THE PROBLEM	-> THE TECHNIQUE	TOPIC
"contiguous subarray"	sliding window / Kadane / prefix	60,63,64
"sum of range, many queries"	prefix sums (long long!)	60
"add v to every element in [l,r]"	difference array	62
"maximum sum subarray"	Kadane	63
"longest/shortest subarray with"	variable window (non-negative)	64
"sorted array" + "pair/triplet"	two pointers from both ends	65
"find the middle / cycle"	fast and slow pointers	66
"appears more than n/2 times"	Boyer-Moore vote + VERIFY	67
"one number missing / appears once"	XOR	68
"duplicate, O(1) space, read-only"	Floyd cycle on a[i] as next index	69
"move all X to one side"	read/write compaction	50,51,58
"0s, 1s and 2s"	Dutch national flag	59
"rotate by k"	3 reversals, $k \% n$	57
"next lexicographic arrangement"	next permutation (4 steps)	73
"k-th largest / median"	nth_element, O(n)	84
"sort by two keys, ties matter"	stable_sort + std::tie	85,86
"top k of a million"	partial_sort, O(n log k)	84
"values in a small range"	counting/radix sort, or freq[]	49,81,82

COMPOSITION - NOTHING NEW IS EVER NEEDED

rotation	= reverse + reverse + reverse	(Topic 44)
next perm	= scan + swap + reverse suffix	(Topic 44)
quicksort	= partition, recursively	(Topic 58)
radix sort	= stable counting sort, once per digit	(Topic 81)
dedup/zeros/partition	= ONE read-write loop, different predicate	
range update + range query	= difference array + prefix pass	(Topics 60,62)

COMPLEXITY BUDGET (1 second, $\sim 10^8$ - 10^9 simple ops)

$n \leq 10$	$n!$	permutations
$n \leq 20$	2^n	subsets/bitmask
$n \leq 3000$	n^2	all subarrays, DP tables
$n \leq 10^5$	$n \log n$	sorting, segment tree
$n \leq 10^7$	n	single pass, two pointers, prefix
$n > 10^8$	$O(\log n)/O(1)$	maths, binary search only

THE FIVE QUESTIONS BEFORE WRITING ANY ARRAY ALGORITHM

1. Is it sorted? (unlocks binary search + two pointers)
2. Are the values bounded? (unlocks freq[] / counting sort)
3. Contiguous or not? (subarray -> window; subsequence -> DP)
4. May I modify the input? (unlocks in-place marking, sorting)
5. Do updates and queries interleave? (if yes: Fenwick/segtree, not arrays)

TEST MATRIX FOR EVERY ALGORITHM IN THIS PART

$n=0$ | $n=1$ | all equal | already sorted | reverse sorted | all negative
 $k=0$ | $k=n$ | $k>n$ | duplicates present | INT_MAX values (overflow) | empty range

PART IV

Two Dimensions

A matrix is a one-dimensional array with a multiplication in the index expression. Once that is fixed in your head, every topic here — transpose, rotation, spiral, flood fill, 2-D prefix sums — is index bookkeeping over a flat buffer. Checklist items 90–112.

SECTION 12 — 2D ARRAYS & MATRIX

90. Multi-Dimensional Arrays

C++ has no two-dimensional memory. `int m[3][4]` is an array of 3 objects, each of which is an array of 4 ints — 12 ints laid end to end, row after row. The second subscript is not a second lookup; both subscripts collapse into the single address formula of Topic 8.

```
int m[3][4];
// m[i][j] lives at: base + (i * 4 + j) * sizeof(int)
// The 4 is COLS. The compiler bakes it in from the type, which is why
// the column count is part of the type and the row count is not (Topic 18).

int t[2][3][4];      // 24 ints: t[i][j][k] at base + ((i*3 + j)*4 + k)*4
// Generally: stride of the last dimension is 1, and each dimension's
// stride is the product of all dimensions to its right.
```

```
int m[3][4] = {{1,2,3,4},{5,6,7,8},{9,10,11,12}};
```

logical	actual memory (row-major, Topic 9)
j=0 1 2 3	
i=0 [1 2 3 4]	[1 2 3 4 5 6 7 8 9 10 11 12]
i=1 [5 6 7 8]	^ ^ ^
i=2 [9 10 11 12]	m[0][0] m[1][0] m[2][2]
	+0 +16 bytes +40 bytes


```
sizeof(m)      = 48    (whole block)
sizeof(m[0])   = 16    (one row)
sizeof(m[0][0]) = 4
m[1] is an int[4] that decays to int* pointing at element 4.
```

THE KEY INSIGHT

Prefer a *flat* array plus an explicit index function — `a[i*COLS + j]` — over `int m[R][C]` for anything sized at runtime. It is the same arithmetic written where you can see it, it survives being passed to a function (Topic 18), and it is what every serious numeric library does (Topics 145, 164).

THE TRAP

`void f(int m[][])` does not compile, and `void f(int** m)` compiles but is a *different type* — passing `int m[3][4]` to it is not a conversion, it is a bug. Symptom: "cannot convert `int (*)[4]` to `int**`", or, if you force it with a cast, garbage values and an immediate segfault because the callee dereferences your integers as pointers. You must write `void f(int m[][4], int rows)` or take the flat pointer.

2-D array — Time $O(1)$ access · Space $O(R \cdot C \cdot \text{sizeof}(T))$ contiguous · one row = $C \times \text{sizeof}(T)$ bytes; the row stride is compile-time constant, so the address is still one LEA

91. Jagged Arrays Concept

A jagged array is an array of pointers to independently allocated rows, so rows may have different lengths. It buys ragged shapes and pays with a second pointer dereference and lost contiguity.

```
#include <vector>
std::vector<std::vector<int>> jag = {{1,2},{3,4,5,6},{7}};
// jag[1][2] == 5. Rows live wherever the allocator put them.

int** raw = new int*[3];           // C-style equivalent
raw[0] = new int[2]; raw[1] = new int[4]; raw[2] = new int[1];
// ...
for (int i = 0; i < 3; ++i) delete[] raw[i]; // rows first
delete[] raw;                          // then the spine (Topic 27)
```

vector<vector<int>>	flat vector<int> with COLS
spine [ptr][ptr][ptr]	[1 2 3 4 5 6 7 8 9 10 11 12]
	one allocation, one indirection
v v v	row i starts at i*COLS
[1,2] [3,4,5,6] [7]	
3 extra allocations, 3 cache lines minimum,	
rows arbitrarily far apart in memory	

THE TRAP

`vector<vector<int>>` is the default choice of most C++ beginners for a rectangular matrix, and it is the wrong one. For a 1000×1000 grid it performs 1001 allocations instead of 1, scatters rows across the heap, and defeats the prefetcher — measurably 2–4× slower on a full scan than a flat `vector<int>(R*C)`. Use it only when rows genuinely differ in length (adjacency lists, Topic 152).

jagged — Time $O(1)$ access with **two** dependent loads · Space $O(R \cdot C) + R$ pointers + R allocation headers (~16–32 B each) · rows not contiguous, so no cross-row prefetch

92. 2D Matrix Basics

A matrix is a 2-D array with mathematical intent: rows are equations or records, columns are variables or fields. The vocabulary of the rest of this section: an $R \times C$ matrix has R rows and C columns, is *square* when $R = C$, and `m[i][j]` is read "row i , column j " — always row first.

```
constexpr int R = 3, C = 4;
std::vector<int> m(R * C); // flat, preferred
inline int at(int i, int j) { return i * C + j; } // the index function

m[at(1, 2)] = 7; // row 1, column 2
// Every algorithm below is a different traversal order over at(i, j).
```

C = 4 columns					
	j=0	j=1	j=2	j=3	
	+-----+-----+-----+-----+				
i=0	0,0	0,1	0,2	0,3	row 0
	+-----+-----+-----+-----+				
i=1	1,0	1,1	1,2	1,3	row 1
	+-----+-----+-----+-----+				
i=2	2,0	2,1	2,2	2,3	row 2
	+-----+-----+-----+-----+				
	i indexes DOWN, j indexes ACROSS. Row first, always.				

matrix — the entire section is one array plus the function `i*C + j`; nothing else is added

93. Declaring 2D Matrix

Four ways, in decreasing order of how often they are the right answer.

```
// 1. Flat vector + index function – runtime size, one allocation. Default.
std::vector<int> m(R * C);                m[i * C + j]

// 2. std::vector<std::vector<int>> – only if rows differ in length.
std::vector<std::vector<int>> v(R, std::vector<int>(C, 0));

// 3. std::array – compile-time size, lives on the stack, zero overhead.
std::array<std::array<int, 4>, 3> a{};      // contiguous, size known

// 4. Raw C array – fine at global scope with constant bounds (Topic 173).
int g[1000][1000];                        // 4 MB in .bss, zeroed
```

THE TRAP

`int m[R][C]` with `R`, `C` read from input is a variable-length array: a GCC extension, not standard C++. It compiles with `g++`, fails on MSVC, and puts an unbounded allocation on an 8 MB stack (Topic 7). Symptom: works on your machine at `n=100`, segfaults on the judge at `n=5000`. Use option 1.

declaration – flat vector: 1 allocation · vector-of-vectors: `R+1` · `std::array`: 0 · raw global: 0, but the size must be a compile-time constant

94. Initializing 2D Matrix

```
int a[2][3] = {{1,2,3},{4,5,6}};      // explicit, clearest
int b[2][3] = {1,2,3,4,5,6};          // legal: rows filled in order
int c[2][3] = {};                     // ALL six elements zeroed
int d[2][3] = {{1},{4}};              // {1,0,0},{4,0,0} - rest zero-filled

std::vector<int> m(R * C, 0);           // flat, zeroed
std::vector<std::vector<int>> v(R, std::vector<int>(C, -1));

// identity matrix, flat:
for (int i = 0; i < n; ++i) m[i * n + i] = 1;    // touch the diagonal only
```

THE TRAP

`int a[2][3]`; at block scope is **not** zeroed — it contains whatever was on the stack. Symptom: a summing loop returns a different huge number on each run, or the right answer in a debug build and nonsense at `-O2`. Write `= {}`. Same rule as Topic 129.

initialization – `= {}` is `O(R·C)` writes but usually a single `memset`; a global array is zeroed by the loader at no runtime cost (Topic 7)

95. Taking Matrix Input

```
#include <iostream>
#include <vector>

int main() {
    std::ios::sync_with_stdio(false);    // Topic 173: 3-10x faster input
    std::cin.tie(nullptr);

    int R, C;
    std::cin >> R >> C;
    std::vector<int> m(R * C);
    for (int i = 0; i < R; ++i)
        for (int j = 0; j < C; ++j)
            std::cin >> m[i * C + j];    // reads row by row, whitespace-
                                         // separated; newlines are irrelevant

    return 0;
}
```

THE TRAP

Never size the vector *before* reading *R* and *C*, and never trust that the input has *R·C* numbers. If the stream fails mid-read, `cin` silently leaves the remaining elements at their previous value and every subsequent `>>` is a no-op. Symptom: the first half of the matrix is correct and the second half is zeros. Check `if (!(std::cin >> x)) break;` when input may be malformed.

input – Time $O(R \cdot C)$ · with `sync_with_stdio(false)` about 30–60 ns per integer; without it, 3–10× that

96. Printing Matrix

```
void print(const std::vector<int>& m, int R, int C) {
    for (int i = 0; i < R; ++i) {
        for (int j = 0; j < C; ++j)
            std::cout << m[i * C + j] << (j + 1 == C ? '\n' : ' ');
    }
}
// Aligned columns for debugging: std::cout << std::setw(4) << value;
```

printing IS the row-major traversal: *i* outer, *j* inner
-> visits memory in address order -> 16 ints per cache line (Topic 9)
the same loop with *i* inner and *j* outer prints the TRANSPOSE
and runs up to 22x slower on a large matrix.

print – Time $O(R \cdot C)$ · I/O dominates: `'\n'` not `std::endl`, which flushes and can cost 100× on large output

97. Accessing Rows

A row is a contiguous run of *C* elements. That makes row operations the fast direction: copy, sum, sort or `memcpy` a row and you are walking memory in stride 1.

```
// sum of row i – sequential, prefetcher-friendly
long long rowSum(const std::vector<int>& m, int i, int C) {
    long long s = 0;
    for (int j = 0; j < C; ++j) s += m[i * C + j];
    return s;
}
// a row is also a valid span (Topic 32):
std::span<int> row(&m[i * C], C);           // no copy, bounds carried
std::sort(row.begin(), row.end());         // sorts row i in place
```

row access – Time $O(C) \cdot \text{stride } 1 \cdot 16 \text{ ints per 64-byte line}$, so $C=1000$ costs ~63 cache lines

98. Accessing Columns

A column is $C \times 4$ bytes apart, element to element. Same asymptotic cost, wildly different real cost — this is Topic 9's $22\times$ measured on real data.

```
long long colSum(const std::vector<int>& m, int j, int R, int C) {
    long long s = 0;
    for (int i = 0; i < R; ++i) s += m[i * C + j];    // stride = C ints
    return s;
}
```

row scan -> |###|###|###| every byte of each line used
column scan-> |#...|#...|#...| 4 bytes used of each 64-byte line
one line fetched per element; $16\times$ the bandwidth for the same data

FIX when columns are the hot path: store the transpose (Topic 103), or process the matrix in tiles that fit L1 (Topic 90 of the cache section, 116).

THE KEY INSIGHT

Row and column access have identical Big-O and a $10\text{--}20\times$ difference in wall clock. This is the cleanest example in the book of why complexity analysis alone is not performance analysis — see Topics 113 and 116.

column access – Time $O(R) \cdot \text{stride } C \times 4 \text{ bytes} \cdot \text{up to } 16\times \text{ the memory traffic of a row scan}$; measured $22\times$ slower at 4096×4096

99. Diagonal Values

```
// square matrix n x n, flat
long long primary = 0, secondary = 0;
for (int i = 0; i < n; ++i) {
    primary += m[i * n + i];           // (0,0) (1,1) (2,2) ...
    secondary += m[i * n + (n - 1 - i)]; // (0,n-1) (1,n-2) ...
}
// The primary diagonal is where i == j. The secondary is where i + j == n - 1.
// Both cross at the centre when n is odd: index (n/2, n/2) - counted twice
// if you are summing "all diagonal elements".
```

$n = 4$ $\begin{bmatrix} P & . & . & S \\ . & P & S & . \\ . & S & P & . \\ S & . & . & P \end{bmatrix}$	$i == j$ P primary	$i + j == n - 1 == 3$ S secondary stride of primary = $n + 1$ elements stride of secondary = $n - 1$ elements
---	-----------------------	--

THE TRAP

For odd n the two diagonals share the centre element. "Sum of both diagonals" for $n=3$ with all ones is **5, not 6**. Symptom: your answer is exactly one centre-element too large on odd inputs and correct on even ones — a test that only uses 4×4 will pass.

diagonals – Time $O(n)$ · Space $O(1)$ · stride $n+1$ elements, so every element is on a different cache line for $n \geq 16$

100. Matrix Addition

```
// C = A + B, elementwise, shapes must match exactly
for (int k = 0; k < R * C; ++k) c[k] = a[k] + b[k];
// Note: ONE flat loop, not two nested ones. Elementwise operations do not
// care about the 2-D shape at all - which is the clearest proof that the
// second dimension is fiction (Topic 90). This loop also auto-vectorises
// (Topic 122); the nested version usually does too, but not always.
```

addition – Time $O(R \cdot C)$ · Space $O(1)$ extra if written in place · memory-bound: 3 streams (read a, read b, write c) = 12 bytes moved per int produced

101. Matrix Subtraction

Identical to Topic 100 with one character changed. Worth writing once to see that the shape check, not the arithmetic, is the real content.

```
if (ra != rb || ca != cb) throw std::invalid_argument("shape mismatch");
for (int k = 0; k < ra * ca; ++k) c[k] = a[k] - b[k];
```

THE TRAP

With flat storage, nothing stops you subtracting a 2×6 from a 3×4 — both are 12 ints and the loop runs happily. Symptom: no error, plausible numbers, wrong model. The shape lives outside the buffer, so *you* must check it. This is the argument for wrapping flat storage in a small **Matrix** class that carries R and C .

subtraction – identical to addition: $O(R \cdot C)$ time, memory-bound

102. Matrix Multiplication

$C = A \cdot B$ for $A (R \times K)$ and $B (K \times C)$: every output element is a dot product of a row of A and a column of B . Note the word *column* — that is where the performance goes.

```
// naive i-j-k: for each output, walk a row of A and a COLUMN of B
for (int i = 0; i < R; ++i)
  for (int j = 0; j < C; ++j) {
    double s = 0;
    for (int k = 0; k < K; ++k) s += a[i*K + k] * b[k*C + j];    // b: stride C
    c[i*C + j] = s;
  }

// i-k-j: hoist a[i][k], then stream a ROW of B into a ROW of C
for (int i = 0; i < R; ++i)
  for (int k = 0; k < K; ++k) {
    double aik = a[i*K + k];
    for (int j = 0; j < C; ++j) c[i*C + j] += aik * b[k*C + j]; // both stride 1
  }
```

Measured on this machine, 512×512×512 **double**, same flags (**-O2**), same arithmetic, same result:

i-j-k (row x column)	438.5 ms	
i-k-j (row x row)	92.1 ms	4.8x faster
j-i-k (column outer)	483.8 ms	

i-j-k	i-k-j
A row -> ####	A element -> #
B col -> #	B row -> ####
#	C row -> ####
# stride C	all stride 1
1 useful int per B line	16 useful ints per B line

THE KEY INSIGHT

Loop *order* changes no operation count — $2 \cdot n^3$ floating-point operations either way — and changes runtime by 4.8×. Reorder loops so the innermost index is the last subscript of every array touched. Beyond that, tiling and a real BLAS win another 5–20×; never hand-write matrix multiply for production (Topic 145).

THE TRAP

The i-k-j form *accumulates into* **c**, so **c** must be zeroed first. Symptom: correct on the first call, garbage on the second because the previous result is still there. The i-j-k form assigns and does not care.

multiply — Time $O(R \cdot K \cdot C)$, i.e. $O(n^3)$ · Space $O(1)$ extra · measured 4.8× between loop orders; Strassen is $O(n^{2.81})$ but only wins above $n \approx 1000$ and loses numerical stability

103. Transpose Matrix

```
// square, in place: swap across the primary diagonal, j starts at i+1
for (int i = 0; i < n; ++i)
    for (int j = i + 1; j < n; ++j)
        std::swap(m[i*n + j], m[j*n + i]);    // n(n-1)/2 swaps

// rectangular: cannot be done in place with this trick - shape changes
std::vector<int> t(C * R);
for (int i = 0; i < R; ++i)
    for (int j = 0; j < C; ++j) t[j*R + i] = m[i*C + j];
```

[1 2 3]	j starts at i+1, NOT 0
[4 5 6]	-> if j starts at 0 every pair is swapped twice
[7 8 9]	and you get the original matrix back.
[1 4 7]	transpose turns column access into row access:
[2 5 8]	the standard fix for the 22x penalty of Topic 98
[3 6 9]	when many column scans are coming.

THE TRAP

for (j = 0; j < n; ++j) in the in-place version swaps every pair twice and silently returns the input unchanged. Symptom: the function "works" on symmetric test matrices and on the identity, and does nothing on real data. Test transpose with a matrix where $m[i][j] = i*10 + j$.

transpose – Time $O(n^2)$ · Space $O(1)$ in place for square, $O(R \cdot C)$ for rectangular · cache-hostile at large n (one operand always strided); tiled transpose in 32×32 blocks is $\sim 3\times$ faster above 2048×2048

104. Rotate Matrix

Rotating a square matrix 90° clockwise = **transpose, then reverse each row**. Nothing new is required: it is Topic 103 followed by Topic 44, applied per row.

```
void rotate90(std::vector<int>& m, int n) {
    for (int i = 0; i < n; ++i)                // transpose
        for (int j = i + 1; j < n; ++j)
            std::swap(m[i*n + j], m[j*n + i]);
    for (int i = 0; i < n; ++i)                // reverse each row
        std::reverse(m.begin() + i*n, m.begin() + i*n + n);
}
// 90 anticlockwise = transpose, then reverse each COLUMN
//                      = reverse each row, then transpose
// 180                  = reverse the whole flat buffer (Topic 44) - one pass
```

[1 2 3]	transpose	[1 4 7]	reverse rows	[7 4 1]	
[4 5 6]	->	[2 5 8]	->	[8 5 2]	= 90 clockwise
[7 8 9]		[3 6 9]		[9 6 3]	

180 degrees: `std::reverse(m.begin(), m.end())` on the FLAT buffer. Because reversing the flat array reverses both row order and within-row order at once. This is a one-line rotation and it is easy to miss.

THE KEY INSIGHT

Composition, not invention. Rotation = transpose + reverse; 180° = one flat reverse; rotation by k in 1-D = three reverses (Topic 57). When an interviewer asks for an in-place rotation they are testing whether you can decompose, not whether you memorised a four-way index swap.

rotate 90 – Time $O(n^2)$ · Space $O(1)$ · two passes over the buffer; the four-way cycle swap does it in one pass and is $\sim 1.5\times$ faster but far easier to get wrong

105. Spiral Traversal

Four boundaries that close in. The entire difficulty is the two guards in the middle, which stop a single leftover row or column being emitted twice.

```
std::vector<int> spiral(const std::vector<int>& m, int R, int C) {
    std::vector<int> out;
    int top = 0, bot = R - 1, left = 0, right = C - 1;
    while (top <= bot && left <= right) {
        for (int j = left; j <= right; ++j) out.push_back(m[top*C + j]);
        ++top;
        for (int i = top; i <= bot; ++i) out.push_back(m[i*C + right]);
        --right;
        if (top <= bot) // GUARD: one row left?
            for (int j = right; j >= left; --j) out.push_back(m[bot*C + j]);
        --bot;
        if (left <= right) // GUARD: one column left?
            for (int i = bot; i >= top; --i) out.push_back(m[i*C + left]);
        ++left;
    }
    return out; // out.size() == R*C always
}
```

----->		top row L->R, then top++
^		right col T->B, then right--
		bottom row R->L, then bot--
	v	left col B->T, then left++
<-----		repeat while top<=bot && left<=right

THE TRAP

Omitting the two guards duplicates elements when the remaining region is a single row or single column — which happens for every non-square matrix and for odd n . Symptom: `out.size() > R*C`, with the middle row appearing twice. **Assert `out.size() == R*C`; that single check catches every spiral bug.** Test with 1×5 , 5×1 , 3×4 and 3×3 .

spiral – Time $O(R\cdot C)$, each cell emitted once · Space $O(1)$ beyond the output · four sequential runs per layer, so it is cache-friendly on rows and strided on columns

106. Search in Matrix

Three different problems wear the same name. Choosing wrongly costs a factor of n .

Matrix property	Method	Cost
Unsorted	Scan all cells (row-major)	$O(R \cdot C)$
Fully sorted, rows concatenated ascending	Binary search over the flat index; decode <code>mid</code> as <code>(mid/C, mid%C)</code>	$O(\log RC)$
Rows ascending, columns ascending (not globally sorted)	Staircase from the top-right corner	$O(R + C)$

```
// staircase search: start top-right, each comparison eliminates a whole
// row or a whole column
bool staircase(const std::vector<int>& m, int R, int C, int target) {
    int i = 0, j = C - 1;
    while (i < R && j >= 0) {
        int v = m[i*C + j];
        if (v == target) return true;
        if (v > target) --j;          // whole column j is too big
        else ++i;                    // whole row i is too small
    }
    return false;
}
```

start here -----+	
[1 4 7 11]	v
[2 5 8 12]	
[3 6 9 16]	
[10 13 14 17]	

value > target -> go LEFT
value < target -> go DOWN
top-right (or bottom-left) is the only corner where the two directions disagree - that is why it works.

THE TRAP

Applying flat binary search (row 2) to a merely row-and-column-sorted matrix (row 3) returns false negatives: in the diagram above the flat sequence 1,4,7,11, 2,5,... is not sorted, so binary search may discard the half containing the target. Symptom: found for some targets, not for others, with no pattern. Read the guarantee before choosing the algorithm.

matrix search – unsorted $O(R \cdot C)$ · flat-sorted $O(\log R \cdot C)$ with a div/mod per step · staircase $O(R+C)$, Space $O(1)$, and it is optimal for that guarantee

107. Mirror Matrix

```
// horizontal mirror (left-right flip): reverse each row
for (int i = 0; i < R; ++i)
    std::reverse(m.begin() + i*C, m.begin() + i*C + C);

// vertical mirror (top-bottom flip): swap row i with row R-1-i
for (int i = 0; i < R / 2; ++i)
    std::swap_ranges(m.begin() + i*C, m.begin() + i*C + C,
                     m.begin() + (R - 1 - i)*C);
```

original	horizontal mirror	vertical mirror
[1 2 3]	[3 2 1]	[7 8 9]
[4 5 6]	[6 5 4]	[4 5 6]
[7 8 9]	[9 8 7]	[1 2 3]

transpose + horizontal mirror = rotate 90 cw (Topic 104)
horizontal + vertical mirror = rotate 180 (= one flat reverse)

mirror – Time $O(R \cdot C)$ · Space $O(1)$ · vertical mirror moves whole rows, so it is `memcpy`-speed;
horizontal reverses within a row

108. Boundary Traversal

The outer ring only — the first iteration of the spiral loop (Topic 105), and it inherits the same duplicate-corner problem.

```
void boundary(const std::vector<int>& m, int R, int C) {
    if (R == 1) { for (int j = 0; j < C; ++j) print(m[j]); return; }
    if (C == 1) { for (int i = 0; i < R; ++i) print(m[i*C]); return; }
    for (int j = 0; j < C; ++j)      print(m[0*C + j]);          // top
    for (int i = 1; i < R; ++i)      print(m[i*C + C - 1]);      // right
    for (int j = C - 2; j >= 0; --j) print(m[(R-1)*C + j]);      // bottom
    for (int i = R - 2; i >= 1; --i) print(m[i*C + 0]);          // left
}
// element count must be 2*(R + C) - 4 for R,C >= 2
```

THE TRAP

The four corners belong to two sides each. The loop bounds above (*i* from 1, *j* from *C*−2, *i* down to 1) exist solely to skip them. Symptom: 4 extra elements. Check `count == 2*(R+C) - 4`, and handle single-row and single-column matrices separately — they break the formula.

boundary – Time $O(R + C)$ · Space $O(1)$ · exactly $2(R+C)-4$ elements for $R, C \geq 2$

109. 8 Neighbors

Grid problems — flood fill, Game of Life, path finding, minesweeper — all share one loop shape: for each of 4 or 8 direction offsets, check bounds, then visit.

```
constexpr int dx[8] = {-1,-1,-1, 0, 0, 1, 1, 1};
constexpr int dy[8] = {-1, 0, 1,-1, 1,-1, 0, 1};
// first four entries of a 4-direction version: {-1,0,1,0} / {0,1,0,-1}

for (int d = 0; d < 8; ++d) {
    int ni = i + dx[d], nj = j + dy[d];
    if (ni < 0 || ni >= R || nj < 0 || nj >= C) continue; // ALWAYS first
    visit(m[ni*C + nj]);
}
```

(i-1,j-1)	(i-1,j)	(i-1,j+1)	d: 0	1	2
(i ,j-1)	[i, j]	(i ,j+1)	3	.	4
(i+1,j-1)	(i+1,j)	(i+1,j+1)	5	6	7

The direction-offset array IS the pattern. Two constant arrays replace eight hand-written if-statements, and the bounds check is written once.

THE TRAP

Checking bounds after computing the index — `if (m[ni*C+nj] == 1 && ni >= 0)` — reads out of bounds *before* the guard runs, and for `ni = -1` the flat index `-1*C + nj` lands in the previous row, so the read succeeds and returns a plausible neighbour. Symptom: wrong answers only at the grid edges, never a crash. Bounds first, always, and rely on `&&` short-circuiting.

neighbours — Time $O(8)$ per cell = $O(1)$ · Space $O(1)$ · a full grid sweep is $O(8 \cdot R \cdot C)$; the diagonal neighbours are on different cache lines, so 8-neighbour sweeps cost roughly $2\times$ the 4-neighbour version

110. Flood Fill

Replace a connected region of one value with another. It is depth- or breadth-first search where the graph is implicit: the nodes are cells and the edges are Topic 109's offsets.

```
#include <queue>
void floodFill(std::vector<int>& g, int R, int C,
              int si, int sj, int newVal) {
    int old = g[si*C + sj];
    if (old == newVal) return;           // GUARD: else infinite loop
    constexpr int dx[4] = {-1, 1, 0, 0}, dy[4] = {0, 0, -1, 1};
    std::queue<std::pair<int,int>> q;
    q.push({si, sj});
    g[si*C + sj] = newVal;               // mark ON PUSH, not on pop
    while (!q.empty()) {
        auto [i, j] = q.front(); q.pop();
        for (int d = 0; d < 4; ++d) {
            int ni = i + dx[d], nj = j + dy[d];
            if (ni < 0 || ni >= R || nj < 0 || nj >= C) continue;
            if (g[ni*C + nj] != old) continue;
            g[ni*C + nj] = newVal;       // mark immediately
            q.push({ni, nj});
        }
    }
}
```

start (1,1), old=1, new=2, 4-connectivity

1 1 0	2 2 0	the 1 at (2,2) is NOT filled: it touches
1 1 0	-> 2 2 0	the region only diagonally, and 4-connectivity
0 0 1	0 0 1	does not consider diagonals.

THE TRAP

Two failure modes, both classic. (1) Missing the `old == newVal` guard: the algorithm re-enqueues cells forever and hangs, or exhausts memory. (2) Marking cells when they are *popped* instead of when they are *pushed*: the same cell enters the queue from several neighbours, and the queue grows to $O(R \cdot C)$ duplicates — an $8\times$ memory blow up on a large grid, with correct output, so it only shows up as an out-of-memory on the biggest test.

flood fill – Time $O(R \cdot C)$, each cell enqueued once · Space $O(R \cdot C)$ worst case for the queue (a full grid); recursive DFS uses the call stack instead and overflows at roughly 10^5 depth (Topic 7)

111. Prefix Sum Matrix

Topic 60 in two dimensions. Precompute once in $O(R \cdot C)$, then answer any rectangle-sum query in $O(1)$ with four lookups and inclusion-exclusion.

```
// P is (R+1) x (C+1), zero-padded so no boundary special-casing is needed
std::vector<long long> P((R + 1) * (C + 1), 0);
for (int i = 1; i <= R; ++i)
    for (int j = 1; j <= C; ++j)
        P[i*(C+1) + j] = m[(i-1)*C + (j-1)]
            + P[(i-1)*(C+1) + j]           // above
            + P[i*(C+1) + (j-1)]           // left
            - P[(i-1)*(C+1) + (j-1)];      // added twice, remove once

// sum of the rectangle (r1,c1)..(r2,c2) inclusive, 0-based:
long long sum = P[(r2+1)*(C+1) + c2+1] - P[r1*(C+1) + c2+1]
    - P[(r2+1)*(C+1) + c1] + P[r1*(C+1) + c1];
```

+	-----	+	-----	+		query = D - B - C + A
	A		B			A is subtracted twice by B and C,
+	-----	+	-----	+		so it is added back once.
	C		TARGET			
+	-----	+	-----	+		D = P at the bottom-right corner
	D = whole rectangle from origin					

THE KEY INSIGHT

Pad by one row and one column of zeros. Every off-by-one bug in 2-D prefix sums comes from special-casing `i == 0`; the padding makes the uniform formula correct at the borders for free. Same trick as the sentinel in Topic 60.

THE TRAP

Overflow. A 1000×1000 matrix of values up to 10^6 sums to 10^{12} , which does not fit in `int`. Symptom: negative answers for large rectangles, correct answers for small ones. Use `long long` for the prefix table even when the input is `int` — Topic 130.

2-D prefix sum – Build $O(R \cdot C)$ time and $O(R \cdot C)$ extra space (8 bytes per cell) · Query $O(1)$, exactly 4 loads · breaks even against naive scanning after about 1 query per 4 cells

112. Sparse Matrix

When almost all entries are zero, dense storage wastes memory proportional to $R \cdot C$ while the information is proportional to the number of non-zeros (*nnz*). A $10^5 \times 10^5$ dense `double` matrix is 80 GB; with 0.01% density it is 10^8 non-zeros $\approx$ 1.6 GB in CSR. **This is the topic where the array is the wrong choice** — and knowing when to abandon it is the point.

```

// Coordinate list (COO): simplest, good for building
struct Entry { int r, c; double v; };
std::vector<Entry> coo; // 16 bytes per non-zero

// Compressed Sparse Row (CSR): the production format
std::vector<double> val; // nnz values, row by row
std::vector<int> col; // nnz column indices
std::vector<int> rowPtr; // R+1 entries; row i is [rowPtr[i], rowPtr[i+1])

// y = A*x in CSR - the workhorse of every iterative solver
for (int i = 0; i < R; ++i) {
    double s = 0;
    for (int k = rowPtr[i]; k < rowPtr[i + 1]; ++k) s += val[k] * x[col[k]];
    y[i] = s;
}

```

dense 4x4, 3 non-zeros	CSR
[0 0 5 0]	val = [5, 8, 3]
[0 8 0 0]	col = [2, 1, 0]
[3 0 0 0]	rowPtr = [0, 1, 2, 3, 3]
[0 0 0 0]	^row0 ^r1 ^r2 ^r3=empty

dense: 16 doubles = 128 bytes CSR: 3*8 + 3*4 + 5*4 = 56 bytes
 crossover is roughly $\text{nnz}/(R \cdot C) < 1/3$ for doubles.

THE TRAP

CSR trades $O(1)$ random access for $O(\log \text{nnz_row})$ or a linear scan of the row — `A[i][j]` is no longer a formula, it is a search. Symptom: a "harmless" refactor that reads single elements in a loop turns an $O(\text{nnz})$ algorithm into $O(\text{nnz} \cdot \log n)$ and the solver gets 50× slower. Use CSR for whole-row operations only; if you need random single-element access, you wanted a hash map keyed by (i,j) instead.

sparse — Space $O(\text{nnz})$ vs $O(R \cdot C)$ · SpMV Time $O(\text{nnz})$ · random access degrades from $O(1)$ to $O(\text{row length})$ · `x[col[k]]` is an indirect, unpredictable load, so SpMV runs at ~10–20% of dense peak FLOPS

PRACTICE SET — Topics 90-112

A. Conceptual

1. Why is the column count part of a C++ array's *type* while the row count is not? Connect your answer to the address formula of Topic 8.
2. Give two concrete costs of `vector<vector<int>>` over a flat `vector<int>(R*C)` for a rectangular grid, and one case where the jagged form is genuinely right.
3. Row scan and column scan are both $O(R \cdot C)$ over the same bytes. Explain the measured $22\times$ gap without using the word "slow".
4. Why does 180° rotation need only `std::reverse` on the flat buffer, while 90° needs two passes?
5. i - j - k and i - k - j matrix multiply execute the same $2n^3$ arithmetic operations. Name the quantity that differs, and give its ratio.
6. In the in-place square transpose, why does `j` start at `i + 1`? What exactly does the code compute if it starts at 0?
7. Why must a flood fill mark a cell when it is *pushed* rather than when it is *popped*?
8. Why is the 2-D prefix table sized $(R+1) \times (C+1)$ rather than $R \times C$?
9. State the inclusion-exclusion formula for a rectangle sum and say in one sentence why the fourth term is added rather than subtracted.
10. Give the density threshold at which CSR stops saving memory over dense `double` storage, and show the arithmetic.
11. Name the one operation that CSR makes asymptotically worse than a dense array, and the symptom you would see in production.
12. Why does the staircase search start at the top-right corner and not the top-left?

B. Tracing and arithmetic

1. `int m[5][8]` based at `0x2000`. Give the address of `m[3][6]`, and the values of `sizeof(m)`, `sizeof(m[0])` and `sizeof(m[0][0])`.
2. Write out the spiral order of the 3×4 matrix whose entry at (i,j) is $i*4 + j + 1$. How many elements should the output contain?
3. For that same 3×4 matrix, list the boundary traversal and check it against $2(R+C) - 4$.
4. Staircase-search the matrix `[[1,4,7,11],[2,5,8,12],[3,6,9,16],[10,13,14,17]]` for 14. List every cell visited, in order, and count the comparisons. Then do the same for 15.
5. Build the 2-D prefix table for `[[1,2],[3,4]]` by hand (3×3 , zero-padded) and use it to compute the sum of the rectangle $(0,1) \dots (1,1)$.
6. A 1000×1000 `int` matrix holds values up to 10^6 . What is the largest possible rectangle sum, and how many bits does it need? What does an `int` prefix table print for the whole-matrix query?
7. Give the exact memory in bytes for a 1000×1000 `int` matrix stored (a) flat, (b) as `vector<vector<int>>`. Assume 24 bytes of vector object per row and a 16-byte allocator header per allocation.
8. A 4×4 matrix has 3 non-zeros. Give its COO and CSR representations and the byte count of each, using 8-byte values and 4-byte indices.
9. Sum of both diagonals of the 3×3 all-ones matrix: give the value your code prints if you do not handle the centre, and the correct value.

C. Code tasks — build it, break it, observe, fix it

1. **Loop order, measured.** Implement all three multiply orders from Topic 102 for 512×512 `double`. Report your three timings and the ratio. Then run `perf stat -e instructions,cache-misses` on the fastest and slowest and confirm that instruction counts are within a few percent while cache misses are not.
2. **Break it: spiral without guards.** Delete the two `if (top <= bot) / if (left <= right)` guards from Topic 105. Run it on 3×4 , 1×5 , 5×1 and 3×3 , printing `out.size()` against $R \times C$ each time. Record which shapes break and by how many elements, then restore the guards and add `assert(out.size() == (size_t)R*C)`.
3. **Break it: transpose from $j = 0$.** Change `j = i + 1` to `j = 0`. Test with the matrix `m[i][j] = i*10 + j` and then with a symmetric matrix. Explain in one sentence why a test suite built only from symmetric matrices would have shipped this bug.
4. **Break it: bounds check after the read.** In the 8-neighbour loop, move the bounds test to *after* the array read. Run over a 4×4 grid and print the value seen for the neighbour above row 0. Rebuild with `-fsanitize=address` and record whether it fires — then explain why it usually does *not* for a flat vector, and does for a raw stack array.
5. **Break it: int prefix table.** Build the 2-D prefix sum for a 1000×1000 matrix filled with 10^6 using `int`. Query the whole matrix, record the (negative) answer, then rebuild with `-fsanitize=undefined` and paste the runtime diagnostic. Fix with `long long`.
6. **Break it: flood fill that hangs.** Remove the `old == newVal` guard and call the function with `newVal` equal to the starting cell's value on a 500×500 grid. Observe what happens to memory (watch `ps` or the queue size). Fix it, then change marking-on-push to marking-on-pop and report the peak queue size for both versions on the same grid.
7. **Layout benchmark.** Fill a 2000×2000 grid both as a flat `vector<int>` and as `vector<vector<int>>`, sum both, and report the ratio and the number of allocations each performs (count them with a global `operator new` counter).
8. **Sparse crossover.** Implement CSR SpMV. For a 2000×2000 matrix, measure dense multiply-by-vector versus CSR SpMV at densities 50%, 10%, 1% and 0.1%, and find the density at which CSR first wins.

ANSWER KEY — Topics 90-112

A. Conceptual

1. Because the address formula needs the **row stride**, which is `COLS * sizeof(T)`. Rows can be counted at runtime; the multiplier cannot be discovered from the pointer. Hence `int m[][4]` is a valid parameter and `int m[3][]` is not.
2. **R+1 allocations instead of 1**, and **rows scattered so cross-row prefetch is impossible** (plus 24 bytes of vector object and an allocator header per row). Right when rows genuinely differ in length — adjacency lists, ragged records.
3. Memory is delivered in **64-byte lines**. The row scan consumes all 16 ints of each line it pays for; the column scan consumes 4 bytes of each line and the line is evicted before the loop returns to it, so it pays up to 16× the memory traffic and defeats the stride prefetcher.
4. Because reversing the flat buffer reverses **row order and within-row order simultaneously**, which is exactly 180°. A 90° rotation changes the *shape* of the index mapping (i,j)→(j, n-1-i), which no single linear reversal expresses.
5. **Memory traffic / cache misses**. Measured here: 438.5 ms vs 92.1 ms = **4.8×**, with identical FLOP counts.
6. Because `swap(a,b)` is its own inverse. Starting at 0 swaps every off-diagonal pair **twice**, so the matrix is returned **unchanged**.
7. Because a cell can be reached from up to four neighbours before it is popped. Marking on pop lets it be enqueued four times: correct output, but the queue grows by a constant factor (up to ~4×) and memory blows up on large grids.
8. So that the recurrence `P[i][j] = m + P[i-1][j] + P[i][j-1] - P[i-1][j-1]` is valid at `i = 0` and `j = 0` without special cases: the padded row and column supply the zeros.
9. `D - B - C + A`. The top-left block **A** is contained in both **B** and **C**, so subtracting both removes it twice; it must be added back once.
10. Dense costs 8 bytes per entry. CSR costs 8 (value) + 4 (column) = 12 bytes per non-zero, plus 4(R+1). So CSR wins when `12 * nnz < 8 * R * C`, i.e. density **below 2/3** in the strictest reading — but the practical threshold, once you account for the loss of vectorisation and the indirect `x[col[k]]` load, is around **10%**.
11. **Random single-element access**: O(1) becomes a scan or binary search of the row. Symptom: an innocuous refactor to element-wise access makes the solver 10-50× slower with no change in output.
12. Because top-right is the only corner where the two comparison outcomes point in **different, non-overlapping directions** (bigger → go left, smaller → go down). At top-left both "bigger" moves would be off-grid, so no elimination is possible. Bottom-left works equally well.

B. Tracing and arithmetic

1. Offset = $(3 \times 8 + 6) \times 4 = 30 \times 4 = 120 = 0x78 \rightarrow$ **0x2078**. `sizeof(m) = 160`, `sizeof(m[0]) = 32`, `sizeof(m[0][0]) = 4`.
2. **1 2 3 4 8 12 11 10 9 5 6 7** — exactly **12** elements.
3. Boundary: **1 2 3 4 8 12 11 10 9 5 = 10** elements; $2(3+4) - 4 =$ **10**. ✓
4. Searching 14: (0,3)=11<14 → down; (1,3)=12<14 → down; (2,3)=16>14 → left; (2,2)=9<14 → down; (3,2)=**14 found. 5 comparisons**. Searching 15: same first four steps, then (3,2)=14<15 → down → `i = 4 = R`, loop ends, **false** after **6** comparisons. Worst case is $R + C - 1 = 7$.

5. $P = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 3 \\ 0 & 4 & 10 \end{bmatrix}$. Rectangle (0,1)..(1,1) = $P[2][2] - P[0][2] - P[2][1] + P[0][1] = 10 - 3 - 4 + 0 = 3$ (= 2 + 4 ✓).
6. Max sum = $10^6 \times 10^6 = 10^{12}$, needing **40 bits**. An `int` table overflows; $10^{12} \bmod 2^{32} = 3,567,587,328$ which as a signed 32-bit value prints **-727,379,968**. Signed overflow is UB, so the exact value is not guaranteed — but it will be wrong and usually negative.
7. (a) $1000 \times 1000 \times 4 = 4,000,000$ **bytes** in one allocation. (b) 4,000,000 for the data + $1000 \times (24 + 16)$ for row objects and headers + 1000×16 header on each row block $\approx 4,056,000$ **bytes**, spread over **1001 allocations**.
8. COO: 3 entries $\times (4 + 4 + 8) = 48$ **bytes**. CSR: val $3 \times 8 = 24$, col $3 \times 4 = 12$, rowPtr $5 \times 4 = 20 \rightarrow 56$ **bytes**. Dense: $16 \times 8 = 128$ **bytes**. (COO beats CSR at this size; CSR wins as soon as rows are long, because it stores one pointer per row rather than one row index per entry.)
9. Naive prints **6**; correct is **5**, because the centre (1,1) lies on both diagonals and must be counted once.

C. What to verify

1. Reference timings on the machine this book was built on: **i-j-k 438.5 ms, i-k-j 92.1 ms, j-i-k 483.8 ms** — ratio **4.8×**. `perf` should show instruction counts within ~2% and cache misses differing by an order of magnitude. **That contrast is the entire lesson.**
2. 3×4 without guards emits **14 elements instead of 12** (the bottom row is re-emitted); 1×5 emits **10 instead of 5**; 5×1 emits **10 instead of 5**; 3×3 emits **13 instead of 9**. Square even-sized matrices such as 4×4 pass — which is why the bug survives most test suites.
3. With `j = 0` the matrix is returned **identical to the input**. A symmetric matrix is its own transpose, so both the correct and the broken version produce the same output and the test passes.
4. The neighbour above row 0 has flat index `-1*C + j`, which is `j - C` — a valid index into the vector's own buffer whenever `j < C` is offset far enough, or a read just before the buffer. For a `std::vector` the read commonly lands **inside the heap block or its header, so ASan stays silent**; for a raw stack array ASan reports `stack-buffer-underflow`. **The lesson: the sanitizer does not save you from a logic error that happens to stay in bounds.**
5. The whole-matrix query returns a **negative** number (see B6). UBSan reports `signed integer overflow: ... cannot be represented in type 'int'` at the accumulation line. With `long long` the answer is **1,000,000,000,000**.
6. Without the guard the queue never empties: memory climbs until the process is killed (or, with recursion, an immediate stack overflow). Peak queue size on a uniform 500×500 grid: **mark-on-push $\approx O(\sqrt{\text{area}}) \approx$ a few hundred entries for a frontier; mark-on-pop grows to several thousand**, roughly 3-4× larger, because each cell is enqueued once per adjacent neighbour.
7. Expect the flat version to be **1.5-4× faster** on the sum, with **1 allocation vs 2001**. If your machine shows less than 1.5×, your rows happened to be allocated consecutively — shuffle the allocation order (fill the rows in random order) and re-measure.
8. CSR SpMV should lose badly at 50% density (indirection plus no vectorisation), be roughly break-even somewhere around **5-15%**, and win by 10× or more at 0.1%. The exact cross-over is machine-dependent; report yours with the numbers, not an adjective.

HANDNOTE SUMMARY CARD — 2D Arrays & Matrices

2D ARRAYS & MATRICES (Topics 90-112)

THE ONE FACT: there is no 2D memory. $m[i][j] == \text{flat}[i * \text{COLS} + j]$
 $\text{sizeof}(m) = R * C * T$ $\text{sizeof}(m[0]) = C * T$ $m[1]$ decays to int^* , not int^{**}
 $\text{int } m[][4]$ is a legal parameter; $\text{int } m[3][]$ is not - the STRIDE is the type.

STORAGE CHOICE

<code>flat vector<int>(R*C)</code>	1 alloc, contiguous	DEFAULT
<code>vector<vector<int>></code>	R+1 allocs, 2 loads/access	ragged rows only
<code>array<array<int,C>,R></code>	0 allocs, stack, compile-time size	
<code>int g[1000][1000]</code>	4 MB in .bss, zeroed, global scope only	
<code>int m[R][C]</code> with runtime R	-> VLA: GCC extension, blows the 8 MB stack	

TRAVERSAL COST (the 22x lesson, Topic 9/98)

row scan	stride 1	16 ints per 64B line	7.8 ms	@4096 ²
column scan	stride C*4	4 bytes per 64B line	172.1 ms	@4096 ²
matmul i-j-k	438.5 ms	i-k-j	92.1 ms	j-i-k 483.8 ms @512 ³ double

SAME instruction count. Innermost loop must vary the LAST subscript.

COMPOSITIONS - nothing here is new

transpose	swap $m[i][j], m[j][i]$ for $j = i+1 \dots n-1$ ($j=0$ = no-op)
rotate 90 cw	transpose + reverse each row
rotate 90 ccw	transpose + reverse each column
rotate 180	std::reverse over the WHOLE FLAT buffer (one line)
h-mirror	reverse each row v-mirror swap_ranges rows i, R-1-i
diagonals	$i=i$ (primary), $i+(n-1-i)$ (secondary), share centre if odd

INDEX FORMULAS

primary diag stride	n+1	secondary diag stride	n-1
boundary count	$2 * (R+C) - 4$ ($R, C \geq 2$)	spiral count	$R * C$ exactly
8 neighbours	$dx[8] = \{-1, -1, -1, 0, 0, 1, 1, 1\}$	$dy[8] = \{-1, 0, 1, -1, 1, -1, 0, 1\}$	
2D prefix, padded	$P[i][j] = m + P[i-1][j] + P[i][j-1] - P[i-1][j-1]$		
rectangle sum	$D - B - C + A$	4 loads, $O(1)$	

SEARCH: read the guarantee first

unsorted	full scan	$O(R * C)$
rows concatenated sorted	flat binary search	$O(\log RC)$, mid -> /C, %C
rows AND columns sorted	staircase top-right	$O(R+C)$, Space $O(1)$

using flat binary search on the staircase case = silent false negatives

TRAPS, ONE LINE EACH

<code>int m[2][3]; no = {}</code>	garbage; sums differ every run
transpose with $j = 0$	returns the input; symmetric tests pass
spiral without the two guards	3x4 emits 14 of 12; 4x4 passes
bounds test after the read	edge reads succeed, wrong answers only at the border, ASan often silent on vectors
<code>int 2D prefix table</code>	1000x1000 of $1e6$ -> $1e12$ -> negative
flood fill, no old==new guard	infinite loop / OOM
flood fill, mark on pop	3-4x queue growth, output still correct
i-k-j without zeroing C	correct once, garbage on the second call
<code>vector<vector></code> for a rectangle	1001 allocs, 1.5-4x slower scan
CSR + random element access	$O(1)$ becomes $O(\text{row})$; solver 10-50x slower

WHEN THE ARRAY IS WRONG (Topic 112)

density < ~10% non-zero -> CSR: $\text{val}[], \text{col}[], \text{rowPtr}[R+1]$; SpMV is $O(\text{nnz})$
 $1e5 \times 1e5$ dense double = 80 GB. Same matrix at 0.01% = ~1.6 GB in CSR.

PART IV CHEATSHEET — Grids in One Page

PART IV: THE GRID PLAYBOOK

DECIDE STORAGE FIRST

rectangular + runtime size	flat <code>vector<T>(R*C)</code> , <code>at(i,j)=i*C+j</code>
rectangular + compile-time size	<code>std::array</code> or global raw array
ragged rows	<code>vector<vector<T>></code>
mostly zeros (< ~10%)	CSR
huge and disk-backed	memory-mapped, row-major (Topic 162)

THE FOUR TRAVERSAL SHAPES, AND WHAT THEY SOLVE

1. full row-major sweep sum, print, elementwise add/sub, image filters
 `for(i) for(j) f(a[i*C+j])` `0(R*C)`, stride 1
2. layer / boundary walk spiral, boundary, ring rotation
 top,bot,left,right closing in; guard singleton row and column
3. direction offsets flood fill, Game of Life, BFS/DFS on a grid,
 `dx[]/dy[]` + bounds FIRST minesweeper, island counting, shortest path
4. corner-elimination walk staircase search on a sorted grid `0(R+C)`

QUERY vs UPDATE: pick the precomputation

many range-sum queries, static grid	2D prefix sums build <code>0(RC)</code> , query <code>0(1)</code>
many range UPDATES, few queries	2D difference (Topic 62 in 2D)
both heavy	2D Fenwick / segment tree - out of scope here, and the signal you outgrew arrays

CHECKS THAT CATCH ALMOST EVERY BUG IN THIS PART

<code>assert(out.size() == (size_t)R*C)</code>	spiral, any full traversal
<code>assert(count == 2*(R+C) - 4)</code>	boundary
test with <code>m[i][j] = i*10 + j</code>	transpose, rotate, mirror
test <code>1xN</code> , <code>Nx1</code> , <code>1x1</code> , <code>3x4</code> (non-square, odd)	every layer algorithm
long long for anything summed	prefix tables, dot products
zero C before an accumulating multiply	i-k-j

CROSS-REFERENCES

Topic 8/9	address formula and row-major - everything here is that formula
Topic 44	reverse: the primitive under rotate 90/180 and mirror
Topic 60	1D prefix sums; Topic 111 is the same trick with 4 corners
Topic 116	cache lines: why loop order beats micro-optimisation
Topic 152	adjacency lists: the honest use of <code>vector<vector></code>
Topic 164	GPU arrays: flat storage plus a stride is the only representation

PART V

Cost, Cache and Correctness

Big-O is where performance analysis starts, not where it ends: two $O(n)$ loops over the same bytes differ by $22\times$. This part covers the constant factors the notation hides, and then the six ways array code breaks — with the symptom each one actually presents. Checklist items 113–136.

SECTION 13 — COMPLEXITY & PERFORMANCE

113. Time Complexity

Big-O counts how the operation count grows with n , deliberately discarding the constant. For arrays the whole table follows from one distinction: *does this operation move elements?*

Operation	Time	Why
Access / update <code>a[i]</code>	$O(1)$	Address formula, Topic 8
Append within capacity	$O(1)$	One store, nothing moves
Append at capacity	$O(n)$ once, $O(1)$ amortized	Reallocate + copy, Topic 115
Insert / delete at position p	$O(n - p)$	Shift the tail, Topics 39–40
Delete unordered (swap with last)	$O(1)$	Order sacrificed
Linear search	$O(n)$	$n/2$ on a hit, n on a miss
Binary search (sorted)	$O(\log n)$	20 steps at 10^6 , 30 at 10^9
Comparison sort	$O(n \log n)$	Topic 74
Counting / radix sort	$O(n + K)$, $O(d \cdot n)$	Non-comparison, Topics 81–82
Full scan of a matrix	$O(R \cdot C)$	But $22\times$ apart by loop order

THE TRAP

Big-O compares *algorithms*, not *implementations*. Row-major and column-major traversal are both $\Theta(R \cdot C)$ and differ by $22\times$ (Topic 9); `std::list` and `std::vector` both do $O(n)$ traversal and differ by $240\times$ on 10^7 ints (measured in Topic 148). Symptom of trusting the notation alone: an "optimised" rewrite that is asymptotically better and measurably slower. Always state the n at which the asymptotics take over.

complexity – the array's bargain: $O(1)$ access and scanning speed, paid for with $O(n)$ structural change

114. Space Complexity

Two numbers matter and they are routinely conflated: total memory the data occupies, and *auxiliary* memory the algorithm needs beyond the input. "In place" means auxiliary space is $O(1)$ or $O(\log n)$, not that no memory is used.

```
// O(1) auxiliary - in place
void reverse(int* a, int n) {
    for (int i = 0, j = n - 1; i < j; ++i, --j) std::swap(a[i], a[j]);
}
// O(n) auxiliary - not in place, even though it looks small
std::vector<int> reverseCopy(const std::vector<int>& a) {
    return std::vector<int>(a.rbegin(), a.rend()); // allocates n ints
}
```

Algorithm	Auxiliary space
Reverse, rotate-by-reversal, DNF partition	$O(1)$
Quicksort (recursion, tail-optimised)	$O(\log n)$
Merge sort	$O(n)$ — the reason it is not always preferred
Counting sort	$O(K)$, K = value range
Prefix sums, difference array	$O(n)$
2-D prefix sums	$O(R \cdot C)$, 8 bytes per cell

THE KEY INSIGHT

The recursion stack counts. A "no extra array" recursive algorithm with depth n uses $O(n)$ space and, at roughly 10^5 frames on an 8 MB stack, it does not merely get slow — it segfaults (Topic 7).

space — an `int` array of n costs exactly $4n$ bytes; `vector` adds 24 bytes of object plus up to 100% slack from doubling (Topic 115)

115. Amortized Complexity

`push_back` is $O(n)$ whenever it reallocates and $O(1)$ otherwise, yet n appends cost $O(n)$ in total. Amortized analysis is the accounting that proves it: with a growth factor of 2, appending n elements copies $1 + 2 + 4 + \dots + n/2 < n$ elements in total, so the copying adds at most one extra move per element.

```
// libstdc++ growth, measured: capacity doubles from 1
// 1 2 4 8 16 32 64 128 256 512 1024 2048 ... 1048576
// push_back of 1,000,000 ints performed exactly 21 reallocations.

// Measured cost of not reserving, 10,000,000 ints, same binary:
//   push_back with no reserve   215.8 ms
//   push_back after reserve      99.1 ms      2.2x
// The 2.2x is the copying plus the allocator traffic, not the appends.
```

```
capacity: 1   2   4   8   16
copies:   0 + 1 + 2 + 4 + 8 = 15 when growing to 16
          total copies < n -> amortized O(1) per push_back

growth factor 2 : <= 1 extra copy per element, up to 100% memory slack
growth factor 1.5: more copies, less slack, and blocks can be reused
growth factor 1  : O(n) copies per append -> O(n^2) total. Never do this.
```

THE TRAP

Amortized $O(1)$ is an average over the sequence, not a bound on any single call. One `push_back` in a 10-million-element vector copies 40 MB and takes tens of milliseconds. Symptom: a real-time or latency-sensitive service with a clean median and a p99.9 spike every few seconds. Fix: `reserve()` upfront, or use a deque-like structure that never reallocates.

THE SECOND TRAP

Reallocation **invalidates every pointer, reference and iterator** into the vector. `int& r = v[0]; v.push_back(x); r = 5;` writes to freed memory. Symptom: use-after-free that appears only when the append happens to hit the capacity boundary — so it reproduces once in every few hundred runs. ASan catches it immediately.

amortized – Time $O(1)$ amortized, $O(n)$ worst case per call · Space up to $2\times$ the data during the copy (old and new block both live) · 21 reallocations to reach 10^6 ; reserve removes all of them

116. Performance & Cache Behavior

The gap between a cache hit and a DRAM access is roughly 4 cycles versus 200 — a factor of 50. That is why an $O(n)$ array scan beats an $O(n)$ pointer chase by $240\times$, and why loop order beats almost every micro-optimisation you could make inside the loop.

Level	Typical size	Latency	Ints delivered per access
Register	~1 KB	0 cycles	1
L1 data	32-48 KB	~4 cycles	16 (one 64 B line)
L2	256 KB-2 MB	~12-20 cycles	16
L3	8-32 MB	~40-60 cycles	16
DRAM	GB	~200 cycles	16

sequential array scan [#### #### #### ####] one miss per 16 ints prefetcher predicts the stride and fetches ahead	random pointer chase [#..] ---> [#..] ---> [#..] one miss per element, and the address is not known until the previous load completes -> no prefetch, no overlap
measured, 10^7 ints: array 5.6 ms scattered linked list 1346.6 ms = $240\times$	

THE KEY INSIGHT

Modern CPUs are not slow at arithmetic; they are slow at waiting. An array's real advantage over a node-based structure is not the $O(1)$ index — it is that the next thing you need is already on the line you just paid for.

cache – the constant Big-O hides: a factor of 50 per access and up to $240\times$ on a full traversal · your working set fits L2 below ~64 K ints

117. CPU Cache Lines

Memory moves between DRAM and cache in fixed 64-byte blocks. You never load one `int`; you load the 64-byte line containing it — and 15 neighbours come free.

```
// Confirm on Linux:
//   getconf LEVEL1_DCACHE_LINESIZE      -> 64   (measured on this machine)
// Elements per 64-byte line:
//   char 64 | short 32 | int/float 16 | double/pointer 8 | 32-byte struct 2
```

```
64-byte line, ints:
| 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 |
one DRAM round trip (~200 cycles) delivers ALL of these.

reading a[0]..a[15] -> 1 miss + 15 hits
reading a[0],a[16],a[32]... (stride 16) -> 1 miss PER ELEMENT, 16x traffic
This is exactly the column-scan penalty of Topic 98.
```

cache line – 64 bytes on x86-64 and Apple silicon (128 B on some POWER) · one line = 16 ints, 8 doubles · alignment matters: a 64-byte struct straddling two lines costs two fetches (Topic 121)

118. Spatial Locality

Spatial locality is the property that *if you touched address x, you will soon touch addresses near x*. Arrays have it by construction; that is the whole performance story. The hardware rewards it twice: the line you fetched holds the next 15 elements, and the stride prefetcher, seeing a constant stride, issues loads before you ask.

```
// GOOD: array of structs, only when you use most fields
struct Particle { float x, y, z, vx, vy, vz; };          // 24 bytes
std::vector<Particle> p(n);
for (auto& q : p) q.x += q.vx;          // loads 24 B, uses 8 -2.7 lines/element

// BETTER for one-field passes: struct of arrays (SoA)
struct Particles { std::vector<float> x, vx, /* ... */; };
for (int i = 0; i < n; ++i) px[i] += pvx[i]; // 100% of every line used,
                                              // and this loop vectorises
```

THE KEY INSIGHT

AoS versus SoA is not style. If a hot loop touches one field of a 64-byte struct, AoS wastes 87% of the bandwidth it pays for. SoA is why game engines, databases (columnar storage) and NumPy are laid out the way they are — Topics 101 and 145.

spatial locality – turns 16 potential misses into 1 · the prefetcher recognises constant strides up to about 2 KB and gives up on random patterns

119. Temporal Locality

Temporal locality is *reusing the same data before it is evicted*. It is the property an array scan does **not** have: each element is touched once and never again, so a 200 MB array streams through the cache and evicts everything else. Blocking (tiling) is how you manufacture temporal locality that the naive loop lacks.

```
// Naive: the whole of B is swept for every row of A. If B does not fit in
// cache, every sweep re-reads it from DRAM.
for (i) for (k) for (j) C[i][j] += A[i][k] * B[k][j];

// Tiled: work on TILE x TILE blocks that fit L1, reusing each block TILE times
constexpr int T = 64; // 64*64*8 B = 32 KB
for (int ii = 0; ii < n; ii += T)
    for (int kk = 0; kk < n; kk += T)
        for (int jj = 0; jj < n; jj += T)
            for (int i = ii; i < std::min(ii+T, n); ++i)
                for (int k = kk; k < std::min(kk+T, n); ++k)
                    for (int j = jj; j < std::min(jj+T, n); ++j)
                        C[i*n+j] += A[i*n+k] * B[k*n+j];
```

temporal locality – tiling turns $O(n^3)$ DRAM traffic into $O(n^3/\sqrt{M})$ · a further 2–4× over the best simple loop order at $n \geq 1024$, and the reason tuned BLAS beats hand-written code by 5–20× (Topic 102)

120. False Sharing

Two threads writing to *different* elements of the same array can serialise as badly as if they shared one variable, because coherence works at cache-line granularity: 16 consecutive ints live on one line, and only one core may own that line for writing at a time.

```
// BAD: 8 threads, counters 4 bytes apart - all on ONE 64-byte line
int counts[8];
// thread t does: for (...) ++counts[t];
// Every increment steals the line from the other 7 cores.
// Typical measurement on a 4-8 core x86: 5-20x slower than the fixed version,
// and it gets WORSE as you add threads.

// FIX 1: pad each counter onto its own line
struct alignas(64) Counter { long long v; }; // 64 bytes each
std::vector<Counter> counts(8);

// FIX 2 (better): accumulate in a local, write once at the end
long long local = 0;
for (...) ++local;
counts[t].v = local; // one write per thread
```

```
one 64-byte line
[c0|c1|c2|c3|c4|c5|c6|c7]   core 0 writes c0 -> line becomes Modified on core 0
                             core 1 writes c1 -> line invalidated on core 0,
                             transferred to core 1 ...
ping-pong: ~100 ns per transfer, for a 1 ns increment.
```

THE TRAP

The symptom is uniquely misleading: **the program is correct, and adding threads makes it slower**. No race, no data corruption, no sanitizer warning — ThreadSanitizer is silent because there is no data race. Diagnose with `perf stat -e cache-misses` or by padding and re-measuring. Suspect false sharing whenever per-thread output slots are adjacent in an array.

false sharing – 0 correctness cost, up to 20× throughput cost · fix costs 64 bytes per thread · the general rule: never let two threads write within 64 bytes of each other

121. Alignment and Padding

The compiler inserts padding so each member sits on an address divisible by its own size. That is why `struct { char c; int i; }` is 8 bytes, not 5 — and why field *order* changes the size of your array by up to 2×.

```
struct Bad { char a; int b; char c; }; // 1 +3pad +4 +1 +3pad = 12 bytes
struct Good { int b; char a; char c; }; // 4 +1 +1 +2pad = 8 bytes
// Measured here: sizeof(struct{char c; int i;}) == 8.
// A 10-million-element array: Bad = 120 MB, Good = 80 MB. Same data.

static_assert(sizeof(Good) == 8); // pin it down in the code
// alignas(64) forces a whole cache line (Topic 120); alignas(32) is what
// AVX loads want (Topic 122).
```

```
struct Bad { char a; int b; char c; };
byte:  0   1 2 3   4 5 6 7   8   9 10 11
      [a] [pad pad pad][b b b b] [c] [pad pad pad]
           ^ b must start at a multiple of 4   ^ trailing pad so that an
                                                ARRAY of Bad stays aligned
```

Rule of thumb: declare members in DECREASING size order.

THE KEY INSIGHT

Trailing padding exists so that `a[i+1]` is aligned too — the struct's size must be a multiple of its strictest alignment. Padding is therefore an array-layout property, not a struct property, and it is the reason Topic 8's uniform stride survives structs at all.

padding – costs 0–50% of your array's memory, silently · reordering fields is free and can save 33% (12 → 8 bytes here) · check with `-Wpadded` or `offsetof`

122. SIMD and Vectorization

One SSE instruction adds four `float`s; AVX2 adds eight. The compiler will do this for you — but only if the loop is a simple, aliasing-free, unit-stride pass over an array. Arrays are the only data structure that vectorises naturally.

```
// Measured on this machine, 8192 floats resident in L1, 20,000 passes:
// -O2 -ffast-math 50.4 ms
// -O2 -ffast-math -fno-tree-vectorize 199.8 ms 4.0x
for (int i = 0; i < n; ++i) x += a[i] * a[i];

// Check what the compiler actually did:
// g++ -O2 -fopt-info-vec prints "loop vectorized" lines
// g++ -O2 -fopt-info-vec-missed tells you WHY it refused
```

scalar	SIMD (128-bit, 4 floats)
a0*a0 -> acc	[a0 a1 a2 a3] * [a0 a1 a2 a3]
a1*a1 -> acc	-> [p0 p1 p2 p3] -> 4 accumulators
a2*a2 -> acc	one instruction per 4 elements;
a3*a3 -> acc	a horizontal add at the very end

THE TRAP

The reduction above needs `-ffast-math`, because summing in a different order changes a floating-point result — and it did: **163,848,160 vectorised versus 163,842,048 scalar**, a relative difference of 4×10^{-5} from the same input. Symptom: a numerical test that passes at `-O0` and fails at `-O2`, or results that differ between machines. Never enable `-ffast-math` globally in code that must be reproducible; vectorise integer loops freely, they are exact.

THE SECOND TRAP

SIMD only helps when you are not already waiting for memory. The same experiment on a 200 MB array gave only **25.2 ms versus 32.2 ms (1.28×)** — the loop was bandwidth-bound, so wider arithmetic bought almost nothing. Establish whether you are memory-bound *before* reaching for intrinsics.

SIMD — up to 4× measured on L1-resident float data, 1.28× on a DRAM-bound scan · requires unit stride, no aliasing (`__restrict` helps), no early exit · pointer-chasing structures cannot vectorise at all

SECTION 14 — BUGS, SAFETY & BEST PRACTICES

123. Bounds Checking

C++ does not bounds-check `operator[]` on anything — raw arrays, `std::array` or `std::vector`. You opt in, per call site with `.at()`, or per build with a hardened standard library.

```
std::vector<int> v(10);
v[20];           // UB. No check. Reads whatever is 80 bytes past the block.
v.at(20);        // throws std::out_of_range - a real, catchable error
// Build-wide options:
// -D_GLIBCXX_ASSERTIONS libstdc++: checks operator[], iterators, .front()
// -D_LIBCPP_HARDENING_MODE=_LIBCPP_HARDENING_MODE_FAST libc++
// -fsanitize=address catches heap/stack overflow at runtime (debug only)
```

Measured cost of `.at()` on a 50-million-element sum on this machine: **25.8 ms versus 25.2 ms — 2%**. The branch is perfectly predicted and the loop is memory-bound, so the check is nearly free.

THE KEY INSIGHT

"Bounds checking is too slow" is folklore that a five-minute measurement usually refutes. Measure your own hot loop; if the check costs 2%, use `.at()` or ship with `-D_GLIBCXX_ASSERTIONS` and buy a class of bug out of existence.

bounds checking — `[]` 0 ns, no safety · `.at()` ~2% measured on a memory-bound loop, throws · `_GLIBCXX_ASSERTIONS` similar, aborts · ASan 2–3× slower and 3× memory, debug builds only

124. Undefined Behavior

Out-of-bounds access is not "reads garbage". It is *undefined behaviour*: the standard places no requirement on the program at all, and the optimiser is entitled to assume it never happens. That assumption is what makes UB dangerous rather than merely wrong.

```
int a[5];
for (int i = 0; i <= 5; ++i) a[i] = 0;
// The compiler may reason: "i <= 5 would be UB at i == 5, therefore i < 5,
// therefore this loop runs exactly 5 times" - or it may not, and you get a
// silent write past the end. Both are conforming. The behaviour can change
// with -O2, with a compiler upgrade, or with an unrelated edit elsewhere.
```

UB in array code	Typical symptom
Index out of range	Corrupted neighbour variable; crash far away
Forming <code>&a[n+1]</code> or <code>a - 1</code>	Works at -O0, breaks at -O2
Reading an uninitialised element	Different answer per run
Signed integer overflow in an index	Loop never terminates
Using a pointer after reallocation	Intermittent use-after-free
<code>delete[]</code> vs <code>delete</code> mismatch	Heap corruption, crash later

THE TRAP

UB has no locality: the corruption and the crash are usually in different files. Chasing the crash site is wasted effort. Reproduce under `-fsanitize=address,undefined` — which reports the *first* offending access with a stack trace — before reading any of the code you suspect.

UB – cost: unbounded, and it varies with optimisation level · detection: ASan + UBSan catch most array UB at ~2–3× runtime

125. Common Bugs

The catalogue, with the symptom that identifies each one. Learn the right-hand column — it is what you will actually see.

Bug	Cause	Symptom
Off-by-one	<code>i <= n</code> , or <code>n</code> instead of <code>n-1</code>	One wrong element, or a corrupted adjacent variable
Reverse loop underflow	<code>size_t i = n-1; i >= 0</code>	Infinite loop; at <code>n = 0</code> , immediate crash
Memory leak	Missing <code>delete[]</code>	RSS grows monotonically
Dangling pointer	Pointer outlives the block	Intermittent garbage; ASan "heap-use-after-free"
Uninitialised read	<code>No = {}</code>	Different result each run
Iterator invalidation	<code>push_back</code> during iteration	Crash or skipped elements at capacity boundaries
Shallow copy	Copying a struct holding a raw pointer	Double free at scope exit
Wrong <code>sizeof</code>	<code>sizeof(p)</code> after decay	Loop runs $8/4 = 2$ times (Topic 17)

bugs – every entry above is prevented by one of: `vector`, a range-for, `size_t` discipline, or a sanitizer in CI

126. Memory Leaks

```
void leak() {
    int* a = new int[1000];
    if (bad_input) return;          // LEAK: 4000 bytes, every call
    delete[] a;
}
// Also a leak, and a common one:
int* p = new int[10];
p = new int[20];                   // the first block is now unreachable

// Detect:
// g++ -fsanitize=address ...      prints "detected memory leaks" at exit,
//                               with the allocation stack trace
// valgrind --leak-check=full ./prog
```

Expected ASan output shape: `Direct leak of 4000 byte(s) in 1 object(s) allocated from: #0 operator new[] ... #1 leak() at prog.cpp:2.`

THE TRAP

An early `return`, a `break`, or a thrown exception between `new[]` and `delete[]` leaks silently. Symptom: a long-running service whose RSS climbs by a fixed amount per request until the OOM killer intervenes overnight. This is why RAII (Topic 29) is not a style preference: it is the only construct that survives early exits.

leaks – cost: the process dies eventually · fix: `std::vector` or `unique_ptr<T[]>` makes them structurally impossible

127. Buffer Overflow

Writing past the end of an array overwrites whatever the allocator or the stack frame put next — another variable, a heap header, or a return address. This is the mechanism behind a large share of historical remote-code-execution vulnerabilities.

```
char buf[8];
std::strcpy(buf, "this string is far too long"); // 28 bytes into 8
// On the stack: overwrites saved registers and the return address.
// Symptom: crash on RETURN from the function, not at the copy.

// Safe versions:
std::snprintf(buf, sizeof buf, "%s", src); // truncates, always NUL-terminates
std::string s = src;                       // grows as needed (Topic 35)
```

stack frame	after strcpy of 28 bytes into buf[8]
[buf 8 bytes]	[buf][overwritten][overwritten]
[saved rbp]	^ locals clobbered
[return addr]	^ return address clobbered -> jumps anywhere
^ higher addresses	ASan: "stack-buffer-overflow, WRITE of size 28"

buffer overflow – cost: memory corruption or arbitrary code execution · detection: ASan at 2–3× runtime, or `-D_FORTIFY_SOURCE=2 -O2` at near-zero cost for the common `str*/mem*` cases

128. Dangling Pointer

```
int* danglingLocal() {
    int a[5] = {1,2,3,4,5};
    return a; // a dies at the brace; the pointer does not
}           // -Wreturn-local-addr warns. Listen to it.

std::vector<int> v{1,2,3};
int* p = &v[0];
v.push_back(4); // may reallocate
*p = 99;        // heap-use-after-free (Topic 115)

int* q = new int[10];
delete[] q;
q[0] = 1; // use after free
q = nullptr; // habit: null it immediately after delete
```

THE TRAP

Freed memory usually still holds the old values for a while, so the broken program prints the right answer during testing and the wrong one under load, when another allocation reuses the block. Symptom: "works on my machine", fails in production, never reproduces in the debugger. ASan reports `heap-use-after-free` plus the stack traces of both the free and the access — use it and stop guessing.

dangling – cost: nondeterministic corruption · returning `std::vector` by value costs one move (zero copies since C++11), which is why Topic 19 says to do exactly that

129. Uninitialized Values

```
int a[5];                // block scope: 5 garbage ints
int b[5] = {};           // all zero
int c[5] = {1};          // {1,0,0,0,0} - the rest ARE zeroed
static int d[5];         // static/global: always zeroed
std::vector<int> v(5);     // value-initialised: all zero
std::vector<int> w; w.reserve(5); // capacity 5, size 0, NO elements exist

int* h = new int[5];      // garbage
int* z = new int[5]();    // zeroed - the () matters
```

THE TRAP

Reading an uninitialised value is UB, and on Linux fresh pages arrive zeroed, so it often *looks* initialised in a small test program and misbehaves once the allocator starts recycling memory. Symptom: a different answer on each run, or an answer that changes when you add an unrelated variable. Catch it with `-fsanitize=memory` (Clang) or Valgrind's `--track-origins=yes`.

uninitialised – `= {}` costs one `memset`, about 3 GB/s; skipping it costs correctness · at 10^7 ints the `memset` is ~13 ms – measure before you decide it matters

130. Integer Overflow in Indexing

```
int lo = 1'000'000'000, hi = 2'000'000'000;
int mid = (lo + hi) / 2;           // OVERFLOWS: 3e9 > INT_MAX (2.147e9)
int ok  = lo + (hi - lo) / 2;      // correct, and the standard idiom

int n = 100'000;
int total = n * n;                // 10^10 overflows int -> 1410065408
long long good = 1LL * n * n;     // promote BEFORE multiplying

// This bug lived in java.util.Arrays.binarySearch for nine years.
```

THE TRAP

Signed overflow is UB, so the optimiser may assume `lo + hi` never overflows and delete your safety check. The bug only appears for arrays larger than 2^{30} elements or values near `INT_MAX` — never in a unit test. Symptom: a negative `mid`, an out-of-bounds access and a crash only on the largest input. Fix: `lo + (hi - lo) / 2`, and `1LL *` before any multiplication that could reach 2×10^9 (Topics 42, 111).

index overflow — `int` holds indices to 2.1×10^9 · `size_t` is 64-bit and unsigned · UBSan (`-fsanitize=signed-integer-overflow`) reports the exact line

131. Best Practices

In priority order. The first three prevent the majority of the bugs in Topic 125.

1. Use `std::vector` by default, `std::array` for fixed compile-time sizes, raw arrays essentially never (Topic 132).
2. Never write a manual index loop when an algorithm exists — range-for or `<algorithm>` cannot go out of bounds (Topic 136).
3. Build tests and CI with sanitizers on. `-fsanitize=address,undefined` costs 2–3× runtime and finds the bugs that reach production.
4. Pass `std::span`, not `(T*, size_t)` — the size cannot desynchronise (Topic 32).
5. Prefer `.at()` at boundaries — parsing input, handling indices from users — where the 2% is irrelevant.
6. Use `size_t` or a signed type consistently, and compile with `-Wall -Wextra -Wconversion -Wsign-compare`.
7. Name your sizes. `constexpr int MAX_USERS = 128`; not `128` in nine places (Topic 135).
8. Reserve when the size is known — 2.2× measured on 10^7 appends (Topic 115).

best practice — total cost: a few percent of runtime and one compiler flag line · total benefit: the entire bug table of Topic 125

132. Prefer vector / array

Use	When	Overhead vs raw
<code>std::vector<T></code>	Size known at runtime, or it changes	24-byte object; identical element access
<code>std::array<T,N></code>	Size is a compile-time constant	Zero — same bytes as <code>T[N]</code>
<code>std::span<T></code>	A parameter viewing someone else's data	16 bytes, no ownership
<code>T[N]</code> raw	Global tables in CP; C interop; embedded	—

```
// Everything the raw array loses, and the containers keep:
//  .size() survives being passed to a function    (Topic 17: decay)
//  copy and assignment work                      (raw arrays cannot be assigned)
//  .at(), .begin().end(), range-for, structured bindings
//  automatic release on scope exit or exception  (Topic 29)
//  std::array compiles to identical assembly for a[i]. Check with -S if unsure.
```

containers — `array`: 0 bytes and 0 cycles of overhead · `vector`: 24-byte object, one indirection to the heap block, same $O(1)$ access · the argument for raw arrays is almost always nostalgia

133. Use `size_t` Carefully

```
std::vector<int> v;                                // empty
for (size_t i = 0; i < v.size() - 1; ++i) { }      // v.size() is 0u:
                                                    // 0u - 1 == 18446744073709551615
                                                    // loop runs ~1.8e19 times -> crash

for (size_t i = v.size(); i-- > 0;) { }           // correct reverse loop
for (int i = (int)v.size() - 1; i >= 0; --i) { }   // also correct, with a cast

int n = -1;
if (n < v.size()) { }                             // n converts to a HUGE unsigned:
                                                    // the comparison is FALSE. -Wsign-compare
```

THE TRAP

Unsigned arithmetic never goes negative; it wraps. Every `size() - 1`, `i - 1` and `i >= 0` on a `size_t` is a candidate bug, and the empty container is the input that triggers it. Symptom: hang or immediate segfault, but *only* on empty input — which is exactly the case most tests omit. Always test with `n = 0`.

`size_t` – 64-bit unsigned; wraps silently • `-Wsign-compare` and `-Wconversion` catch most cases at compile time, at zero runtime cost

134. Use `.at()` When Needed

```
// index from an untrusted source: ALWAYS .at() or an explicit check
int idx = parseUserInput();
try {
    process(v.at(idx));                          // throws std::out_of_range
} catch (const std::out_of_range& e) {
    log("bad index: %s", e.what());
}
// hot inner loop over a range you just validated: [] is fine
for (size_t i = 0; i < v.size(); ++i) sum += v[i];
```

Rule: `.at()` at the boundary where an index enters your program, `[]` inside loops whose bounds you control. The measured cost of `.at()` in a memory-bound loop here was **2%** — so if you are unsure, use `.at()` and measure later.

`.at()` – one compare and a predictable branch; 25.8 ms vs 25.8 ms on 50M elements • throws rather than corrupting, which is the whole point

135. Avoid Magic Numbers

```
// Before: change the size, and you must find every 128 - including the one
// in a different file that meant something else.
int users[128];
for (int i = 0; i < 128; ++i) { }
memset(users, 0, 128); // BUG: 128 BYTES, not 128 ints

// After:
constexpr int MAX_USERS = 128;
std::array<int, MAX_USERS> users{};
for (int i = 0; i < MAX_USERS; ++i) { }
// or better: for (auto& u : users) { } - no bound to get wrong
// and: std::size(arr), arr.size(), sizeof(arr)/sizeof(arr[0]) for raw arrays
```

THE TRAP

The `memset` line above is the classic: it zeroes 128 bytes of a 512-byte array, leaving three quarters uninitialised. Symptom: the first 32 entries behave and the rest are garbage. Always `sizeof(arr)`, never a literal count, and prefer `= {}`.

magic numbers – `constexpr` costs nothing at runtime · the bug it prevents (byte count vs element count) is one of the most common in C code

136. Prefer Algorithms Library

Every hand-written index loop is a chance to write `<=`. The standard algorithms have no indices to get wrong, express intent in the name, and are usually better optimised than the loop you would write.

```
#include <algorithm>
#include <numeric>

std::sort(v.begin(), v.end()); // Topic 84
std::reverse(v.begin(), v.end()); // Topic 44
auto it = std::find(v.begin(), v.end(), x); // Topic 41
bool ok = std::binary_search(v.begin(), v.end(), x); // Topic 42, needs sorted
long long s = std::accumulate(v.begin(), v.end(), 0LL); // 0LL! Topic 130
int c = std::count(v.begin(), v.end(), 7); // Topic 49
v.erase(std::remove(v.begin(), v.end(), 0), v.end()); // erase-remove, Topic 40
auto [mn, mx] = std::minmax_element(v.begin(), v.end()); // Topic 48
std::iota(v.begin(), v.end(), 0); // 0,1,2,...
bool sorted = std::is_sorted(v.begin(), v.end()); // Topic 47
```

THE TRAP

`std::accumulate(v.begin(), v.end(), 0)` deduces the accumulator type from the initial value: `0` is `int`, so summing a vector of large ints overflows even though the result is stored in a `long long`. Symptom: a negative total for large inputs. Write `0LL`. This is the single most common STL bug in competitive programming.

algorithms – same or better asymptotics, zero index bugs · `std::sort` is 2–3× faster than a hand-written quicksort · `remove` does not shrink the container: it returns the new logical end (Topic 50)

PRACTICE SET — Topics 113-136

A. Conceptual

1. Two loops are both $\Theta(n)$. Give three distinct mechanisms that can make one $20\times$ slower than the other.
2. Explain why `push_back` is amortized $O(1)$ with growth factor 2 but $O(n)$ per call with growth factor 1. Show the total copy count in each case.
3. What exactly does "in place" guarantee, and which two common sorts fail it?
4. Why does a linked-list traversal defeat the hardware prefetcher when an array traversal does not? Your answer must mention *dependent loads*.
5. False sharing produces no data race and no wrong answers. Describe the only symptom, and the two ways to fix it.
6. Why does `struct { char a; int b; char c; }` occupy 12 bytes and not 6? Explain the trailing padding specifically in terms of arrays.
7. Name the two preconditions a loop must satisfy before the compiler will auto-vectorise it, and one flag that tells you why it refused.
8. Why is out-of-bounds access described as undefined behaviour rather than "reads adjacent memory"? Give a consequence that only appears at -O2.
9. Give the exact reason `for (size_t i = v.size() - 1; i >= 0; --i)` is wrong in two independent ways.
10. Why does `std::accumulate(b, e, 0)` overflow even when you assign the result to a `long long`?
11. Your profiler shows a hot loop is memory-bound. Explain why hand-written AVX intrinsics are unlikely to help, using the numbers from Topic 122.
12. Give a case where `.at()` is the wrong choice, and one where `[]` is professional negligence.

B. Tracing and arithmetic

1. A vector doubles from capacity 1. How many reallocations to reach 1,000,000 elements, and how many element-copies in total? Compare the copy count with n .
2. An `int` array of 10^7 elements. Give its size in MB, the number of 64-byte cache lines it spans, and the minimum number of DRAM accesses a full sequential scan needs.
3. At ~ 200 cycles per DRAM access and 3 GHz, what is the lower bound on the time to scan that array if every line missed? Compare with the measured 5.6 ms from Topic 116 and explain the difference.
4. 8 threads increment `int counts[8]`. How many distinct cache lines are involved? How many are involved after `struct alignas(64) Counter { long long v; };`?
5. Compute `sizeof` for: `{char,int,char}`, `{int,char,char}`, `{double,char}`, `{char,double}`. Then give the memory of a 10^7 -element array of the first two and the difference in MB.
6. `lo = 1'500'000'000`, `hi = 2'000'000'000`, both `int`. Compute `(lo+hi)/2` as 32-bit two's complement, and then `lo + (hi-lo)/2`. State both results.
7. A `size_t` loop runs `i = 0; i < v.size() - 1` on an empty vector. Write the exact value of `v.size() - 1` in decimal.
8. `memset(users, 0, 128)` on `int users[128]`. How many elements are actually zeroed, and how many bytes should have been passed?
9. A 200 MB array is scanned at a measured 25.2 ms. Compute the achieved bandwidth in GB/s and say whether SIMD could plausibly help.

C. Code tasks — build it, break it, observe, fix it

1. **Amortized growth, measured.** Instrument `push_back` to 10^7 : count reallocations by watching `capacity()`, and time it with and without `reserve`. Reference numbers: **21 reallocations to 10^6 ; 215.8 ms vs 99.1 ms** at 10^7 . Then plot per-append latency and find the spikes.
2. **Break it: iterator invalidation.** Take `int& r = v[0];` on a vector at exactly its capacity, then `push_back`, then write through `r`. Run it 100 times plain — record how often it appears to work — then once under `-fsanitize=address` and paste the diagnostic.
3. **Break it: the reverse loop.** Write `for (size_t i = v.size() - 1; i >= 0; --i)` and run it on a 3-element vector and then on an empty one. Record what happens in each case, then fix it with `for (size_t i = v.size(); i-- > 0;)` and verify both inputs.
4. **Break it: signed overflow in binary search.** Implement binary search with `mid = (lo + hi) / 2` over a synthetic index range near `INT_MAX` (you do not need the memory — search over a virtual range with a predicate). Run under `-fsanitize=signed-integer-overflow` and paste the report, then fix with `lo + (hi - lo) / 2`.
5. **Break it: the memset magic number.** Reproduce `memset(users, 0, 128)` on `int users[128]`, print the first 40 elements, and identify exactly where the zeros stop. Fix with `sizeof users` and confirm.
6. **Bounds-check cost.** Time a 50-million-element sum with `[]`, with `.at()`, and with `-D_GLIBCXX_ASSERTIONS`. Reference: **25.2 vs 25.8 ms (2%)**. Then repeat on an array small enough to fit L1 and report how the ratio changes — and explain why.
7. **Vectorisation.** Compile the sum-of-squares loop with and without `-fno-tree-vectorize` at both 8192 floats and 50M floats. Reference: **4.0× (L1) and 1.28× (DRAM)**. Print the sums from both builds and note whether they are bit-identical.
8. **Padding.** Reorder the fields of a struct from `{char,int,char}` to `{int,char,char}`, allocate 10^7 of them, and compare RSS from `/proc/self/status`. Add `static_assert` on both sizes.

ANSWER KEY — Topics 113-136

A. Conceptual

1. **(i) Cache-line utilisation** (stride 1 vs stride 16 → up to 16× traffic); **(ii) prefetchability** (constant stride vs dependent pointer loads); **(iii) vectorisation** (unit-stride loops become 4-8 wide). Branch misprediction is a legitimate fourth.
2. Factor 2: copies are $1+2+4+\dots+n/2 < n$, so **< 1 extra copy per element → O(1) amortized**. Factor 1 (grow by a constant): copies are $1+2+3+\dots+n = n(n+1)/2 = \Theta(n^2)$, i.e. $\Theta(n)$ per append.
3. It guarantees **O(1) or O(log n) auxiliary space**, not zero memory. **Merge sort (O(n) buffer) and counting sort (O(K))** fail it.
4. Because the address of the next node **is the result of the current load** — a dependent load. The CPU cannot issue the next fetch until the previous one returns, so ~200-cycle latencies serialise instead of overlapping. Array addresses are computed from the index, so dozens of loads are in flight at once.
5. Symptom: **the program is correct but gets slower as you add threads**, and no sanitizer complains. Fixes: **pad each slot to 64 bytes (alignas(64))**, or **accumulate into a local and write once**.
6. `char(1) + 3 pad + int(4) + char(1) + 3 trailing pad = 12`. The **trailing** padding exists so that in an array, element $i+1$ starts at a multiple of 4 — the struct's size must be a multiple of its strictest alignment or Topic 8's uniform stride would break alignment for every subsequent element.
7. **Unit stride and no possible aliasing** between the pointers (plus no early exit, and a countable trip count). `-fopt-info-vec-missed` reports the refusal reason.
8. Because the standard imposes **no requirement at all**, which licenses the optimiser to assume it cannot happen. Consequence at -O2: a loop written `i <= n` can be transformed on the assumption that `i < n`, or a null/bounds check can be deleted as unreachable — so the behaviour changes with the optimisation level.
9. **(i)** On an empty vector `size() - 1` wraps to 1.8×10^{19} and the first indexing crashes. **(ii)** `i >= 0` is always true for an unsigned type, so the loop never terminates — it wraps at 0 and continues.
10. Because the accumulator type is deduced from the *initial value*: `0` is `int`, so the addition is performed in `int` and overflows before the result is ever converted.
11. Because the bottleneck is DRAM bandwidth, not instruction throughput. Wider arithmetic does not make memory arrive sooner: the measured gain was **1.28×** on a DRAM-bound scan versus **4.0×** on L1-resident data.
12. Wrong choice: **a tight inner loop whose bounds you just computed and validated** — pay nothing for a check you already did. Negligence: **an index that came from user input, a file or a network packet** used with `[]`.

B. Tracing and arithmetic

1. Capacities 1,2,4,...,1048576: **21 reallocations** (measured). Total copies = $1+2+\dots+524288 = 1,048,575 \approx n$. So slightly **fewer than one extra copy per element**.
2. $10^7 \times 4 \text{ B} = 40 \text{ MB}$ (38.1 MiB); $40,000,000 / 64 = 625,000$ **cache lines**; a sequential scan needs at minimum **625,000 DRAM accesses** (one per line).
3. $625,000 \times 200 \text{ cycles} = 1.25 \times 10^8 \text{ cycles} \div 3 \times 10^9 \text{ Hz} = 41.7 \text{ ms}$ if every line missed and nothing overlapped. Measured was **5.6 ms** — about 7× faster, because the **prefetcher**

issues many line fetches concurrently, so latencies overlap and the scan runs at bandwidth, not at latency. **That gap is what prefetchability means.**

4. $8 \text{ ints} \times 4 \text{ B} = 32 \text{ bytes} \rightarrow$ **1 cache line** (possibly 2 if the array straddles a boundary). After padding: **8 lines**, one per counter — and the false sharing disappears.
5. `{char,int,char} = 12; {int,char,char} = 8; {double,char} = 16; {char,double} = 16`. At 10^7 elements: **120 MB vs 80 MB — 40 MB saved** by field order alone.
6. `lo+hi = 3,500,000,000 > INT_MAX`. As 32-bit two's complement: $3,500,000,000 - 4,294,967,296 = -794,967,296$, halved $\rightarrow$ **-397,483,648** (and it is UB, so this is the typical result, not a guaranteed one). Correct form: $1,500,000,000 + 250,000,000 =$ **1,750,000,000**.
7. **18,446,744,073,709,551,615** ($2^{64} - 1$).
8. **32 elements zeroed** ($128 \text{ bytes} \div 4$); it should have been `sizeof users = 512 bytes`.
9. $200 \text{ MB in } 25.2 \text{ ms} = 0.2 \text{ GB} / 0.0252 \text{ s} \approx$ **7.9 GB/s**. That is in the range of single-core DRAM bandwidth, so the loop is **memory-bound and SIMD cannot help** — confirmed by the measured $1.28\times$.

C. What to verify

1. Exactly **21 reallocations** to 10^6 with libstdc++ (capacity doubles from 1). Timing reference **215.8 ms vs 99.1 ms = 2.2 $\times$** . The per-append latency plot should show **21 spikes of geometrically increasing height**, the last copying 32 MB — that spike is the p99.9 problem described in Topic 115.
2. Plain builds often print the expected value, because the freed block is usually still mapped and untouched — expect it to "work" in a majority of runs. ASan reports **heap-use-after-free, WRITE of size 4**, with the *freed by thread T0 here* stack pointing at `push_back`'s reallocation. **The lesson is the inconsistency**, not the crash.
3. On 3 elements: prints indices 2,1,0 and then **does not stop** — `i` wraps from 0 to $2^{64}-1$ and the next access segfaults. On the empty vector: crashes on the **first** iteration. The fixed form prints 2,1,0 and does nothing at all for the empty vector.
4. UBSan prints **runtime error: signed integer overflow: 1500000000 + 2000000000 cannot be represented in type 'int'** at the `mid` line. After the fix, no diagnostic, and the search terminates correctly at the top of the range.
5. Zeros stop after element **31**; element 32 onwards is whatever was on the stack or heap. With `sizeof users` (512) the whole array is zeroed.
6. Reference: **25.2 ms vs 25.8 ms — about 2%**. On an L1-resident array the loop becomes compute-bound, so the extra compare and branch is a larger share: expect the gap to widen to roughly **10-30%**. The honest conclusion is that the cost depends entirely on whether you were waiting for memory.
7. Reference: **50.4 ms vs 199.8 ms (4.0 $\times$)** at 8192 floats; **25.2 ms vs 32.2 ms (1.28 $\times$)** at 50M. The sums are **not** bit-identical — 163,848,160 vs 163,842,048 — because vectorised reduction reassociates the additions. **That difference is the trap in Topic 122, observed rather than read.**
8. `{char,int,char}` is 12 bytes $\rightarrow$ $\sim 120 \text{ MB RSS}$; `{int,char,char}` is 8 $\rightarrow$ $\sim 80 \text{ MB}$. Expect the RSS difference to be close to **40 MB**, minus page-granularity noise.

HANDNOTE SUMMARY CARD — Cost, Cache & Safety

COST, CACHE AND CORRECTNESS (Topics 113-136)

COMPLEXITY: the only question is "does it MOVE elements?"

access/update $O(1)$ | append $O(1)$ amortized | insert/delete at p $O(n-p)$
linear search $O(n)$ | binary search $O(\log n)$ | comparison sort $O(n \log n)$
counting sort $O(n+K)$ | matrix scan $O(R \times C)$ but 22x apart by loop order

AMORTIZED (growth factor 2, libstdc++ doubles from 1)

copies to reach $n = 1+2+4+\dots+n/2 < n \rightarrow < 1$ extra copy per element
measured: 21 reallocations to $1e6$; $1e7$ push_back 215.8 ms, reserved 99.1 ms
growth factor 1 $\rightarrow \Theta(n^2)$ total. ANY single push back is still $O(n)$.
reallocation invalidates EVERY pointer, reference and iterator.

THE MEMORY HIERARCHY (the constant Big-O hides)

L1 ~4 cyc | L2 ~12-20 | L3 ~40-60 | DRAM ~200 64-byte line = 16 ints
array scan 10^7 ints 5.6 ms 1 miss per 16 elements, prefetched
scattered list, same data 1346.6 ms 240x - dependent loads, no prefetch
row vs column scan 22x SAME instructions
matmul i-k-j vs i-j-k 4.8x SAME FLOP count
Reason: the CPU is not slow at arithmetic, it is slow at WAITING.

LAYOUT

AoS: struct{x,y,z,vx,vy,vz} - touching one field wastes 87% of the line
SoA: parallel arrays - full line utilisation AND it vectorises
padding: {char,int,char}=12, {int,char,char}=8 $\rightarrow$ 40 MB per $1e7$ elements
declare members in DECREASING size order; static_assert the sizeof
false sharing: 2 threads writing within 64 B $\rightarrow$ up to 20x, NO race, NO
sanitizer warning. Fix: alignas(64) per slot, or accumulate in a local.
SIMD: 4.0x on L1-resident floats, 1.28x when DRAM-bound. -ffast-math CHANGED
the sum (163848160 vs 163842048). Check with -fopt-info-vec-missed.

THE BUG TABLE - learn the SYMPTOM column

i <= n	one bad element / clobbered neighbour variable
size_t i = n-1; i >= 0	infinite loop; instant crash when n == 0
(lo+hi)/2	negative mid, only on huge inputs; UB
accumulate(b,e,0)	negative total on big sums - write 0LL
missing delete[]	RSS climbs forever; ASan "Direct leak of N bytes"
ptr after push_back	heap-use-after-free, reproduces ~1 run in 100s
int a[5]; (no = {})	different answer every run
strcpy into a short buffer	crash on RETURN, not at the copy
memset(a, 0, 128) on int[128]	only 32 elements zeroed
delete vs delete[]	heap corruption, crash somewhere else entirely

DEFENCES, CHEAPEST FIRST

-Wall -Wextra -Wconversion -Wsign-compare	compile time, free
constexpr sizes, sizeof arr, range-for	free
std::vector / std::array / std::span	0 overhead for array; 24 B vector
.at() at every trust boundary	measured 2% on a 50M sum
-D_GLIBCXX_ASSERTIONS in CI	similar, aborts on bad index
-fsanitize=address,undefined in tests	2-3x runtime, finds the rest
reserve() when the size is known	2.2x on $1e7$ appends

PART V CHEATSHEET — Diagnosis in One Page

PART V: FROM SYMPTOM TO CAUSE

THE SYMPTOM YOU SEE	-> WHAT IT USUALLY IS	TOPIC
crash on RETURN from a function	stack buffer overflow	127
crash far from the bad code	any UB; run ASan first	124
different answer every run	uninitialised read	129
right at -O0, wrong at -O2	UB the optimiser exploited	124,122
works in tests, fails in production	dangling pointer / use-after-free after reallocation	128,115
hangs, or crashes only on n == 0	size_t underflow	133
negative total on large input	int overflow (accumulate, mid)	130,111
RSS climbs monotonically	leak on an early-return path	126
correct, but SLOWER with threads	false sharing	120
p99 latency spikes, clean median	push_back reallocation	115
only the first 32 of 128 zeroed	memset byte-vs-element count	135
fast on my laptop, slow on the box	working set outgrew L2/L3	116
wrong only at grid edges	bounds test after the read	109

THE MEASUREMENT LADDER - in this order, never skip a rung

1. Is it correct? -fsanitize=address,undefined
 2. What is the n? asymptotics only matter past the crossover
 3. Am I memory-bound? bytes moved / seconds -> GB/s. ~8 GB/s = bound.
 4. Is it the layout? row vs column, AoS vs SoA, padding, tiling
 5. Did it vectorise? -fopt-info-vec ; only now consider intrinsics
- perf stat -e instructions,cache-misses,cache-references ./prog

NUMBERS WORTH KNOWING BY HEART

cache line 64 B = 16 int = 8 double L1 ~32 KB L2 ~256 KB-2 MB
DRAM latency ~200 cycles ~ 60-70 ns single-core bandwidth ~8-10 GB/s
int = 4 B, holds to 2.1e9 long long = 8 B, holds to 9.2e18
1e7 int = 40 MB = 625,000 cache lines 8 MB stack = ~2e6 int
binary search: 20 steps at 1e6, 30 at 1e9
malloc ~50-100 ns | stack alloc ~1 ns | L1 hit ~4 cycles

CROSS-REFERENCES

- Topic 8/9 the address formula; every number on this card descends from it
Topic 31 vector growth is the amortized story in the standard library
Topic 102 loop order: the cleanest demonstration that 0() is not time
Topic 148 240x: arrays vs linked lists, measured on 10^7 ints
Topic 163-4 parallel and GPU arrays: false sharing and coalescing are the same cache-line argument, one level up

PART VI

Where Arrays Live, and What Else There Is

Nine places arrays are load-bearing in real systems, then six honest comparisons — including the three structures that are secretly arrays and the two cases where you should stop using one. Checklist items 137-153.

SECTION 15 — REAL-WORLD USES

137. Real-World Use Cases

Away from interview problems, arrays appear in production for four distinct reasons. Naming the reason tells you which properties you must preserve.

Reason	Example	Property relied on
Hardware demands it	DMA buffers, GPU vertex data, packet frames	Contiguity — the device reads a base address and a length
Index <i>is</i> the key	Lookup tables, pixel grids, process tables	O(1) addressing with no hashing
Throughput matters	Columnar databases, tensors, audio frames	Cache lines and SIMD (Topics 117, 122)
Size is bounded and known	Embedded sensors, fixed protocols	No allocator needed at all

THE KEY INSIGHT

Every case below is one of these four. If you cannot say which one you are in, you probably want `std::vector` and should stop optimising.

real-world — the array is the default; the burden of proof is on anything with pointers in it

138. Buffers

A buffer is a fixed array plus a policy for where the next byte goes. The **ring buffer** is the canonical form: a producer writes at `head`, a consumer reads at `tail`, both wrapping modulo the capacity. Nothing is ever shifted, so every operation is O(1).

```
template <typename T, size_t N>      // N a power of two: & (N-1) beats %
class Ring {
    T buf[N];
    size_t head = 0, tail = 0;      // head == tail means EMPTY
public:
    bool push(const T& v) {
        if ((head + 1) & (N - 1) == tail) return false;    // full
        buf[head] = v;
        head = (head + 1) & (N - 1);
        return true;
    }
    bool pop(T& out) {
        if (head == tail) return false;                    // empty
        out = buf[tail];
        tail = (tail + 1) & (N - 1);
        return true;
    }
};
```

```

N = 8, tail = 5, head = 2      (wrapped)
index  0    1    2    3    4    5    6    7
      [ D ][ E ][   ][   ][   ][ A ][ B ][ C ]
              ^head                ^tail
logical order: A B C D E      size = (head - tail) & (N-1) = 5

& (N-1) works ONLY when N is a power of two. Otherwise use %,
which costs ~20-40 cycles against ~1 for the mask.

```

THE TRAP

Using `head == tail` for both "empty" and "full" makes them indistinguishable, so a full buffer silently reports empty and you lose N elements. The version above sacrifices one slot to disambiguate — capacity is $N-1$, not N . Symptom: under sustained load the consumer sees nothing while the producer reports success. Alternatives: keep a separate count, or use free-running 64-bit indices and mask only when indexing.

ring buffer — push/pop $O(1)$, no allocation, no shifting · Space $N \cdot \text{sizeof}(T)$, one wasted slot · lock-free single-producer/single-consumer with atomics, and false sharing (Topic 120) if head and tail share a line

139. Lookup Tables

Precompute an answer for every possible input and index it directly. The array replaces a computation with a load — profitable whenever the computation exceeds ~ 4 cycles and the domain is small enough to stay in cache.

```

// popcount for a byte: 256 entries, 256 bytes, fits in 4 cache lines
constexpr auto POPCNT = [] {
    std::array<uint8_t, 256> t{};
    for (int i = 0; i < 256; ++i) t[i] = (i & 1) + t[i / 2];
    return t;
}();                                     // computed at COMPILE time (Topic 158)

// sin table for an audio oscillator: 4096 floats = 16 KB, L1-resident
// factorials mod p for combinatorics: fact[200000] - one pass, then  $O(1)$  nCr
// CRC-32: 256-entry table turns a bit loop into one byte-at-a-time step

```

THE TRAP

A table larger than L2 can be *slower* than recomputing: a DRAM miss is ~ 200 cycles, and integer multiply is 3. Symptom: an "optimisation" that improves a microbenchmark (table hot in L1) and regresses the real workload (table evicted between calls). Size the table against the cache, not against convenience — 256 entries good, 16 million entries usually bad.

lookup table — Time $O(1)$ load vs $O(k)$ compute · Space = domain size · profitable while the table stays under $\sim L2$ (256 KB–2 MB); modern CPUs also have a `popcnt` instruction that beats any table

140. Image Processing

An image is a flat array of pixels with a stride. Every filter in this book's Topic 109 shape — blur, edge detection, erosion — is a 2-D array traversal with neighbour offsets.


```
// 8-bit RGB, width w, height h. Interleaved layout:
// index of channel c of pixel (x, y) = (y * w + x) * 3 + c
// Note "stride", not "w * 3": rows are often padded to a multiple of 4 bytes.
uint8_t* px = img + y * stride + x * 3;

// 3x3 box blur, one channel - the hot loop of every image library
for (int y = 1; y < h - 1; ++y)
    for (int x = 1; x < w - 1; ++x) {
        int s = 0;
        for (int dy = -1; dy <= 1; ++dy)
            for (int dx = -1; dx <= 1; ++dx)
                s += src[(y + dy) * w + (x + dx)];
        dst[y * w + x] = s / 9;
    }
```

THE TRAP

Two classics. **Stride $\neq$ width $\times$ channels** — most formats pad rows for alignment, and assuming otherwise produces an image that shears progressively down the frame. **In-place filtering** reads pixels you have already overwritten, so the blur smears in the scan direction: you need a separate destination or a rolling line buffer.

image — a 4K RGBA frame is $3840 \times 2160 \times 4 = 33.2$ MB, larger than any L3 · a full-frame pass at 8 GB/s is ~ 4 ms, so 60 fps allows about 4 passes · separable filters turn 9 loads per pixel into 6

141. Game Development

```
// Tile map: one flat array, index = y * width + x (Topic 90)
std::vector<uint8_t> tiles(w * h);

// Entity data, SoA - the "data-oriented design" style (Topic 118)
struct World {
    std::vector<float> x, y, vx, vy;    // parallel arrays, one per field
    std::vector<uint32_t> id;
};
// physics step: pure sequential streaming, vectorises, no pointer chasing
for (size_t i = 0; i < w.x.size(); ++i) { w.x[i] += w.vx[i] * dt;
                                           w.y[i] += w.vy[i] * dt; }

// Removing an entity mid-frame: swap-with-last, NOT shift (Topic 40)
std::swap(x[i], x.back()); x.pop_back();    // O(1), order not preserved
```

game — 10,000 entities $\times$ 4 floats = 160 KB per array, L2-resident · SoA + swap-erase keeps the whole update $O(n)$ with unit stride · a 16 ms frame at 8 GB/s allows ~ 130 MB of traffic total

142. Databases

A database page is an 8 KB byte array with a *slot directory*: an array of offsets growing from one end while the row data grows from the other. That layout lets rows be variable-length while keeping the slot index $O(1)$ — the same trade as Topic 91's jagged rows, done inside one buffer.

```

8 KB page
+-----+-----+-----+-----+-----+-----+-----+-----+
| header | slot0| slot1| slot2| ---- free space ---| row2| row1 | row0 |
+-----+-----+-----+-----+-----+-----+-----+-----+
                ^ array of 2-byte offsets                ^ variable-length rows
                grows right ->                            <- grows left

slot i gives the byte offset of row i: 0(1) lookup by slot number.
Deleting a row clears the slot; the space is reclaimed by compaction,
which is Topic 40's shift, performed on a page instead of an array.

```

Columnar stores (Parquet, ClickHouse) go further and store each column as its own array — Topic 118's SoA argument, at the storage layer: a query touching 3 of 50 columns reads 6% of the bytes and vectorises over each.

database – page 8 KB, slot 2–4 bytes, 0(1) by slot · B-tree nodes are sorted arrays searched by binary search (Topic 42), sized to one page precisely so the search is cache- and disk-friendly

143. Operating Systems

```

// Kernel tables are arrays indexed by a small integer - that is what a
// "handle" or "descriptor" IS:
// fd 3 -> current->files->fdt->fd[3]      an array of struct file*
// PID   -> hashed or radix-tree indexed, historically a flat table
// page tables: a 4-level RADIX TREE of 512-entry arrays (x86-64)
// virtual address bits 47:39 -> index into level 4 (512 entries * 8 B
// = 4096 B = exactly one page). Each level is an array lookup.

```

THE KEY INSIGHT

A file descriptor is not an object, it is an *index*. That is why descriptors are small non-negative integers, why they are reused immediately after **close**, and why the lowest free one is always allocated — Topic 8's address formula, exposed as an API.

OS tables – 0(1) by descriptor · page table walk: 4 dependent array lookups, ~100+ cycles on a TLB miss, which is why huge pages exist

144. Embedded Systems

```

// No allocator, no exceptions, fixed memory budget. Static arrays only.
static uint16_t adc_samples[256];      // .bss, zeroed at reset
static volatile uint8_t uart_rx[64];   // volatile: written by an ISR
// A ring buffer (Topic 138) sized to a power of two is the standard ISR-to-
// main-loop channel: the ISR pushes, the main loop pops, no locking needed
// for one producer and one consumer.

// Memory-mapped hardware registers are arrays at a fixed address:
#define GPIO ((volatile uint32_t*)0x40020000)
GPIO[5] = 1;                          // register 5 - the address formula on real hardware

```

THE TRAP

Omitting `volatile` on a buffer shared with an interrupt handler lets the compiler cache the value in a register, so the main loop never observes the ISR's write. Symptom: the program works with optimisation off and hangs at `-O2` — the same signature as UB, but here the code is legal and the compiler is right.

embedded — Space is the binding constraint: a 20 KB SRAM part allows about 10,000 `uint16_t` · static arrays have zero allocation cost and deterministic worst-case timing, which dynamic memory cannot promise

145. Machine Learning

A tensor is a flat array plus a *shape* and a *stride* vector. Everything exotic-sounding — reshaping, transposing, broadcasting, slicing — is arithmetic on strides, with the data untouched.

```
// NumPy/PyTorch model: data + shape + strides (in elements)
//  shape   = (2, 3, 4)           24 elements, one contiguous buffer
//  strides = (12, 4, 1)          row-major: stride[i] = product of shape[i+1:]
//  index(i,j,k) = i*12 + j*4 + k    Topic 90, generalised to n dims

// transpose() returns a VIEW: same buffer, strides swapped to (4, 12, 1).
// No copy. This is why .contiguous() exists - some kernels require stride 1.
// A 512x512 float32 matmul is  $2 \times 512^3 = 268$  MFLOP; a tuned BLAS sustains
// 10-100 GFLOP/s. The naive loop of Topic 102 managed  $512^3 \times 2 / 0.0921$  s
// = 2.9 GFLOP/s.
```

tensors — Space: float32 = 4 B/element, so a $1024 \times 1024 \times 1024$ tensor is 4 GB · transpose/reshape are $O(1)$ metadata edits · training throughput is bounded by memory bandwidth far more often than by FLOPs

146. Graphics Programming

```
// A vertex buffer is an array uploaded to the GPU as raw bytes.
struct Vertex { float pos[3]; float normal[3]; float uv[2]; }; // 32 bytes
std::vector<Vertex> verts(n);
glBufferData(GL_ARRAY_BUFFER, verts.size() * sizeof(Vertex),
             verts.data(), GL_STATIC_DRAW);
// verts.data() is the base address; the GPU is told stride = 32 and the byte
// OFFSET of each attribute. It is Topic 8's formula, executed by hardware.

// Index buffer: an array of uint32 indices into the vertex array, so shared
// vertices are stored once. A cube: 8 vertices, 36 indices instead of 36
// vertices -  $8 \times 32 + 36 \times 4 = 400$  bytes instead of 1152.
```

graphics — 32-byte vertex = 2 per cache line · indexed drawing saves $\sim 3\times$ on typical meshes · the GPU's post-transform vertex cache rewards index locality, so mesh reordering is a real optimisation

147. Networking

```
// A packet is a byte array with a layout, parsed by offset - not by struct
// casting, which breaks on alignment and endianness.
uint16_t port = (uint16_t(buf[0]) << 8) | buf[1];    // big-endian on the wire
// Ethernet 14 B | IPv4 20 B | TCP 20 B | payload up to 1460 B (1500 MTU)

// Scatter-gather: an ARRAY OF BUFFERS, so a header and a body are sent
// without being copied into one block first:
struct iovec iov[2] = {{hdr, hdrlen}, {body, bodylen}};
writev(fd, iov, 2);          // one syscall, zero copies
```

THE TRAP

Casting a received buffer to a packed struct pointer assumes the host's endianness and alignment match the wire format. Symptom: correct on x86, garbage on a big-endian or strict-alignment target, and a possible bus error. Parse byte by byte, or use `ntohs/ntohl`.

networking – MTU 1500 B, so a 1 MB response is ~700 packets · scatter-gather removes one full copy per send · ring buffers (Topic 138) are how NIC drivers hand packets to the kernel

SECTION 16 — COMPARISONS WITH OTHER STRUCTURES

148. Arrays vs Linked List

The comparison every course teaches wrongly. On paper the linked list wins insertion; in practice you almost never have the pointer already, and the traversal cost swamps everything.

```
// Measured on this machine, 10,000,000 ints, summing every element:
//  std::vector<int> contiguous scan          5.6 ms
//  linked list, nodes shuffled in memory 1346.6 ms      240x slower
//  The list allocated 10^7 nodes of 16 bytes = 160 MB against the array's
//  40 MB, and every step is a dependent load the prefetcher cannot predict.
```

Operation	Array	Linked list
Access by index	O(1)	O(n)
Insert/delete <i>given a pointer</i>	O(n) shift	O(1)
Insert/delete <i>given an index</i>	O(n)	O(n) — the walk dominates
Full traversal	5.6 ms	1346.6 ms
Memory per int	4 B	16–32 B (value + pointer + header)
Splice two sequences	O(n)	O(1)
Stable references under growth	No (Topic 115)	Yes

THE KEY INSIGHT

Use a linked list when you need **O(1) splice** or **references that survive insertion** — an LRU cache's intrusive list, an allocator's free list, a scheduler's run queue. Everything else, use a vector, and consider swap-with-last (Topic 40) instead of shifting.

arrays vs lists – array wins traversal by 240× measured and memory by 4× · list wins splice and reference stability · "O(1) insertion" is real only when you already hold the node

149. Arrays vs Stack

A stack is not an alternative to an array — it is an array with two operations and one index. `std::stack` defaults to `std::deque`, but `std::vector` is the natural backing store.

```
std::vector<int> st;
st.push_back(x);           // push: O(1) amortized
int top = st.back();        // top: O(1)
st.pop_back();             // pop: O(1), no shifting - it is the END
// Fixed-capacity version with no allocation at all:
int buf[N]; int sp = 0;
buf[sp++] = x;             // push
int v = buf[--sp];         // pop      - two instructions each
```

THE TRAP

Pushing and popping at the *front* of a vector is $O(n)$ per operation, turning a linear algorithm into a quadratic one. Symptom: the algorithm is fine at $n = 1000$ and takes minutes at $n = 100,000$. The stack discipline exists precisely because the cheap end of an array is the back.

stack – push/pop/top $O(1)$ · Space $O(n)$ · with `reserve`, zero allocations and no pointer overhead – this is why recursion-to-iteration conversions use a vector

150. Arrays vs Queue

A naive array queue that pops from the front is $O(n)$ per pop. Two standard fixes: the ring buffer (Topic 138), or — for the many algorithms that never need to reclaim space — a plain vector with a moving `head` index.

```
// BFS queue, the competitive-programming idiom: no pops at all
std::vector<int> q; q.reserve(n);
size_t head = 0;
q.push_back(start);
while (head < q.size()) {
    int u = q[head++]; // O(1), nothing shifts
    for (int v : adj[u]) if (!seen[v]) { seen[v] = true; q.push_back(v); }
}
// Memory is O(total pushes) rather than O(max queue size) - deliberate:
// it also leaves you the full visit order afterwards, for free.
```

queue – ring buffer: $O(1)$ both ends, fixed memory · vector + head index: $O(1)$ both ends, memory grows to total pushes · `std::deque`: $O(1)$ both ends, chunked storage, $\sim 2\times$ slower to traverse than a vector

151. Arrays vs Hash Table

Both give $O(1)$ lookup. The array's index is the key itself and costs one multiply; the hash table computes a hash, masks it, probes, and compares — and loses cache locality on the probe. Use the array whenever the key space is small and dense.

	Array (direct index)	Hash table
Key requirement	Small dense integer	Anything hashable
Lookup cost	1 multiply + 1 load	hash + mask + probe + compare
Typical measured ratio	1x	5–10x slower per lookup
Memory for 10^6 ints	4 MB	~ 40 – 60 MB (buckets, nodes, load factor)
Sparse keys (e.g. 64-bit IDs)	Impossible	The only option
Iteration in key order	Free	Requires a sort

```
// Frequency counting, values 0..255: the array wins outright (Topic 49)
int freq[256] = {};
for (unsigned char c : data) ++freq[c];          // ~1 ns per element

// Same code with unordered_map<char,int>: 5-10x slower and ~10x the memory,
// for identical output. The map is right only when the key space is unknown
// or sparse - user IDs, strings, coordinates.
```

array vs hash – array: $O(1)$ with a tiny constant, Space $O(\text{key range})$ · hash: $O(1)$ expected with a 5–10× constant, Space $O(n)/\text{load factor}$, worst case $O(n)$ on collisions

152. Arrays vs Tree

Trees buy *ordered* operations that arrays cannot do in less than $O(n)$: insert while keeping sorted order, predecessor/successor, range queries with updates. A sorted array beats a tree on everything static.

Operation	Sorted array	Balanced BST
Search	$O(\log n)$, cache-friendly	$O(\log n)$, pointer chase
Insert / delete	$O(n)$ shift	$O(\log n)$
Successor / range scan	$O(1)$ / sequential	$O(\log n)$ / pointer walk
Memory per int	4 B	~40 B (2 pointers, parent, colour, header)
Build from n items	$O(n \log n)$ sort	$O(n \log n)$ inserts, ~10× slower

```
// Static data queried many times? Sort once, binary search forever.
std::sort(v.begin(), v.end());
bool found = std::binary_search(v.begin(), v.end(), x);
// Interleaved inserts and queries? std::set / std::map. The crossover is
// roughly: if fewer than ~log(n) inserts happen between query batches, the
// sorted array still wins, because a re-sort is  $O(n \log n)$  but sequential.
// Adjacency lists (vector<vector<int>>, Topic 91) are the honest hybrid:
// a tree/graph shape stored in arrays.
```

array vs tree – array: 4 B/element, binary search touches ~log n cache lines · BST: ~40 B/element and every level is a dependent load; a B-tree (arrays of keys per node) exists precisely to fix that

153. Arrays vs Heap

A binary heap *is* an array. The tree is implicit in the index arithmetic: no pointers are stored, and the structure is exactly Topic 8's formula applied twice.

```
// For node i (0-based):
//   parent = (i - 1) / 2   left = 2i + 1   right = 2i + 2
std::vector<int> h;
std::push_heap(h.begin(), h.end()); // after push_back:  $O(\log n)$  sift up
std::pop_heap(h.begin(), h.end());  // moves max to the back:  $O(\log n)$ 
h.pop_back();
std::make_heap(h.begin(), h.end()); //  $O(n)$ , not  $O(n \log n)$ 
```

array:	[9 7 8 3 6 2]	implicit tree:
index	0 1 2 3 4 5	<pre> 9 / \ 7 8 / \ / 3 6 2 </pre>

children of 1 are 3 and 4
parent of 5 is $(5-1)/2 = 2$

No pointers stored. 4 bytes per element instead of ~40 for a node tree,
and the top of the heap is always index 0 - one load.

THE KEY INSIGHT

Heap, hash table and matrix are all the same move: replace a pointer with an arithmetic rule on an index. When you meet a new structure, ask whether its links could be computed instead of stored — that question is the whole content of Part VI.

heap – push/pop $O(\log n)$, top $O(1)$, build $O(n)$ · Space 4 B per int with zero pointer overhead ·
sift-down jumps by $2i+1$, so deep levels miss cache; a 4-ary heap is measurably faster at large n
for this reason

PRACTICE SET — Topics 137-153

A. Conceptual

1. Name the four reasons production code uses arrays, and give one system from this section for each.
2. Why must a ring buffer either waste one slot or keep a separate count?
3. Why is $\& (N-1)$ valid only for power-of-two capacities, and what does it save over $\%$?
4. Give the condition under which a lookup table is *slower* than recomputing the value.
5. Explain why a file descriptor is a small integer, in terms of Topic 8.
6. What is a tensor's *stride* vector, and why does `transpose()` cost $O(1)$?
7. Why does a columnar database read fewer bytes for a query touching 3 of 50 columns? Connect it to AoS vs SoA (Topic 118).
8. State the two situations where a linked list genuinely beats a vector, and say why "O(1) insertion" is not usually one of them.
9. Why is pushing at the front of a vector $O(n)$ while pushing at the back is $O(1)$ amortized?
10. Give two properties a key must have for a direct-index array to beat a hash table.
11. A binary heap stores no pointers. Write the three index formulas and explain what replaced the pointers.
12. When is a sorted array strictly better than a `std::set` containing the same data? When is it strictly worse?

B. Tracing and arithmetic

1. A ring buffer with $N = 8$, `tail = 5`, `head = 2`. How many elements does it hold? Which index does the next `push` write to? After how many more pushes does it report full?
2. A 4K RGBA frame: give the byte size, the number of 64-byte cache lines, and the time for one full pass at 8 GB/s. How many passes fit in a 16.7 ms frame?
3. A 1500-byte MTU with 14 B Ethernet, 20 B IPv4 and 20 B TCP headers. How many payload bytes per packet, and how many packets for a 1 MB response?
4. 10^7 ints: give the memory for (a) `vector<int>`, (b) a singly linked list at 16 B per node, (c) a `std::set<int>` at ~40 B per node. Express (b) and (c) as multiples of (a).
5. In a heap stored as `[9,7,8,3,6,2]`: give the parent of index 5, the children of index 1, and the index of the last non-leaf node for $n = 6$.
6. A vertex is `{float pos[3]; float normal[3]; float uv[2];}`. Give its size, vertices per cache line, and the total bytes for a 100,000-vertex mesh drawn with 300,000 indices of 4 bytes — versus the same mesh with no index buffer.
7. Frequency counting over 10^7 bytes with `int freq[256]`: how much memory does the table use, and does it fit in L1? Compare with `unordered_map<char,int>` at ~48 B per node for 256 distinct keys.
8. A 512×512 `float` matrix multiply is $2 \cdot 512^3$ FLOP. Compute the GFLOP/s of the 92.1 ms i-k-j run from Topic 102.
9. A page is 8 KB with a 24-byte header and 4-byte slots. How many 100-byte rows fit, accounting for their slots?

C. Code tasks — build it, break it, observe, fix it

1. **Ring buffer.** Implement the power-of-two ring from Topic 138 with `push`, `pop` and `size`. Test it by pushing and popping 10^6 times in an interleaved pattern and asserting the sequence comes out in order.
2. **Break it: full looks empty.** Change the ring so that a full buffer has `head == tail` (use all N slots, no wasted one, no count). Fill it completely, then call `pop`. Record what it returns and how many elements you silently lost. Fix it both ways — the wasted slot and the explicit count — and verify with the same test.
3. **Break it: front-push quadratic.** Build a queue two ways: `v.insert(v.begin(), x) / v.erase(v.begin())`, and the vector-plus-head-index BFS idiom. Time both for $n = 10^4, 10^5, 2 \times 10^5$. Plot the times and confirm one curve is quadratic. Report the ratio at the largest n .
4. **The 240× measurement.** Reproduce Topic 148: sum 10^7 ints in a `vector` and in a linked list whose nodes are allocated in shuffled order. Report both timings, the memory used by each (`/proc/self/status VmRSS`), and then repeat with the list nodes allocated *in order* — explain why that version is much faster and why it is not representative.
5. **Break it: table too big.** Write a function computed two ways — directly (a few multiplies) and from a lookup table. Measure with a 256-entry table, then a 64-million-entry table (256 MB), calling it on random indices. Report both timings and identify the crossover where the table stops helping.
6. **Break it: in-place blur.** Implement the 3×3 box blur writing into the source array. Run it on a small test image with a single white pixel and print the result grid. Describe the asymmetry you see, then fix it with a destination buffer and compare the two outputs numerically.
7. **Array vs hash.** Count byte frequencies over 10^7 random bytes with `int freq[256]` and with `unordered_map<unsigned char, int>`. Report both timings, the ratio, and peak RSS for each.
8. **Heap without pointers.** Implement `siftUp` and `siftDown` over a `vector<int>`, then verify against `std::make_heap/push_heap/pop_heap` on 10^6 random values. Count comparisons for `make_heap` and confirm it is $O(n)$, not $O(n \log n)$ — the count should be under $2n$.

ANSWER KEY — Topics 137-153

A. Conceptual

1. **Hardware demands contiguity** (NIC ring buffers, GPU vertex buffers); **the index is the key** (fd tables, pixel grids, lookup tables); **throughput** (columnar DBs, tensors); **bounded known size** (embedded sensor buffers).
2. Because `head == tail` is the natural test for empty, and after N pushes the head wraps back onto the tail — **the two states become indistinguishable**. Wasting a slot makes full `head+1 == tail`; a count distinguishes them directly.
3. Because `x & (N-1) == x % N` only when N is a power of two — **N-1** must be a mask of all ones. It saves a hardware division: **~1 cycle instead of ~20-40**.
4. When the table **does not fit in cache**: a DRAM miss is ~200 cycles while the computation may be 3-10. A table above L2, accessed randomly, is slower than recomputing.
5. Because it is an **index into the process's open-file array**, not a pointer. Hence small, non-negative, immediately reusable, and always the lowest free slot.
6. The stride vector gives **how many elements to step for each dimension**; the index is $\sum i_k \cdot \text{stride}_k$. Transpose just **permutes the strides** — the buffer is untouched, so it is O(1) metadata, and the result is non-contiguous.
7. Because each column is stored as its own contiguous array, so the query reads only those three arrays — **~6% of the bytes**. Row storage is AoS: it drags all 50 fields through the cache to use 3.
8. **O(1) splice** and **references that stay valid across insertions**. "O(1) insertion" assumes you already hold the node pointer; if you have an index you must walk there, which is O(n) and 240× slower per step than an array scan.
9. Because inserting at the front must **shift all n elements** to keep the layout dense, while appending writes one element (and only occasionally reallocates).
10. The key must be a **small integer** and the key space must be **dense** (most slots used). Otherwise the array is either impossible or mostly empty.
11. `parent = (i-1)/2`, `left = 2i+1`, `right = 2i+2`. **Arithmetic on the index replaced the stored pointers** — saving ~36 bytes per element and one dependent load per level.
12. **Strictly better** when the data is static or rebuilt in batches and you only search and scan: 4 B vs ~40 B per element, and cache-friendly binary search. **Strictly worse** when inserts and deletes are interleaved with queries: O(n) shift per insert against the set's O(log n).

B. Tracing and arithmetic

1. $\text{Size} = (\text{head} - \text{tail}) \& 7 = (2 - 5) \& 7 = 5$. Next push writes index 2, then head becomes 3. It reports full when `(head+1)&7 == tail == 5`, i.e. head = 4 — so after **2 more pushes** (capacity is $N-1 = 7$, and 5 are held).
2. $3840 \times 2160 \times 4 = 33,177,600 \text{ B} \approx 33.2 \text{ MB}$; $\div 64 = 518,400 \text{ cache lines}$; at 8 GB/s one pass $\approx 4.15 \text{ ms}$ (read only; 8.3 ms if you also write). About **4 read passes** fit in a 16.7 ms frame — which is why real pipelines fuse filters into one pass.
3. $1500 - 54 = 1446 \text{ payload bytes}$; $1,048,576 / 1446 = 726 \text{ packets}$.
4. (a) **40 MB**. (b) $10^7 \times 16 = 160 \text{ MB} = 4\times$. (c) $10^7 \times 40 = 400 \text{ MB} = 10\times$.
5. Parent of 5 = $(5-1)/2 = 2$. Children of 1 = **3 and 4**. Last non-leaf = $n/2 - 1 = 2$.
6. Vertex = 8 floats = **32 B**; **2 per 64-byte line**. Indexed: $100,000 \times 32 + 300,000 \times 4 = 3,200,000 + 1,200,000 = 4.4 \text{ MB}$. Non-indexed: $300,000 \times 32 = 9.6 \text{ MB}$ — **2.2× more**.

7. `int freq[256] = 1024 B`, comfortably in L1 (32 KB). The hash map: $256 \times 48 = 12,288 \text{ B}$ plus the bucket array — still small, but each lookup costs a hash, a mask, a pointer dereference and a compare, so the measured gap is **5-10×** in time.
8. $2 \times 512^3 = 268,435,456 \text{ FLOP} \div 0.0921 \text{ s} = \mathbf{2.9 \text{ GFLOP/s}}$. A tuned BLAS on the same core reaches 10-100 GFLOP/s — **the naive loop leaves 5-30×** on the table even after the loop-order fix.
9. Usable = $8192 - 24 = 8168 \text{ B}$. Each row costs $100 + 4 = 104 \text{ B}$, so $8168/104 = \mathbf{78 \text{ rows}}$ ($78 \times 104 = 8112 \text{ B}$, leaving 56 B free).

C. What to verify

1. The interleaved test must come out in exact FIFO order, and `size()` must never exceed **N-1**. Assert both.
2. With all N slots used and no count, a completely full buffer has `head == tail`, so `pop` reports **empty and returns false** — you have silently lost **all N elements**. Both fixes restore correct behaviour; the count version gives you the full N of capacity at the cost of one extra word and, in a lock-free setting, one more atomic.
3. The `insert(begin)/erase(begin)` version is $\Theta(n^2)$: expect roughly **100×** the time for **10×** the n. At $n = 2 \times 10^5$ the gap against the head-index version is typically **3-4 orders of magnitude** (milliseconds vs seconds).
4. Reference: **5.6 ms vs 1346.6 ms = 240×**, with RSS about **40 MB vs 160 MB**. Allocating the nodes in order makes them nearly contiguous, so the scan may come within **2-4×** of the array — it is unrepresentative because a real list has been inserted into and deleted from over time, which scatters it. **The number you should quote is the shuffled one.**
5. The 256-entry table should beat direct computation comfortably; the 256 MB table, accessed randomly, should be **slower than recomputing** — every access is a DRAM miss plus a TLB miss. The crossover typically appears once the table exceeds L2 (~1-2 MB).
6. The in-place version **smears the white pixel in the scan direction** (down and to the right), because those neighbours have already been overwritten with blurred values. Differences of several units per pixel will appear in the lower-right quadrant only. The out-of-place version is symmetric — a 3×3 block of equal values around the original pixel.
7. Expect the array to win by **5-10×** in time and roughly **10×** in the memory attributable to the counter, with the array version running at close to memory bandwidth.
8. `make_heap` on 10^6 elements should perform **fewer than 2×10^6 comparisons** (the bound is $2n$), confirming $O(n)$ — while n inserts via `push_heap` take $\sim n \log n \approx 2 \times 10^7$. Your `siftDown` output must match `std::is_heap` exactly.

HANDNOTE SUMMARY CARD — Real Uses & Comparisons

WHERE ARRAYS LIVE, AND WHAT ELSE THERE IS (Topics 137-153)

FOUR REASONS PRODUCTION CODE USES AN ARRAY

hardware demands contiguity	NIC rings, GPU vertex buffers, DMA
the index IS the key	fd tables, pixel grids, lookup tables
throughput	columnar DBs, tensors, audio frames
bounded, known size	embedded sensors, fixed protocol frames

RING BUFFER (Topic 138) - the shape of every buffer in a real system

push at head, pop at tail, both & (N-1); N MUST be a power of two
head == tail means EMPTY -> you must waste one slot or keep a count
capacity N-1. SPSC needs no lock. Pad head and tail apart (Topic 120).

SYSTEM FACTS WORTH CARRYING

4K RGBA frame	33.2 MB, ~4.2 ms per pass at 8 GB/s, ~4 passes per frame
image stride	!= width*channels; rows are padded. Assuming so shears.
MTU 1500	1446 payload bytes after 14+20+20; 1 MB = 726 packets
DB page 8 KB	slot directory from one end, rows from the other
fd / PID / GPU vtx	all "handle == array index"
tensor	flat buffer + shape + strides; transpose is O(1) metadata
vertex 32 B	2 per cache line; index buffer saves ~2.2x on a mesh
embedded	static arrays only; volatile on anything an ISR writes

THE COMPARISON TABLE (all measured or standard)

	access	insert@ptr	traverse 1e7	bytes/int	splice
array/vector	O(1)	O(n)	5.6 ms	4	O(n)
linked list	O(n)	O(1)	1346.6 ms	16-32	O(1)
sorted array	O(log n)	O(n)	cache-friendly	4	-
balanced BST	O(log n)	O(log n)	pointer chase	~40	-
hash table	O(1) exp	O(1) exp	5-10x per lookup	40-60	-
binary heap	top O(1)	O(log n)	implicit tree	4	-

240x is the number to remember: contiguous scan vs scattered node walk.

THE STRUCTURES THAT ARE SECRETLY ARRAYS

stack	vector + push_back/pop_back	the cheap end is the BACK
queue	ring buffer, or vector + head idx	never erase(begin())
heap	parent (i-1)/2, left 2i+1, right 2i+2	pointers -> arithmetic
hash table	array of buckets indexed by hash & (N-1)	
B-tree node	sorted array of keys sized to one page	
matrix	flat array + stride	(Topic 90)

WHEN TO STOP USING AN ARRAY - say this out loud in an interview

interleaved insert/delete in the middle, by index	-> BST / skip list
sparse or non-integer keys	-> hash table
need references stable across insertion	-> list / deque
need O(1) splice of whole sequences	-> linked list
mostly zeros (< ~10% dense)	-> CSR (Topic 112)
data larger than RAM	-> mmap (Topic 162)

PART VI CHEATSHEET — Choosing the Structure

PART VI: THE DECISION IN ONE PAGE

START HERE

Do you index by a small dense integer?	yes -> ARRAY. Stop.
Is the data static once built?	yes -> SORTED ARRAY + binary search. Stop.
Are inserts/deletes interleaved with queries?	yes -> continue below
Do you need "the smallest/largest" repeatedly?	yes -> BINARY HEAP (an array)
Are the keys sparse, or not integers?	yes -> HASH TABLE
Do you need ordered range scans AND updates?	yes -> BALANCED BST / B-tree
Do you need O(1) splice or stable references?	yes -> LINKED LIST
Is it > 90% zeros?	yes -> CSR
Is it bigger than RAM?	yes -> MMAP / EXTERNAL SORT

COSTS TO QUOTE WHEN DEFENDING THE CHOICE

array scan 1e7 int	5.6 ms	40 MB	
scattered list, same	1346.6 ms	160 MB	240x, 4x memory
set of 1e7		~400 MB	10x memory
hash lookup vs freq[256]	5-10x slower	~10x memory	
binary search 1e6	20 steps	1e9 -> 30 steps	
push_back 1e7	215.8 ms	99.1 ms with reserve	

THE THREE QUESTIONS THAT SETTLE MOST DESIGN ARGUMENTS

1. What is n, and does the asymptotic difference bite before n gets there?
2. What is the access pattern - sequential, random, or by key?
3. Is the structure built once or mutated continuously?

CROSS-REFERENCES

Topic 40	swap-with-last: how to delete from an array in O(1) when order does not matter - the fix that saves most "I need a list" cases
Topic 91	vector: correct for adjacency lists, wrong for grids
Topic 112	CSR: the honest exit from dense arrays
Topic 115	reallocation invalidates references - the real cost of vector
Topic 138	ring buffer: the queue that never shifts and never allocates
Topic 153	heap: the cleanest example of computing links instead of storing

PART VII

Advanced C++ and the Interview

The parts of the standard library that exist because raw arrays lose their size, the places arrays stop being ordinary memory — bits, mapped files, GPUs, disk — and then the interview and practice material that turns 177 topics into something you can perform under pressure. Checklist items 154–177.

SECTION 17 — ADVANCED C++ & SYSTEM-LEVEL ARRAYS

154. Arrays in STL Algorithms

Every algorithm in `<algorithm>` and `<numeric>` works on a raw array, because they take iterators and a pointer *is* an iterator (Topic 155). No conversion, no wrapper, no cost.

```
#include <algorithm>
#include <numeric>
int a[8] = {5, 3, 9, 1, 3, 7, 3, 2};
int* first = a; int* last = a + 8;           // or std::begin/std::end(a)

std::find(first, last, 9);                   // pointer to a[2], else last
std::count(first, last, 3);                  // 3
std::accumulate(first, last, 0LL);          // 33 - 0LL, not 0 (Topic 136)
std::fill(first, last, 0);                   // memset-speed for trivial T
std::copy(first, last, dst);                 // memmove for trivial T
std::iota(first, last, 1);                   // 1,2,3,...
std::max_element(first, last);               // pointer to the max
std::partial_sum(first, last, out);          // prefix sums (Topic 60)
std::adjacent_difference(first, last, out);  // the inverse (Topic 62)
std::rotate(first, first + k, last);        // rotate by k (Topic 57)
std::nth_element(first, first + k, last);    // k-th smallest, O(n) average
```

You were about to write	Use instead
A loop that finds a value	<code>find</code> / <code>find_if</code>
A loop that counts	<code>count</code> / <code>count_if</code>
A loop that sums	<code>accumulate</code> (with <code>0LL</code>)
A loop that filters into a new array	<code>copy_if</code> / <code>remove_if</code>
A loop that finds the k-th smallest	<code>nth_element</code> — $O(n)$, not $O(n \log n)$
A "is every element..." check	<code>all_of</code> / <code>any_of</code> / <code>none_of</code>

THE TRAP

`std::begin(a)` and `sizeof(a)` stop working the moment the array decays to a pointer (Topic 17), so these calls compile inside the function that declared the array and fail — or, worse, silently use `sizeof(int*)` — one call level down. Symptom: a loop that processes exactly 2 elements of a 1000-element array. Pass `std::span` (Topic 32).

STL on arrays — identical asymptotics, usually faster constants (`fill`/`copy` become `memset`/`memmove` for trivially copyable types) · zero abstraction penalty at -O2

155. Iterators

An iterator generalises the pointer: `*it`, `++it`, `it != end`. For an array the iterator *is* a raw pointer, which is why array code and container code can share one algorithm library.

```
int a[5] = {1,2,3,4,5};
for (int* it = std::begin(a); it != std::end(a); ++it) sum += *it;
for (int x : a) sum += x; // the same loop, written sanely

// end() is the ONE-PAST-THE-END address - legal to form, UB to dereference
// (Topic 5). That is why every range in C++ is half-open [first, last):
// size      = last - first
// empty     = (first == last)
// midpoint  = first + (last - first) / 2 // never (first+last)/2
```

```
a: [ 1 | 2 | 3 | 4 | 5 ]
    ^           ^
    begin()      end()  <- points PAST the last element, never read

half-open [first, last): no +1/-1 anywhere, empty is first==last,
and concatenating [a,b) with [b,c) gives [a,c) with no overlap.
```

iterators – dereference and increment are $O(1)$ and compile to the same instructions as indexing · random-access iterators (array, vector, deque) support `it + n` in $O(1)$; list iterators do not

156. Range Library

```
#include <ranges>
#include <algorithm>
std::vector<int> v{5, 2, 8, 1, 9, 4};

std::ranges::sort(v); // no .begin()/.end() pair
auto it = std::ranges::find(v, 8);

// Views are LAZY and non-owning - no intermediate array is materialised:
for (int x : v | std::views::filter([](int n){ return n % 2 == 0; })
          | std::views::transform([](int n){ return n * n; }))
    std::cout << x << ' '; // 4 64 16

auto firstThree = v | std::views::take(3);
auto reversed   = v | std::views::reverse;
```

THE TRAP

A view does not own its data and does not extend its lifetime. Returning a view over a local vector, or over a temporary (`makeVector()` | `views::take(3)`), leaves a dangling range — the same bug as Topic 128 with new syntax. Symptom: garbage on iteration, ASan `heap-use-after-free`. Return the vector, or materialise with `ranges::to` (C++23).

ranges – same complexity, zero-cost at -O2 for simple pipelines · lazy: `filter` | `transform` makes one pass and allocates nothing · compile times and error messages are the real price

157. Templates with Arrays

A template can capture an array's size at compile time, which a plain pointer parameter cannot — this is how `std::size` and `std::begin` work for raw arrays.

```
// N is DEDUCED from the argument: no size parameter, no decay (Topic 23)
template <typename T, std::size_t N>
constexpr std::size_t arraySize(T (&)[N]) noexcept { return N; }

int a[42];
static_assert(arraySize(a) == 42);           // checked at compile time

// Generic sum over any container OR raw array:
template <typename Range>
auto sum(const Range& r) {
    using V = std::decay_t<decltype(*std::begin(r))>;
    return std::accumulate(std::begin(r), std::end(r), V{});
}
```

templates – zero runtime cost; the size is a compile-time constant · one instantiation per distinct N, so heavy use grows the binary · prefer `std::span` when you do not need N in the type

158. Compile-Time Arrays

```
// Built during compilation; the binary contains the finished table.
constexpr std::array<int, 10> squares = [] {
    std::array<int, 10> t{};
    for (int i = 0; i < 10; ++i) t[i] = i * i;
    return t;
}();
static_assert(squares[7] == 49);           // verified by the compiler

// Typical uses: CRC tables, sine tables, parser transition tables,
// factorials, powers of two - anything in Topic 139 with a fixed domain.
```

THE KEY INSIGHT

`constexpr` moves work from runtime to compile time *and* lets the compiler check invariants with `static_assert`. A table that is wrong now fails the build instead of the customer.

constexpr arrays – 0 ns at runtime, table lives in `.rodata` · costs compile time and binary size · still subject to Topic 139's cache rule: a 4 MB constexpr table is still a 4 MB table

159. Bit Arrays

When each element is one boolean, storing a byte per element wastes 87.5% of the memory. Packing 64 flags per `uint64_t` makes a 10^9 -flag sieve fit in 125 MB instead of 1 GB, and makes whole-word operations possible.

```
std::vector<uint64_t> bits(n / 64 + 1, 0);
inline void set (size_t i) { bits[i >> 6] |= (1ULL << (i & 63)); }
inline void clear(size_t i) { bits[i >> 6] &= ~(1ULL << (i & 63)); }
inline bool test (size_t i) { return bits[i >> 6] >> (i & 63) & 1ULL; }
// i >> 6 is i / 64; i & 63 is i % 64. Both are one instruction.
// Population count of the whole array: __builtin_popcountll per word.
```

bit array – Space $n/8$ bytes (8× less than `bool[]`) · set/test 0(1), 2–3 instructions · a 64× denser scan means 64× fewer cache lines, so sieves and bitmap indexes get a large real speedup

160. std::bitset

```
#include <bitset>
std::bitset<1024> b;           // 128 bytes, size fixed at compile time
b.set(7); b.reset(7); b.flip();
b.count();                    // popcount, one instruction per word
b1 &= b2;                      // whole-set AND: 16 words = 16 ops
// Classic use: subset-sum / knapsack reachability in O(n * W / 64)
std::bitset<100001> reach; reach[0] = 1;
for (int w : items) reach |= reach << w;    // 64x faster than a bool DP loop
```

bitset – Space $N/8$ bytes, compile-time N · whole-set ops are $O(N/64)$ words · the `reach |= reach << w` trick is the standard 64x constant-factor win in competitive programming (Topic 173)

161. vector<bool>

The standard library's most famous mistake: `std::vector<bool>` is a specialisation that packs bits, so it is *not* a container of `bool` and breaks code that assumes it is.

```
std::vector<bool> v(8, true);
// bool& r = v[0];           // DOES NOT COMPILE: operator[] returns a PROXY
auto r = v[0];                // r is std::vector<bool>::reference, not bool
bool* p = &v[0];              // DOES NOT COMPILE: there is no bool to address
// v.data() does not exist. Generic template code silently misbehaves.

// Want packed bits? -> std::bitset or Topic 159's manual version
// Want a real container of bool? -> std::vector<char> or std::vector<uint8_t>
```

THE TRAP

A template written and tested against `vector<int>` fails to compile, or quietly changes meaning, when instantiated with `vector<bool>` — because `auto x = v[i]` captures a proxy that *refers back into the vector*. If the vector is then resized, the proxy dangles. Symptom: a template that works for every type except `bool`.

`vector<bool>` – Space $n/8$ bytes · access needs a shift and mask, so it is slower per element than `vector<char>` despite being 8x smaller · no `data()`, no pointers, no references

162. Memory-Mapped Arrays

`mmap` makes a file appear as an array in your address space. The kernel loads pages on demand, so you can "index" a 100 GB file with ordinary array syntax and never call `read`.

```
#include <sys/mman.h>
#include <fcntl.h>
int fd = open("data.bin", O_RDONLY);
size_t bytes = lseek(fd, 0, SEEK_END);
const int* a = (const int*)mmap(nullptr, bytes, PROT_READ, MAP_PRIVATE, fd, 0);
// a[1'000'000'000] now works. The page fault does the I/O.
madvise((void*)a, bytes, MADV_SEQUENTIAL); // tell the kernel your pattern
munmap((void*)a, bytes);
```

THE TRAP

Random access to a mapped file is not $O(1)$ in any useful sense: a page fault to disk costs $\sim 100\ \mu\text{s}$ on SSD and $\sim 10\ \text{ms}$ on spinning rust — 10^3 to 10^5 times an L1 hit. Symptom: a program that is instant on a small test file and appears to hang on the real one, with near-zero CPU utilisation. Check `/proc/self/stat` major faults before blaming your algorithm.

mmap – Space: virtual, only touched pages are resident · sequential access is excellent (the kernel reads ahead); random access costs a page fault · 4 KB page granularity, so touching 1 int loads 4096 bytes

163. Parallel Array Processing

```
#include <execution>
std::sort(std::execution::par_unseq, v.begin(), v.end());
long long s = std::reduce(std::execution::par, v.begin(), v.end(), 0LL);

// Manual split - the pattern to understand, because the library hides it:
// give each thread a CONTIGUOUS chunk, and a private accumulator.
auto worker = [&](size_t lo, size_t hi, long long& out) {
    long long local = 0; // local, not out[t] (Topic 120)
    for (size_t i = lo; i < hi; ++i) local += v[i];
    out = local; // one write at the end
};
```

THE TRAP

Two failure modes. **False sharing** if each thread writes to adjacent slots of a results array — correct output, up to $20\times$ slowdown (Topic 120). **Bandwidth saturation** if the loop is memory-bound: a scan already running at $\sim 8\ \text{GB/s}$ on one core will not speed up on four, because the cores share the memory controller. Symptom: 4 threads, $1.1\times$ speedup.

parallel – ideal speedup = cores, real speedup = $\min(\text{cores}, \text{bandwidth limit} / \text{single-core rate})$ · contiguous chunking is the only partitioning that preserves prefetching · threads are worth it above roughly 10^5 elements of real work

164. GPU Arrays

```
// CUDA: the array is the only data structure. One thread per element.
__global__ void saxpy(int n, float a, const float* x, float* y) {
    int i = blockIdx.x * blockDim.x + threadIdx.x; // the index formula,
    if (i < n) y[i] = a * x[i] + y[i]; // computed per thread
}
saxpy<<<(n + 255) / 256, 256>>>(n, 2.0f, d_x, d_y);
// COALESCING: consecutive threads must touch consecutive addresses. Then a
// 32-thread warp's loads merge into a few 128-byte transactions. A strided
// or pointer-chasing access pattern costs up to 32 separate transactions.
```

THE KEY INSIGHT

Coalescing is Topic 117's cache-line argument with a wider line and 32 requesters. It is also why linked structures are effectively unusable on a GPU, and why every ML framework stores tensors as flat arrays with strides (Topic 145).

GPU – 10^3 – 10^4 threads, ~500 GB/s–3 TB/s bandwidth · PCIe transfer ~10–25 GB/s, so a kernel must do enough work to repay the copy · uncoalesced access can cost 10–30× on the same kernel

165. External Memory Arrays

When the array does not fit in RAM, the cost model changes: you count *block transfers*, not comparisons. Sorting 100 GB with 8 GB of RAM is the canonical example, and the answer is a k-way external merge sort — Topic 46's merge, run on files.

100 GB input, 8 GB RAM, 1 MB blocks

PASS 1 read 8 GB, sort in memory, write a sorted RUN x13 runs
PASS 2 open all 13 runs, merge with a heap of 13 heads, stream out
(each run needs only one buffered block in RAM at a time)

total I/O: 2 passes over 100 GB = 200 GB of transfer, sequential.
A naive "sort in place on disk" does random I/O and takes days.

external – cost model: $O((n/B) \cdot \log_{\{M/B\}}(n/B))$ block transfers, not comparisons · sequential disk ~500 MB/s–7 GB/s (NVMe), random 4 KB reads 10–100× worse · this is also why B-trees have huge fan-out (Topic 152)

166. Lesser-Known Facts

- `a[i] == i[a]`, because both are `*(a+i)` (Topic 5).
- `sizeof(arr)/sizeof(arr[0])` silently breaks after decay — `std::size(arr)` refuses to compile instead, which is better.
- Zero-length arrays are illegal in standard C++, but `struct { int n; char data[]; }` (a flexible array member) is legal C and widely used in kernels and network code.
- Arrays cannot be assigned or returned by value — but a `struct` containing one can be, which is exactly what `std::array` exploits.
- Pointer subtraction yields `ptrdiff_t`, which is **signed**, while `size()` is unsigned — a common source of `-Wsign-compare` warnings.
- Comparing pointers into *different* arrays with `<` is unspecified behaviour, though `==` is fine.
- `new int[0]` is legal, returns a unique non-null pointer, and must still be `delete[]`d.
- An `int a[1000]` declared `static` inside a function is zeroed once, keeps its value across calls, and is not on the stack.

trivia – each item above corresponds to a real bug class; the flexible array member is the only one you should reach for deliberately

167. Unique Uses

Technique	What the array is doing
Counting sort / bucketing	The <i>index</i> carries the value (Topic 81)
Union-Find	<code>parent[i]</code> — a whole forest with no nodes
Fenwick tree	A tree of prefix sums stored in one array by bit tricks
Sieve of Eratosthenes	A bit array indexed by the number itself
State machines	<code>next[state][input]</code> — a matrix replaces branching
Cycle detection	<code>visited[]</code> as a timestamp array, not a bool
Sparse set	Two arrays giving $O(1)$ insert, delete, lookup <i>and</i> $O(k)$ iteration over present elements

THE KEY INSIGHT

Every entry here replaces a data structure with an index. That is the single transferable idea of this whole book: *if the link can be computed, do not store it.*

unique uses — all $O(1)$ per operation with 4–8 bytes per element and no pointers · all rely on a small dense key space (Topic 151)

SECTION 18 — INTERVIEW, DSA & CP

168. Interview Notes

Array interviews are not a memory test. They check whether you can name a pattern from a phrasing, state a cost, and handle the empty input. Do these five things in order, every time.

1. **Restate the guarantee.** Sorted? Distinct? Value range? Can I modify the input? Each answer removes an entire family of solutions — sorted unlocks two pointers and binary search; a bounded range unlocks counting (Topic 49).
2. **State the brute force and its cost**, in one sentence. "Check all pairs, $O(n^2)$." You now have a correct baseline and a target to beat.
3. **Name the pattern out loud** before coding — "contiguous subarray with a sum condition, so sliding window" (Topic 64). The table in Topic 174 is that mapping.
4. **Write the loop invariant as a comment**, then the code. Most two-pointer and window bugs are invariant bugs, not typing bugs.
5. **Trace the test matrix aloud:** $n = 0$, $n = 1$, all equal, already sorted, reverse sorted, target absent, duplicates, overflow at 10^9 .

THE TRAP

Optimising before stating the brute force. If you jump straight to a clever $O(n)$ trick and it is wrong, you have nothing to fall back on and no baseline to discuss. A stated $O(n^2)$ with a correct trace scores better than a broken $O(n)$.

interview — expected: brute force in ~2 min, optimal approach named in ~5, code in ~15, edge cases traced in ~5 · the cost strip of every topic in this book is the sentence they want to hear

169. Beginner Problems

Problem	Technique	Topic
Sum / average / min / max in one pass	Single scan	48
Second largest without sorting	Two trackers, one pass	48
Linear search with index return	Scan	41
Reverse in place	Two pointers	44
Check palindrome	Two pointers	65
Count occurrences of each value (0-255)	Frequency array	49
Remove duplicates from a sorted array	Read/write pointers	50
Move zeros to the end, order preserved	Write pointer	51
Left-rotate by one	Save, shift, restore	53
Merge two sorted arrays	Two pointers	46

beginner — all $O(n)$ or $O(n \log n)$, all $O(1)$ extra space except merge · if any of these takes more than 10 minutes, redo the Part IV practice set

170. Intermediate Problems

Problem	Technique	Topic
Maximum subarray sum	Kadane	63
Sum of every window of size k	Fixed sliding window	64
Longest subarray with sum $\leq S$ (positives)	Variable window	64
Range sum with many queries	Prefix sums	60
Add v to every element of many ranges	Difference array	62
Two-sum in a sorted array	Two pointers	65
Sort 0s, 1s and 2s in one pass	Dutch national flag	59
Rotate by k in $O(1)$ space	Three reversals	57
Majority element ($> n/2$)	Boyer-Moore + verify	67
Missing number in $1 \dots n$	Sum or XOR	68
Find the duplicate, read-only input	Floyd cycle detection	66, 69
Next lexicographic permutation	Scan, swap, reverse suffix	73
Merge overlapping intervals	Sort by start, then sweep	84
Product of all except self, no division	Prefix $\times$ suffix	60, 61

intermediate – this table is roughly 80% of array interview questions · every row is $O(n)$ or $O(n \log n)$, and most are $O(1)$ extra space

171. Advanced Problems

Problem	Technique	Topic
Longest increasing subsequence, $O(n \log n)$	Tails array + binary search	42, 71
Maximum in every window of size k	Monotonic deque	64
Largest rectangle in a histogram	Monotonic stack	149
Trapping rain water	Two pointers or prefix max	61, 65
Subarray sum equals k (negatives allowed)	Prefix sums + hash map	60, 151
k-th smallest element	<code>nth_element</code> / quickselect	79, 154
Median of two sorted arrays, $O(\log n)$	Binary search on the partition	42
Search in a rotated sorted array	Binary search with a sorted-half test	42, 57
Count inversions	Merge sort with a counter	78
Maximum rectangle of 1s in a binary matrix	Histogram per row	92
Spiral / rotate / set-matrix-zeros in place	Index composition	104, 105
Range sum on a 2-D grid, many queries	2-D prefix sums	111

THE TRAP

"Sliding window" only works when the quantity is **monotone** as the window grows — sums of non-negative numbers, distinct-count, and similar. With negative numbers present, a shrinking window can improve the sum, so the window argument collapses and you need prefix sums plus a hash map instead. Symptom: a solution that passes every positive-only test and fails on the first case with a negative value.

advanced – the added ingredient is always one of: a monotonic structure, a second array, or binary search over the *answer* rather than the data

172. DSA Roadmap

Arrays are the first structure for a reason: six later structures are arrays underneath (Topic 153), and three later techniques are array patterns reused.

```
ARRAYS (this book)
|
+--> Strings                char arrays + a length      (Topics 35-37)
+--> Stack / Queue          arrays with two indices    (Topics 149, 150)
+--> Hash Tables            array of buckets + hash     (Topic 151)
+--> Heaps / Priority Q      implicit tree in an array  (Topic 153)
+--> Binary Search          the pattern, not the array  (Topic 42)
|
|      +--> binary search on the ANSWER, monotone predicates
+--> Sorting                comparison + counting/radix (Topics 74-83)
|
|      +--> Greedy: sort, then sweep
+--> Two pointers / windows --> Linked lists, strings, intervals
+--> Prefix sums -----> Fenwick / segment trees, 2D queries
+--> Matrices -----> Graphs as adjacency matrices, DP tables
|
|      +--> Dynamic programming IS an array
|              of subproblem answers
```

THE KEY INSIGHT

Dynamic programming, the topic students fear most, is nothing but "fill an array in an order such that every entry's dependencies are already filled." You have already done it: prefix sums (Topic 60) is a one-line DP.

roadmap – next: strings, then stacks/queues, then hashing, then recursion and trees · every one of them reuses this book's index arithmetic

173. Competitive Programming

```
#include <bits/stdc++.h> // one header, contest only
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr); // 3-10x faster I/O
    // ...
}
// Global arrays, not local: no 8 MB stack limit, zero-initialised for free
const int MAXN = 200005;
int a[MAXN], freq[MAXN];
long long pre[MAXN]; // ALWAYS long long for prefix sums

// Constant-factor tools that decide contests:
// bitset<N> for DP reachability 64x (Topic 160)
// counting sort when values are bounded O(n) (Topic 81)
// two pointers instead of a nested loop n^2 -> n
// precomputed factorials + modular inverses O(1) nCr
// difference array for range updates O(1) per update
```

n	Feasible complexity at $\sim 10^8$ ops/s
$n \leq 10$	$O(n!)$ or $O(2^n \cdot n)$
$n \leq 20$	$O(2^n)$
$n \leq 500$	$O(n^3)$
$n \leq 5,000$	$O(n^2)$
$n \leq 10^6$	$O(n \log n)$
$n \leq 10^8$	$O(n)$, and mind the constant

THE TRAP

Reading the constraint table backwards is the fastest way to a wrong-answer verdict: $n \leq 10^5$ with values up to 10^9 means the *values* overflow `int` in a sum long before n does. Symptom: accepted on the samples, wrong answer on test 7. Default to `long long` for anything accumulated (Topics 130, 136).

CP – `sync_with_stdio(false)`: 3–10× on I/O-heavy problems · global arrays: no stack limit, pre-zeroed · `bitset`: 64× on boolean DP · these four are worth more than any algorithmic cleverness under contest time

174. Practice Roadmap

The mapping from problem phrasing to technique. This is the table to read before an interview; everything else in the book is the derivation.

The problem says...	Reach for	Topic
"contiguous subarray"	Sliding window / Kadane / prefix sums	60, 63, 64
"subsequence" (not contiguous)	DP, or sorting — not a window	71
"sum of a range, many queries"	Prefix sums (long long)	60
"add v to every element in [l, r]"	Difference array	62
"sorted array" + "pair/triplet"	Two pointers from both ends	65
"sorted" + "find/insert position"	Binary search / lower_bound	42
"minimum largest" / "maximum smallest"	Binary search on the answer	42
"in place" / "O(1) extra space"	Two pointers, reversal, swapping	44, 57, 59
"k-th smallest/largest"	Heap of size k, or quickselect	79, 153
"appears more than n/2 times"	Boyer-Moore, then verify	67
"values are 0...K, K small"	Counting / frequency array	49, 81
"find the cycle" / "the middle"	Fast and slow pointers	66
"maximum/minimum in each window"	Monotonic deque	64
"next greater/smaller element"	Monotonic stack	149
"rotate" / "shift by k"	Three reversals, $k \% n$ first	56, 57
"intervals" / "meetings"	Sort by start, sweep	84
"grid" / "island" / "region"	Direction offsets + BFS/DFS	109, 110
"rectangle sum in a matrix"	2-D prefix sums	111
"mostly zeros"	Sparse representation	112

practice – work the ladder in order: Topic 169 until each is under 10 minutes, then 170 until you name the pattern before coding, then 171 · 20 problems from Topic 170 beats 100 random ones

175. Mini Projects

Each of these forces one property of arrays to matter, and each is small enough to finish in an evening.

Project	What it forces you to confront
Student marks report	Parallel arrays vs array of structs; min/max/average in one pass (Topic 118)
Fixed-capacity inventory	Insert, delete-by-shift, and the moment you switch to swap-with-last (Topics 39, 40)
PGM image filter	Stride vs width, in-place aliasing, 3×3 neighbourhoods (Topics 140, 109)
Tic-tac-toe / Conway's Life	Flat grid indexing, 8 neighbours, double buffering (Topics 90, 109)
Ring-buffer logger	Wraparound arithmetic, full-vs-empty, a fixed memory budget (Topic 138)
Sieve of Eratosthenes to 10^8	Bit packing, cache blocking, the memory wall (Topics 159, 116)
Your own Vector<T>	Growth factor, reallocation, copy vs move, RAII (Topics 26, 29, 115)
CSV to columnar loader	AoS vs SoA measured on a real query (Topics 118, 142)

projects – the `Vector<T>` one is the highest-value exercise in this book: it makes Topics 24–31 and 115 concrete in about 150 lines

176. Final Cheatsheet

Everything worth carrying into an exam room is consolidated in the end matter that follows this section: a **master cheatsheet** covering all 177 topics on four cards, the **pattern-recognition table** mapping phrasings to techniques, and the **revision tracker**. The ten lines below are the irreducible core — if you remember nothing else, remember these.

- | | |
|---|--|
| 1. <code>address(a[i]) = base + i * sizeof(T)</code> | everything else follows |
| 2. <code>a[i] == *(a+i) == i[a]</code> | subscript is not a primitive |
| 3. anything that MOVES elements is $O(n)$ | insert, delete, front-push |
| 4. 64-byte line = 16 ints | row-major, last index innermost |
| 5. doubling growth $\rightarrow O(1)$ amortized | <code>reserve()</code> removes 21 reallocs |
| 6. sorted? $\rightarrow$ binary search / two pointers | the guarantee picks the algorithm |
| 7. bounded values? $\rightarrow$ counting, $O(n+K)$ | the index carries the value |
| 8. contiguous? $\rightarrow$ window/Kadane/prefix | "subsequence" means it is NOT |
| 9. long long for anything accumulated | and $lo + (hi-lo)/2$, never $(lo+hi)/2$ |
| 10. test $n=0$, $n=1$, all-equal, and the empty | most bugs live in exactly these |

cheatsheet – ten lines, and every one of them is a number-carrying claim you can defend

SECTION 19 — FINAL

177. Summary & Conclusion

One hundred and seventy-seven topics reduce to a single sentence: **an array is a rule for computing an address, and every property — good and bad — is a consequence of that rule.**

What the rule bought

- **O(1) access** at any index, with no traversal and no comparison — one multiply and one add, folded into the load instruction.
- **Contiguity**, which the hardware rewards twice: one 64-byte line delivers 16 ints, and a constant stride is what the prefetcher is built to predict. Measured, that is **240×** against a scattered node walk on 10^7 ints, and **22×** between two loop orders over the same matrix.
- **A substrate**: strings, stacks, queues, heaps, hash tables, matrices, tensors, database pages, GPU buffers and DP tables are all this one structure with an index convention on top.

What the rule cost

- **A fixed size**, escaped only by allocating a new block and copying — the entire content of `std::vector` and of amortized analysis.
- **O(n) structural change**, because density is what makes the multiplication valid. Every insert and delete in the middle pays for the O(1) access you enjoy everywhere else.
- **No bounds check**, because the formula is happy to compute an address for any integer. That single fact generates most of Topic 125's bug table.

When to walk away from it

Say these out loud; they are the most valuable sentences in the book. Interleaved insertions and deletions in the middle → a balanced tree or a list. Sparse or non-integer keys → a hash table. More than ~90% zeros → CSR. Data larger than RAM → memory mapping or an external merge. References that must survive growth → a node-based container. **Knowing when the structure is wrong is the difference between having learned arrays and having memorised them.**

What comes next

Strings are char arrays with a terminator and a library. Stacks and queues are arrays with two indices. Hashing is an array plus a function. Heaps are arrays plus index arithmetic. Recursion and trees are where the array finally stops being enough — and you will meet them already knowing exactly what you gave up. Work the tracker at the back: Day 0 today, then +1, +7, +30, +90.

THE LAST INSIGHT

If the link can be computed, do not store it. That is what an array is, what a heap is, what a hash table is, and what a matrix is — and it is the idea you should carry into every structure in the rest of Phase 1.

arrays — Access $O(1)$ · Insert/delete $O(n)$ · Search $O(n)$, or $O(\log n)$ sorted · Space $O(n)$ with zero per-element overhead · the default choice, and the one you must be able to argue against

PRACTICE SET — Topics 154-177

A. Conceptual

1. Why do STL algorithms work on raw arrays with no adapter?
2. Why is every range in C++ half-open? Give two concrete simplifications it buys.
3. What does a `views::filter | views::transform` pipeline allocate, and how many passes does it make?
4. Why can a function template deduce an array's size when a pointer parameter cannot?
5. Give two things `constexpr` tables buy over runtime-built ones, and the one property of Topic 139 they do *not* escape.
6. Explain `i >> 6` and `i & 63` in a bit array, and why both are single instructions.
7. Name three ways `std::vector<bool>` breaks code written for `std::vector<T>`.
8. Why is random access into an `mmap`ed 100 GB file not usefully $O(1)$? Quantify.
9. You parallelise a memory-bound scan across 4 cores and get $1.1\times$ speedup. Explain, and say what measurement would confirm your explanation.
10. Define GPU coalescing in terms of Topic 117, and say why linked structures are unusable on a GPU.
11. Why does external merge sort count block transfers instead of comparisons?
12. The problem says "longest subsequence" rather than "longest subarray". Which techniques does that single word eliminate, and why?

B. Tracing and arithmetic

1. A bit array for 10^9 flags: give the memory in MB, the number of 64-bit words, and the same figure for `bool[]` and `vector<bool>`.
2. For bit index 1,000,000: compute the word index and the bit offset used by Topic 159's macros.
3. `std::bitset<100001>`: how many bytes, and how many word operations does one `reach |= reach << w` perform? Compare with a `bool` DP loop over the same 100,001 states.
4. An `mmap`ed file is read at random, 4 KB pages, 100 μ s per major fault. How long do 10,000 random reads take? How long if the same 10,000 reads were sequential over a warm 40 MB region?
5. Sorting 100 GB with 8 GB of RAM: how many initial runs, how many passes, and how many total bytes of I/O?
6. At 10^8 operations per second, give the largest n you can accept for $O(n^2)$, $O(n \log n)$ and $O(2^n)$ within a 1-second limit.
7. A contest problem has $n \leq 2 \times 10^5$ and values up to 10^9 . Give the maximum possible array sum and state which type is required.
8. A 32-thread warp reads `x[i]` with `i = tid`, then with `i = tid * 16`. How many 128-byte transactions in each case (4-byte floats)?
9. `std::array<int,10>` versus `int[10]`: give `sizeof` for both and say what the difference costs at runtime.

C. Code tasks — build it, break it, observe, fix it

1. **Break it: decay kills `std::size`.** Write a function taking `int a[]` that returns `sizeof(a)/sizeof(a[0])`, call it with a 1000-element array, and print the result. Then rewrite the

- parameter as `std::span<int>` and compare. Record both numbers and explain the first one exactly.
2. **Break it: dangling view.** Write a function returning `makeVector() | std::views::take(3)`, iterate the result, and print it. Run it plain, then under `-fsanitize=address`, and paste the diagnostic. Fix it two ways: return the vector, or bind the temporary to a named variable first.
 3. **Break it: `vector<bool>` in a template.** Write `template <class V> void firstToTrue(V& v) { auto& r = v[0]; r = true; }` and instantiate it with `vector<char>` and `vector<bool>`. Record the compiler error verbatim, then make it work for both and explain what `auto&` bound to in each case.
 4. **Bit array sieve.** Implement the Sieve of Eratosthenes to 10^8 twice: once with `vector<char>`, once with a packed bit array. Report memory (RSS) and time for both, and the prime count (should be **5,761,455**).
 5. **Break it: false sharing, for real.** Sum an array with 4 threads writing into `long long results[4]`, then into `alignas(64)` slots, then into thread-local accumulators. Report all three timings. If your machine has one core, run it anyway and explain why the effect does not appear — that is a valid observation, not a failed experiment.
 6. **mmap a large file.** Create a 2 GB file of ints, map it, and sum it (a) sequentially and (b) at random indices. Report both times and the major fault counts from `/proc/self/stat`. Then add `madvise(MADV_SEQUENTIAL)` to the sequential run and re-measure.
 7. **Interview drill.** Take five problems from Topic 170. For each, write only: the guarantee you would ask for, the brute force and its cost, the pattern name, and the test matrix — no code, four minutes each. Then implement the two you found hardest.
 8. **Build your own `Vector<T>`.** Implement `push_back`, `reserve`, `operator[]`, copy constructor, move constructor and destructor. Count reallocations to 10^6 (expect **21** with doubling), then change the growth factor to 1.5 and to +1 and report reallocation counts and timings for all three. Verify no leaks under ASan.

ANSWER KEY — Topics 154-177

A. Conceptual

1. Because the algorithms are written against **iterators**, and a raw pointer satisfies every requirement of a random-access iterator — so `int*` is a valid iterator with no wrapper.
2. **size = last – first** with no ± 1 , and **empty is first == last**; ranges also concatenate without overlap. A third: `end()` is expressible as a legal address (Topic 5).
3. **Nothing**, and **one** pass — views are lazy adaptors evaluated per element during iteration, not materialised containers.
4. Because the array-reference parameter `T (&)[N]` **prevents decay**, so `N` is part of the type and is deduced. A pointer parameter has already lost the size (Topic 17).
5. Buys: **zero runtime construction cost** and **compile-time verification with `static_assert`**. Does not escape: **the cache** — a 4 MB `constexpr` table is still a 4 MB table at runtime.
6. `i >> 6` is `i / 64` (which word) and `i & 63` is `i % 64` (which bit). Both are single instructions **because 64 is a power of two**, so division becomes a shift and modulo becomes a mask.
7. (i) `bool& r = v[0]` does not compile — `operator[]` returns a proxy. (ii) `&v[0]` does not compile; there is no `bool` object to address. (iii) `v.data()` does not exist. (A fourth: `auto x = v[i]` captures a proxy that can dangle after a resize.)
8. Because a page not resident costs a **major fault**: $\sim 100\ \mu\text{s}$ on SSD against $\sim 1\ \text{ns}$ for an L1 hit — **about $10^5\times$** . The operation is $O(1)$ in instructions and enormous in time.
9. The scan was already at the **memory-bandwidth limit** ($\sim 8\ \text{GB/s}$ on one core), and all cores share one memory controller, so adding cores adds no bandwidth. Confirm by computing GB/s achieved, or with `perf stat -e cache-misses` plus a bandwidth benchmark.
10. Coalescing means **the 32 threads of a warp touch consecutive addresses, so their loads merge into a few 128-byte transactions** — the cache-line argument with a wider line and 32 simultaneous requesters. Linked structures produce 32 unrelated dependent addresses, so they cannot merge at all and lose an order of magnitude.
11. Because when data does not fit in RAM the dominant cost is **disk I/O, which moves fixed-size blocks**; a comparison is free next to a seek. Random 4 KB reads are $10\text{--}100\times$ slower than sequential, so the algorithm is designed to make transfers sequential.
12. It eliminates **sliding windows and two pointers**, because a subsequence is not contiguous — there is no window to slide. It usually means **DP or sorting** instead (Topic 71).

B. Tracing and arithmetic

1. $10^9 / 8 = 125\ \text{MB}$, i.e. $10^9/64 = 15,625,000\ \text{words}$. `bool[]` is 1 byte each = **1000 MB**. `vector<bool>` also packs, so **125 MB** — but with a slower per-element access than the manual version.
2. $1,000,000 \gg 6 = 15,625$; $1,000,000 \& 63 = 0$ ($1,000,000 = 15,625 \times 64$ exactly).
3. $\lceil 100001/8 \rceil$ rounded to word size = **12,504 bytes**, i.e. **1,563 words** — so one shifted OR is about **1,563 word operations** instead of **100,001** iterations: a **$64\times$ constant-factor win**.
4. $10,000 \times 100\ \mu\text{s} = 1.0\ \text{second}$. Sequential over a warm 40 MB region is a memory scan at $\sim 8\ \text{GB/s} \approx 5\ \text{ms}$ — a factor of about **200**, and the reason `MADV_SEQUENTIAL` exists.
5. $100\ \text{GB} / 8\ \text{GB} = 13\ \text{runs}$ (12.5 rounded up); **2 passes** (one to create runs, one to merge 13 ways); $2 \times 100\ \text{GB read} + 2 \times 100\ \text{GB written} \approx 400\ \text{GB of I/O}$ (or 200 GB if you count each direction once).

6. $O(n^2)$: $n \approx 10^4$. $O(n \log n)$: $n \approx 10^6$ - 10^7 ($\log n \approx 20$ gives 5×10^6). $O(2^n)$: $n \approx 26$ ($2^{26} \approx 6.7 \times 10^7$).
7. $2 \times 10^5 \times 10^9 = 2 \times 10^{14}$, far past `int`'s 2.1×10^9 : **long long required**.
8. Consecutive: $32 \times 4 \text{ B} = 128 \text{ B}$ contiguous $\rightarrow$ **1 transaction**. Stride 16: the 32 addresses span $32 \times 64 \text{ B} = 2048 \text{ B} \rightarrow$ **16 transactions** of 128 B, of which 4 bytes each are used — **16× the traffic**.
9. Both are **40 bytes**. The difference costs **nothing at runtime** — `std::array` is a struct wrapping the same bytes, and `operator[]` compiles to the identical instruction. It gains `.size()`, copyability, and iterators.

C. What to verify

1. The function prints **2** on a 64-bit machine (`sizeof(int*) / sizeof(int) = 8/4`), regardless of the real size. GCC warns `-Wsizeof-pointer-div` if enabled. The `std::span` version prints **1000**. **This is Topic 17 costing you 998 elements**.
2. Plain, it often prints plausible-looking values — the temporary vector's buffer is freed but not yet reused. ASan reports **heap-use-after-free** with the free stack pointing at the end of the full expression. Both fixes eliminate it; naming the vector first extends its lifetime past the iteration.
3. The `vector<bool>` instantiation fails with something close to **cannot bind non-const lvalue reference of type 'std::_Bit_reference&' to an rvalue**. For `vector<char>`, `auto&` bound to a real `char&`; for `vector<bool>` the subscript returns a **temporary proxy object**. Using `auto` (by value) compiles for both and still writes through, because the proxy's assignment operator writes into the vector.
4. Prime count must be exactly **5,761,455**. Memory: **~100 MB (char) vs ~12.5 MB (bits)**. The bit version usually wins on time too despite the shift-and-mask, because it touches $8 \times$ fewer cache lines — report your ratio.
5. Expect adjacent `long long results[4]` to be the slowest, `alignas(64)` and thread-local accumulators to be roughly equal and much faster (commonly **2-20×** apart on a multi-core box). On a single-core machine (this build environment reports `nproc = 1`) the threads never run simultaneously, so **no line ever ping-pongs and all three versions match** — an honest negative result, and worth writing down.
6. Random reads should be dominated by major faults — expect roughly one fault per 4 KB page touched and a time in the seconds. Sequential should approach disk throughput with far fewer faults thanks to readahead; `MADV_SEQUENTIAL` typically improves it further and also lets the kernel drop pages behind you, keeping RSS flat.
7. Self-marked. The drill is complete only if the pattern name came *before* any code, and if every test matrix included **n = 0** and an overflow case.
8. Doubling: **21 reallocations** to 10^6 , total copies $< n$. Factor 1.5: **~35 reallocations**, more copies but less peak memory (and blocks can be reused, since $1.5 <$ the golden ratio). Growth of +1: **~10⁶ reallocations** and $\Theta(n^2)$ copies — expect it to take **minutes rather than milliseconds**, which is the amortized argument of Topic 115 measured rather than read. ASan must report zero leaks.

HANDNOTE SUMMARY CARD — Advanced & Interview

ADVANCED C++, SYSTEMS & INTERVIEW (Topics 154-177)

STL ON RAW ARRAYS - a pointer IS an iterator

find count accumulate(...,0LL) fill copy iota max_element partial_sum
rotate nth_element($O(n)$) all_of/any_of/none_of lower_bound/upper_bound
half-open [first,last): size=last-first, empty=first==last, no +-1 anywhere
ranges: std::ranges::sort(v); v | views::filter | views::transform (LAZY,
0 allocations, 1 pass) - but a view over a temporary DANGLES.

COMPILE TIME AND BITS

template f(T(&)[N]) deduces N; pointers cannot
constexpr std::array + static_assert table built by the compiler
bit array word = $i >> 6$, bit = $i \& 63$ n/8 bytes, 8x less than bool[]
bitset reach |= reach << w 64x on boolean DP
vector is NOT a container of bool: no data(), no &v[0], proxy refs

SYSTEMS-LEVEL ARRAYS

mmap file as an array; major fault ~100 us on SSD = ~1e5 x an L1 hit
parallel contiguous chunks + PRIVATE accumulators; false sharing else
memory-bound scan: 4 cores -> ~1.1x, the controller is the limit
GPU $i = \text{blockIdx} * \text{blockDim} + \text{threadIdx}$; COALESCING = the cache-line
argument with 32 requesters; stride 16 costs 16x the traffic
external count BLOCK TRANSFERS: 100 GB with 8 GB RAM = 13 runs, 2 passes

THE INTERVIEW SEQUENCE - in this order, out loud

- 1 restate the guarantee sorted? distinct? value range? may I modify it?
- 2 brute force + its cost "all pairs, $O(n^2)$ " - a baseline you can fall back on
- 3 NAME the pattern before writing any code
- 4 invariant as a comment then the loop
- 5 trace the test matrix $n=0$, $n=1$, all equal, sorted, reversed, absent,
duplicates, overflow at $1e9$

CONSTRAINT -> COMPLEXITY (contest, ~1e8 ops/s)

$n \leq 10$ $O(n!)$ | $n \leq 20$ $O(2^n)$ | $n \leq 500$ $O(n^3)$ | $n \leq 5e3$ $O(n^2)$
 $n \leq 1e6$ $O(n \log n)$ | $n \leq 1e8$ $O(n)$ values $\leq 1e9$ and $n \geq 1e5$ -> long long

CP TOOLKIT, IN ORDER OF VALUE

ios::sync_with_stdio(false); cin.tie(nullptr); 3-10x on I/O
global arrays (no 8 MB stack limit, pre-zeroed)
long long for every prefix sum and accumulator
bitset for boolean DP (64x) counting sort when values are bounded
difference array for range updates two pointers instead of n^2

THE TEN LINES (Topic 176)

base + $i * \text{sizeof}(T)$ | $a[i] == i[a]$ | moving elements is $O(n)$ | 16 ints per line
doubling -> amortized $O(1)$ | sorted -> binary search/two pointers
bounded values -> counting | contiguous -> window/prefix (NOT subsequence)
long long + $\text{lo} + (\text{hi} - \text{lo}) / 2$ | test $n=0$ first

PART VII CHEATSHEET — Beyond Ordinary Memory

PART VII: WHEN THE ARRAY STOPS BEING PLAIN RAM

THE LADDER OF STORAGE, AND WHAT CHANGES AT EACH RUNG

register/L1	~4 cycles	the compiler's job; keep working sets small
DRAM	~200 cycles	the array's home; 64 B lines, ~8 GB/s
packed bits	8x denser	shift+mask per access; sieves, DP, bitmaps
mmap'd file	~100 us fault	sequential fine, random catastrophic
GPU memory	~500 GB/s+	coalescing mandatory; PCIe copy must be repaid
disk / external	ms	count block transfers, not comparisons

ONE IDEA PER TOOL

span	carries the size across a function boundary	(fixes decay)
ranges	composes passes without materialising them	(but views dangle)
template	captures N at compile time	T (&)[N]
constexpr	moves the table into the binary	static_assert it
bitset	turns 64 boolean states into one word op	
mmap	lets the kernel do the I/O scheduling	
execution::par	parallel only helps if you were NOT bandwidth-bound	

THE FIVE SENTENCES THAT END AN INTERVIEW WELL

"It is $O(n)$ time and $O(1)$ space, and here is the invariant."
"This is a sliding window because the quantity is monotone in the window."
"Sorted changes the answer - two pointers instead of a hash set."
"This overflows int at $n = 1e5$ with values $1e9$, so long long."
"An array is the wrong structure here, because inserts are interleaved."

CROSS-REFERENCES

Topic 17/32	decay is the disease; span is the cure
Topic 117	cache lines - the same argument reappears as GPU coalescing
Topic 139	lookup tables; constexpr does not exempt them from cache limits
Topic 159/160	bit packing; the 64x constant that decides contests
Topic 174	the pattern table - the page to read before any interview
Topic 177	when to walk away from arrays entirely

END MATTER

Master Cheatsheet and Tracker

All 177 topics on four cards, the pattern-recognition table, and the spaced-repetition schedule.
If you revise from nothing else, revise from these pages.

MASTER CHEATSHEET 1 — THE MODEL AND THE MEMORY

EVERYTHING ABOUT ARRAYS - CARD 1 of 4

Topics 1-37, 90-101, 113-122

THE THREE QUESTIONS TO ASK OF ANY ARRAY PROBLEM

1. What is guaranteed? sorted / distinct / value range / may I modify it
2. What moves? anything that shifts elements is $O(n)$. Everything else about arrays is $O(1)$.
3. What is n , and where does the data live? L1 32 KB | L2 ~1 MB | DRAM ~200 cyc

THE FORMULA AND ITS CONSEQUENCES

address($a[i]$) = base + $i * \text{sizeof}(T)$ $a[i] == *(a+i) == i[a]$
same-size elements + contiguous -> the multiply is valid
 $O(1)$ access | $O(n)$ insert/delete | fixed size | NO bounds check
& $a[n]$ legal to form, UB to dereference. & $a[n+1]$ and $a-1$ are UB to form.

MEMORY

static/.bss 0 ns, zeroed, program lifetime BIG arrays go here (CP: global)
stack ~1 ns, garbage, ~8 MB total -> max ~ $2e6$ int / ~ $9e5$ double
heap ~50-100 ns, garbage, until delete[] / destructor
size() = objects that exist | capacity() = bytes owned | [size, capacity) is UB

C++ DECLARATION FORMS

int $a[5] = \{\}$ zeroed int $a[5];$ garbage at block scope
std::array 0 overhead, .size() survives function calls
std::vector $v(n)$ heap, zeroed, growable, 24-byte object
std::span s 16 bytes, non-owning view - USE FOR PARAMETERS
new int $[n]$ / delete[] manual; new int $[n]()$ zeroes; unique_ptr is safer
DECAY: passing an array gives a pointer. sizeof and std::size then LIE.
void $f(\text{int } a[100])$ is void $f(\text{int}^*)$. Pass a span or (ptr, size).

2-D IS A FICTION

$m[i][j] == \text{flat}[i * \text{COLS} + j]$ row-major: LAST index varies fastest
int $m[][4]$ legal param; int $m[3][]$ is not - the stride is part of the type
vector>: R+1 allocations, 2 loads per access. Rectangles -> flat.
measured: row 7.8 ms vs column 172.1 ms @ 4096^2 (22x)
matmul i-k-j 92.1 ms vs i-j-k 438.5 ms @ 512^3 (4.8x)

COST CONSTANTS WORTH MEMORISING

cache line 64 B = 16 int = 8 double L1 hit ~4 cyc, DRAM ~200 cyc
 $1e7$ int = 40 MB = 625,000 lines single-core bandwidth ~8 GB/s
array scan $1e7 = 5.6$ ms | scattered list = 1346.6 ms (240x)
vector doubling: 21 reallocations to $1e6$; reserve saved 2.2x at $1e7$
.at() vs [] on a 50M sum: 25.8 vs 25.2 ms (2%) - bounds checks are cheap
SIMD: 4.0x on L1-resident floats, 1.28x when DRAM-bound
padding: {char,int,char}=12 B, {int,char,char}=8 B -> 40 MB per $1e7$ elements
false sharing: 2 threads writing within 64 B -> up to 20x, no race, no warning

MASTER CHEATSHEET 2 — THE OPERATIONS

EVERYTHING ABOUT ARRAYS - CARD 2 of 4

Topics 38-89

COMPLEXITY TABLE

access/update	$O(1)$	append (capacity)	$O(1)$
append (full)	$O(n)$ once, $O(1)$ amortized	insert/delete at p	$O(n-p)$
delete unordered	$O(1)$ $a[p] = a[--len]$	linear search	$O(n)$
binary search	$O(\log n)$ 20 steps @1e6, 30 @1e9 - requires SORTED		
comparison sort	$O(n \log n)$	counting sort	$O(n+K)$
reverse/rotate	$O(n)$, $O(1)$ space	radix	$O(d*(n+b))$
		merge sorted	$O(n+m)$

TEMPLATES TO MEMORISE

```
insert for(i=len;i>pos;--i) a[i]=a[i-1]; a[pos]=v; ++len;
delete for(i=pos;i+1?--r:done); }
kadane cur=max(a[i],cur+a[i]); best=max(best,cur); init best=a[0]
prefix p[0]=0; p[i+1]=p[i]+a[i]; sum(l,r)=p[r+1]-p[l] LONG LONG
diff d[l]+=v; d[r+1]-=v; then prefix-sum d to recover the array
rotate reverse(0,k-1); reverse(k,n-1); reverse(0,n-1); k %= n FIRST
dnf while(m<=h){ 0:swap(l++,m++) 1:m++ 2:swap(m,h--) } do NOT ++m on 2
```

SORTING - choose by the guarantee, not by habit

std::sort $O(n \log n)$ introsort, UNSTABLE, 2-3x faster than hand-written
std::stable_sort $O(n \log n)$ with an $O(n)$ buffer; preserves equal order
partial_sort / nth_element top k / k-th element in $O(n \log k)$ / $O(n)$
counting values in $0..K$, $K \sim n$ radix fixed-width integers
insertion best for $n < \sim 32$ and nearly-sorted data (it is introsort's base case)
comparator MUST be a strict weak ordering: use $<$, never $<=$, or you get UB

PATTERN LIBRARY, ONE LINE EACH

prefix sum	range sums, many queries	suffix sum	from the right
difference	many range updates, one readout	Kadane	max subarray
sliding window	contiguous + monotone quantity	two pointers	sorted pairs
fast/slow	cycle, middle, duplicate	Boyer-Moore	majority $>n/2$
XOR/sum trick	missing number	next_perm	scan, swap, rev
monotonic stack	next greater/smaller	monotonic deque	window max

MASTER CHEATSHEET 3 — TRAPS AND DIAGNOSIS

EVERYTHING ABOUT ARRAYS - CARD 3 of 4

Topics 102-136, 154-167

SYMPTOM -> CAUSE

crash on RETURN from a function	stack buffer overflow	127
crash far from the offending code	UB; run ASan before reading code	124
different answer every run	uninitialised read	129
correct at -O0, wrong at -O2	UB the optimiser exploited	124
works in tests, fails under load	use-after-free / realloc'd ptr	115,128
hangs or crashes only when n == 0	size_t underflow	133
negative total on large input	int overflow (accumulate, mid)	130
RSS climbs forever	leak on an early-return path	126
correct but SLOWER with more threads	false sharing	120
clean median, terrible p99.9	push_back reallocation spike	115
only the first 32 of 128 zeroed	memset byte-vs-element count	135
wrong answers only at grid edges	bounds test placed after the read	109
spiral emits more than R*C	missing singleton row/col guard	105
transpose returns the input	inner loop started at j = 0	103
loop processes exactly 2 elements	sizeof on a decayed pointer	17
template works for every type but bool	vector proxy references	161
found for some targets, not others	wrong search for the guarantee	106

THE UNIVERSAL TEST MATRIX - run it on everything you write

n = 0 | n = 1 | n = 2 | all elements equal | already sorted | reverse sorted
target at index 0 | target at index n-1 | target absent | duplicates present
negative values | values near INT_MAX | n large enough to exceed L2 (>256 KB)

BUILD LINES

```
g++ -std=c++20 -O2 -Wall -Wextra -Wconversion -Wsign-compare
g++ -g -fsanitize=address,undefined          every break-it exercise
g++ -D_GLIBCXX_ASSERTIONS                     catches v[i] past size()
g++ -O2 -fopt-info-vec-missed                 why the loop did not vectorise
perf stat -e instructions,cache-misses        proves memory vs compute
```

RULES THAT PREVENT MOST OF THE TABLE ABOVE

vector/array/span by default; raw arrays only for globals and C interop
range-for or instead of index loops
long long for every accumulator; 0LL in accumulate; lo + (hi-lo)/2
reverse loops: for (size_t i = n; i-- > 0;)
reserve() when the size is known; never hold a pointer across push_back
sizeof arr, never a literal byte count; constexpr for every size
bounds check BEFORE the index is used, and rely on && short-circuiting

MASTER CHEATSHEET 4 — PATTERN RECOGNITION

EVERYTHING ABOUT ARRAYS - CARD 4 of 4

Topics 168-177 + all patterns

THE PROBLEM SAYS...	-> REACH FOR	TOPIC
"contiguous subarray"	sliding window / Kadane / prefix	60,63,64
"subsequence" (not contiguous)	DP or sorting - NOT a window	71
"sum over a range, many queries"	prefix sums (long long)	60
"add v to every element in [l,r]"	difference array	62
"maximum sum subarray"	Kadane	63
"longest/shortest window such that"	variable window (monotone only)	64
"sorted" + "pair/triplet summing to"	two pointers from both ends	65
"sorted" + "find / insert position"	binary search, lower_bound	42
"minimise the maximum" / vice versa	binary search on the ANSWER	42
"in place" / "O(1) extra space"	two pointers, reversal, swapping	44,57,59
"rotate by k"	k % n, then three reversals	56,57
"k-th smallest/largest"	nth_element, or a heap of size k	79,153
"appears more than n/2 times"	Boyer-Moore vote, then VERIFY	67
"values are 0..K with K small"	counting / frequency array	49,81
"find the cycle" / "find the middle"	fast and slow pointers	66
"find the duplicate, read-only input"	Floyd cycle detection	69
"max/min of every window of size k"	monotonic deque	64
"next greater / previous smaller"	monotonic stack	149
"intervals" / "meetings" / "merge"	sort by start, then sweep	84
"0s, 1s and 2s" / three-way partition	Dutch national flag	59
"grid" / "island" / "region" / "fill"	dx[]/dy[] offsets + BFS or DFS	109,110
"rectangle sum in a matrix"	2-D prefix sums (padded by 1)	111
"rotate the matrix in place"	transpose + reverse each row	103,104
"spiral" / "layer by layer"	four closing boundaries + guards	105
"mostly zeros" / "sparse"	CSR: val[], col[], rowPtr[]	112
"too large for memory"	mmap or external merge sort	162,165
"count set bits" / "reachable states"	bit array or bitset (64x)	159,160

WHEN THE ARRAY IS THE WRONG ANSWER - say these out loud

interleaved insert/delete in the middle, by index	-> balanced BST / skip list
sparse or non-integer keys	-> hash table
references that must survive growth	-> list / deque
O(1) splice of whole sequences	-> linked list
more than ~90% zeros	-> CSR
larger than RAM	-> mmap / external sort

THE TEN LINES

1 base + i*sizeof(T)	6 sorted -> binary search / two pointers
2 a[i] == *(a+i) == i[a]	7 bounded values -> counting, O(n+K)
3 moving elements is O(n)	8 contiguous -> window/prefix, NOT subseq
4 64-byte line = 16 ints	9 long long, and lo + (hi-lo)/2
5 doubling -> amortized O(1)	10 test n=0, n=1, all-equal first

REVISION TRACKER

Fill this in by hand as you go. Write the date in *First* on the day you read the section, then schedule the four reviews from that date: **+1 day**, **+7 days**, **+30 days** and **+90 days**. Reviews are done *from the summary card only* — if the card is not enough, mark the row and re-read the topic, do not re-read the Part. *Practice* is the date you completed that section's practice set including its build-break-fix tasks.

Topic group	First	Rev 1 +1d	Practice	Rev 2 +7d	Rev 3 +30d	Rev 4 +90d
1. Foundation (1-5)						
2. Memory & internals (6-9)						
3. C++ basics (10-15)						
4. Pointers & functions (16-23)						
5. Dynamic memory (24-29)						
6. Modern containers (30-34)						
7. Strings (35-37)						
8. Basic operations (38-51)						
9. Rotation (52-59)						
10. Patterns (60-73)						
11. Sorting (74-89)						
12. 2-D & matrices (90-112)						
13. Complexity & cache (113-122)						
14. Bugs & safety (123-136)						
15. Real-world uses (137-147)						
16. Comparisons (148-153)						
17. Advanced C++ (154-167)						
18. Interview, DSA & CP (168-176)						
19. Final (177)						

Self-test gates

Do not mark a Part complete until you can, from memory and without notes: state the address formula and one consequence of it; write the binary search and the sliding-window templates correctly at the first attempt; name the trap for every topic in that Part; and quote one measured number from its cost strips. If you cannot do the last one, you read the Part but did not run anything — Group C of the practice sets is where the numbers come from.

AFTERWORD

This book made one claim 177 times: an array is a rule for computing an address, and every property it has — the $O(1)$ access you rely on, the $O(n)$ insert you work around, the cache behaviour that makes it $240\times$ faster than a linked list, the missing bounds check that produces half the bug table — is a consequence of that one line of arithmetic.

The measurements in these pages were taken on the machine that built the book, with GCC at `-O2`: 5.6 ms versus 1346.6 ms for the same 10^7 integers; 7.8 ms versus 172.1 ms for the same 64 MB matrix; 438.5 ms versus 92.1 ms for the same 2.512^3 floating-point operations; 50.4 ms versus 199.8 ms for the same sum with and without SIMD. Your numbers will differ, and they should: run them. A book that tells you an array is "cache-friendly" has told you nothing you can defend in an interview or a design review. A number you measured yourself, on hardware you can name, is an argument.

The three habits worth carrying out of this volume are smaller than the book that taught them. Ask what is guaranteed before choosing an algorithm — sorted, distinct, bounded, mutable. Ask what moves, because everything that moves elements is $O(n)$ and everything else is $O(1)$. And when the structure fits badly, say so out loud: interleaved middle insertions, sparse keys, references that must survive growth. Knowing when to abandon the array is the part of this subject that separates the two kinds of engineer.

Next in Phase 1: strings, which are char arrays with a terminator; then stacks and queues, which are arrays with two indices; then hashing, which is an array plus a function; then heaps, which are arrays plus index arithmetic. You have already met all four. Day 0 is today — put the date in the tracker.

